附录 A　支撑材料文件清单

源代码 模型与算法的完整可运行源程序	
文件	**说明**
core.py	实际使用的核心算法源程序文件
correlated_bearing.py	实际使用的核心算法源程序文件
discovery.py	实际使用的核心算法源程序文件
empirical_discovery.py	实际使用的核心算法源程序文件
flexroute.py	实际使用的核心算法源程序文件
policy.py	实际使用的核心算法源程序文件
requirements.txt	运行所需的第三方依赖清单（NumPy、SciPy）
routing.py	实际使用的核心算法源程序文件
visibility_probe.py	实际使用的核心算法源程序文件

结果 正式测试结果表与结果汇总	
文件	**说明**
正式测试结果.csv	正式测试结果表与结果汇总
正式测试结果.json	正式测试结果表与结果汇总
正式测试结果.md	正式测试结果表与结果汇总
正式测试结果.xlsx	正式测试结果表与结果汇总

图表 论文图表、图表源数据和索引	
文件	**说明**
03_图表/图片与表格总览.pdf	论文全部图件与表格的 22 页总览
03_图表/图表清单.md	论文 14 幅图、8 张表的清单和页码对应

AI 说明 按官方要求提交的 AI 工具使用详情	
文件	**说明**
AI工具使用详情.pdf	AI 工具版本、使用环节、主要提示方式、采纳修改与人工核验情况

说明 运行环境、复现步骤和支撑材料清单	
文件	**说明**
支撑材料清单.md	支撑材料目录、文件说明和最终 RAR 对应清单（已去除本机路径）
运行环境与复现说明.pdf	28 文件运行闭包的运行环境、主程序入口、通信接口与最小复现步骤

证据 核心实验证据、失败汇总和索引	
文件	**说明**
失败案例汇总.md	核心实验证据与汇总

证据（续） 核心实验证据、失败汇总和索引	
文件	**说明**
官方演练汇总.md	核心实验证据与汇总
正式测试证据/绑定信息.json	核心实验证据与汇总
正式测试证据/记录/q3-run1/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q3-run1/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q3-run1/结果.json	正式单场测试结果记录
正式测试证据/记录/q3-run2/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q3-run2/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q3-run2/结果.json	正式单场测试结果记录
正式测试证据/记录/q3-run3/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q3-run3/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q3-run3/结果.json	正式单场测试结果记录
正式测试证据/记录/q4-run1/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q4-run1/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q4-run1/结果.json	正式单场测试结果记录
正式测试证据/记录/q4-run2/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q4-run2/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q4-run2/结果.json	正式单场测试结果记录
正式测试证据/记录/q4-run3/上传确认.json	正式单场测试上传确认记录
正式测试证据/记录/q4-run3/源码绑定.json	正式单场测试源码绑定记录
正式测试证据/记录/q4-run3/结果.json	正式单场测试结果记录
消融实验汇总.json	核心实验证据与汇总
行为等价性证据.json	核心实验证据与汇总

附录 B　程序源代码

q1.py

```
from __future__ import annotations

import argparse
import hashlib
import itertools
import json
from pathlib import Path

import numpy as np
from scipy.optimize import linprog
from scipy.spatial import HalfspaceIntersection, QhullError

from q1_geometry import minimum_enclosing_disk, polygon_diameter

def cross(a, b):
    return float(a[0] * b[1] - a[1] * b[0])

def bearing_halfplanes(positions, bearings_deg, error_deg):
    positions = np.asarray(positions, dtype=float)
    bearings = np.asarray(bearings_deg, dtype=float)
    if (positions.ndim != 2 or positions.shape[1] != 2
            or len(positions) == 0 or bearings.shape != (len(positions),)
            or not np.all(np.isfinite(positions))
            or not np.all(np.isfinite(bearings))):
        raise ValueError("Require finite positions (n,2) and bearings (n,), n >= 1")
    if not np.isfinite(error_deg) or not 0 < error_deg < 90:
        raise ValueError("error_deg must lie strictly between 0 and 90")
    angles = np.deg2rad(bearings % 360)
    lower, upper = angles - np.deg2rad(error_deg), angles + np.deg2rad(error_deg)
    # cross(下射线, x-S) >= 0; cross(上射线, x-S) <= 0。
    a = np.empty((2 * len(positions), 2))
    a[0::2] = np.column_stack((np.sin(lower), -np.cos(lower)))
    a[1::2] = np.column_stack((-np.sin(upper), np.cos(upper)))
    b = np.einsum("ij,ij->i", a, np.repeat(positions, 2, axis=0))
    return a, b

def convex_hull(points, tolerance):
    unique = [np.asarray(p) for p in sorted(set(map(tuple, points)))]
    if len(unique) <= 2:
        return np.asarray(unique)

    def chain(seq):
        out = []
        for p in seq:
            while len(out) >= 2 and cross(out[-1] - out[-2], p - out[-1]) <= 0:
                out.pop()
            out.append(p)
        return out

    hull = chain(unique)[:-1] + chain(unique[::-1])[:-1]
    cleaned = []
    for p in hull:
        if not cleaned or np.linalg.norm(p - cleaned[-1]) > tolerance:
            cleaned.append(p)
    if len(cleaned) > 1 and np.linalg.norm(cleaned[0] - cleaned[-1]) <= tolerance:
        cleaned.pop()
```

```
    return np.asarray(cleaned)

def solve(positions, bearings_deg, error_deg, tolerance=1e-9, parallel_tol=1e-12,
          *, intersection_method="auto", clearance_radius_m=20.0, mec_seed=20260910):
    if not 1e-10 <= tolerance <= 1e-4 or not 0 < parallel_tol < 1:
        raise ValueError("Invalid numerical tolerances")
    if intersection_method not in ("auto", "enumeration"):
        raise ValueError("intersection_method must be auto or enumeration")
    if not np.isfinite(clearance_radius_m) or clearance_radius_m < 0:
        raise ValueError("clearance_radius_m must be finite and nonnegative")
    positions = np.asarray(positions, dtype=float)
    # 先校验再平移，尤其要避免对空数组求均值。
    bearing_halfplanes(positions, bearings_deg, error_deg)
    origin = positions.mean(axis=0)
    scale = max(1.0, float(np.max(np.linalg.norm(positions - origin, axis=1))))
    local_positions = (positions - origin) / scale
    a, b = bearing_halfplanes(local_positions, bearings_deg, error_deg)
    options = {"primal_feasibility_tolerance": tolerance,
               "dual_feasibility_tolerance": tolerance}

    def lp(objective):
        result = linprog(objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2,
                         method="highs", options=options)
        if result.status not in (0, 2, 3):
            raise ArithmeticError(f"LP failed: {result.message}")
        return result

    result = {
        "geometry": "intersection_of_forward_angular_wedges",
        "bearing_error_deg": float(error_deg),
        "length_tolerance_m": float(tolerance * scale),
        "formal_interval_certificate": False,
        "intersection_method_requested": intersection_method,
        "clearance_radius_m": float(clearance_radius_m),
        "minimum_enclosing_radius_m": None,
        "minimum_enclosing_center_m": None,
        "clearance_disk_covers": None,
        "clearance_decision": "not_applicable",
    }
    feasibility = lp([0, 0])
    if feasibility.status == 2:
        return dict(result, status="empty", diameter_m=None,
                    same_diameter_disk_covers=None, vertices_m=[])
    directions = np.asarray([[1, 0], [-1, 0], [0, 1], [0, -1]])
    support_values = []
    for direction in directions:
        bound = lp(direction)
        if bound.status == 3:
            return dict(result, status="unbounded", diameter_m=None,
                        diameter_is_infinite=True,
                        same_diameter_disk_covers=None, vertices_m=[])
        if bound.status != 0:
            raise ArithmeticError("Inconsistent LP feasibility decisions")
        support_values.append(bound.fun)

    def audit(vertices):
        if len(vertices) == 0 or not np.all(np.isfinite(vertices)):
            raise ArithmeticError("Unresolved intersection vertices")
        residual = float(np.max(a @ vertices.T - b[:, None]))
        support_error = float(np.max(np.abs(np.min(vertices @ directions.T, axis=0)
                                            - support_values)))
```

```
        if residual > 10 * tolerance or support_error > 10 * tolerance:
            raise ArithmeticError("Vertices disagree with halfplanes or LP support")
        return residual, support_error

    vertices, fallback_reason = None, None
    skipped_pairs = 0
    if intersection_method == "auto":
        # 单位法向量: 最大化内切圆半径以寻找严格内点;
        # 不使用真实源位置或人工包围盒。
        interior = linprog([0, 0, -1], A_ub=np.column_stack((a, np.ones(len(a)))),
                           b_ub=b, bounds=[(None, None)] * 2 + [(0, None)],
                           method="highs", options=options)
        if interior.status == 0 and interior.x[2] > 32 * tolerance:
            try:
                points = HalfspaceIntersection(np.column_stack((a, -b)),
                                               interior.x[:2]).intersections
                vertices = convex_hull(points, tolerance)
                audit(vertices)
            except (QhullError, ArithmeticError, FloatingPointError) as exc:
                vertices = None
                fallback_reason = type(exc).__name__ + ": fast intersection rejected"
        else:
            fallback_reason = "no_resolved_strict_interior"
    if vertices is None:
        candidates = []
        for i, j in itertools.combinations(range(len(a)), 2):
            matrix = a[[i, j]]
            if abs(cross(matrix[0], matrix[1])) <= parallel_tol:
                skipped_pairs += 1
                continue
            point = np.linalg.solve(matrix, b[[i, j]])
            if np.max(a @ point - b) <= tolerance:
                candidates.append(point)
        if not candidates:
            raise ArithmeticError("Bounded LP but no resolved vertices; inspect conditioning")
        vertices = convex_hull(candidates, tolerance)
        used_method = "enumeration"
    else:
        used_method = "qhull"
    residual, max_support_error = audit(vertices)
    diameter, pair = polygon_diameter(vertices)
    midpoint = (vertices[pair[0]] + vertices[pair[1]]) / 2
    distances = np.linalg.norm(vertices - midpoint, axis=1)
    farthest = int(np.argmax(distances))
    excess = float(distances[farthest] - diameter / 2)
    world_vertices = vertices * scale + origin
    disk = minimum_enclosing_disk(vertices, tolerance=tolerance, seed=mec_seed)
    radius, lower = disk["radius"] * scale, disk["lower_bound"] * scale
    length_tolerance = tolerance * scale
    if radius < clearance_radius_m - length_tolerance:
        clearance_decision = "covers_with_margin"
        clearance_covers = True
    elif lower > clearance_radius_m + length_tolerance:
        clearance_decision = "does_not_cover"
        clearance_covers = False
    else:
        clearance_decision = "boundary_requires_review"
        clearance_covers = None
    return dict(
        result,
        status={1: "point", 2: "segment"}.get(len(vertices), "polygon"),
```

```
        vertices_m=world_vertices.tolist(),
        diameter_m=diameter * scale,
        diameter_endpoint_indices=list(pair),
        diameter_endpoints_m=world_vertices[list(pair)].tolist(),
        diameter_disk_center_m=(midpoint * scale + origin).tolist(),
        diameter_disk_radius_m=diameter * scale / 2,
        same_diameter_disk_covers=bool(excess <= tolerance),
        coverage_decision="within_numerical_tolerance" if excess <= tolerance else "does_not_cover
  ",
        maximum_distance_from_diameter_midpoint_m=float(max(distances) * scale),
        maximum_radius_excess_m=excess * scale,
        farthest_vertex_index=farthest,
        max_halfplane_residual_m=residual * scale,
        max_lp_support_error_m=max_support_error * scale,
        parallel_or_near_parallel_pairs_skipped=skipped_pairs,
        intersection_method_used=used_method,
        intersection_fallback_reason=fallback_reason,
        diameter_method="rotating_calipers",
        minimum_enclosing_method="welzl_with_support_audit",
        minimum_enclosing_seed=int(mec_seed),
        minimum_enclosing_center_m=(disk["center"] * scale + origin).tolist(),
        minimum_enclosing_radius_m=radius,
        minimum_enclosing_radius_lower_bound_m=lower,
        minimum_enclosing_optimality_gap_m=disk["optimality_gap"] * scale,
        minimum_enclosing_support_indices=disk["support_indices"],
        minimum_enclosing_support_weights=disk["support_weights"],
        clearance_disk_covers=clearance_covers,
        clearance_decision=clearance_decision,
    )

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("input", type=Path, help="JSON with positions_m and bearings_deg")
    parser.add_argument("--config", type=Path,
                        default=Path(__file__).resolve().parents[1] / "configs/q1.json")
    parser.add_argument("--output", type=Path)
    parser.add_argument("--method", choices=("auto", "enumeration"))
    args = parser.parse_args()
    data = json.loads(args.input.read_text(encoding="utf-8"))
    config = json.loads(args.config.read_text(encoding="utf-8"))
    result = solve(data["positions_m"], data["bearings_deg"],
                   config["bearing_error_deg"],
                   config["normalized_feasibility_tolerance"],
                   config["parallel_determinant_tolerance"],
                   intersection_method=args.method or config.get("intersection_method", "auto"),
                   clearance_radius_m=config.get("clearance_radius_m", 20.0),
                   mec_seed=config.get("minimum_enclosing_seed", 20260910))
    def text_hash(path):
        return hashlib.sha256(path.read_bytes().replace(b"\r\n", b"\n")).hexdigest()

    result["provenance"] = {
        "hash_scheme": "sha256_text_lf",
        "input_sha256": text_hash(args.input),
        "config_sha256": text_hash(args.config),
        "solver_sha256": text_hash(Path(__file__)),
        "geometry_sha256": text_hash(Path(__file__).with_name("q1_geometry.py")),
        "input_description": data.get("description", "user-supplied bearings"),
    }
    encoded = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
```

```
        args.output.write_text(encoded, encoding="utf-8")
    else:
        print(encoded, end="")

if __name__ == "__main__":
    main()
```

q1_geometry.py

```
import itertools
import numpy as np

def cross(a, b):
    return float(a[0] * b[1] - a[1] * b[0])

def polygon_diameter(vertices):
    v = np.asarray(vertices, dtype=float)
    n = len(v)
    if n == 0:
        raise ValueError("Diameter of empty polygon is undefined")
    if n <= 2:
        return float(np.linalg.norm(v[-1] - v[0])), (0, n - 1)
    best, pair, j = -1.0, (0, 0), 1
    for i in range(n):
        ni = (i + 1) % n
        edge = v[ni] - v[i]
        steps = 0
        while cross(edge, v[(j + 1) % n] - v[i]) > cross(edge, v[j] - v[i]):
            j = (j + 1) % n
            steps += 1
            if steps > n:
                raise ArithmeticError("Invalid convex hull in rotating calipers")
        # 检查下一个端点也可覆盖并列的平行支撑边。
        for a, b in ((i, j), (ni, j), (i, (j + 1) % n), (ni, (j + 1) % n)):
            delta = v[a] - v[b]
            squared = float(delta @ delta)
            candidate = tuple(sorted((a, b)))
            if squared > best or (squared == best and candidate < pair):
                best, pair = squared, candidate
    return float(np.sqrt(best)), pair

def minimum_enclosing_disk(points, tolerance=1e-9, seed=20260910):
    original = np.asarray(points, dtype=float)
    if (original.ndim != 2 or original.shape[1] != 2 or not len(original)
            or not np.all(np.isfinite(original)) or not np.isfinite(tolerance)
            or tolerance <= 0):
        raise ValueError("Require finite nonempty (k,2) points and positive tolerance")
    # 额外平移可避免在 solve() 之外调用时发生消减。
    origin = original[0]
    scale = max(1.0, float(np.linalg.norm(original - origin, axis=1).max()))
    normalized = (original - origin) / scale
    order = np.random.default_rng(seed).permutation(len(original))
    v = normalized[order]
    eps = 64 * np.finfo(float).eps

    def contains(disk, p):
        if disk is None:
            return False
        c, r, _ = disk
```

```
        return np.linalg.norm(p - c) <= r + eps * max(1.0, r)

    def boundary_disk(indices):
        if not indices:
            return None
        if len(indices) == 1:
            return v[indices[0]].copy(), 0.0, indices
        candidates = []
        for i, j in itertools.combinations(indices, 2):
            center = (v[i] + v[j]) / 2
            disk = (center, float(np.linalg.norm(v[i] - center)), (i, j))
            if all(contains(disk, v[t]) for t in indices):
                candidates.append(disk)
        if candidates:
            return min(candidates, key=lambda d: d[1])
        i, j, k = indices
        a, b = v[j] - v[i], v[k] - v[i]
        matrix = 2 * np.asarray([a, b])
        try:
            center = v[i] + np.linalg.solve(matrix, [a @ a, b @ b])
        except np.linalg.LinAlgError as exc:
            raise ArithmeticError("Unresolved collinear disk support") from exc
        return center, float(np.linalg.norm(center - v[i])), indices

    # 栈帧：前缀长度、强制边界索引、返回阶段。
    stack = [(len(v), (), 0)]
    disk = None
    while stack:
        n, boundary, phase = stack.pop()
        if phase == 0:
            if n == 0 or len(boundary) == 3:
                disk = boundary_disk(boundary)
            else:
                stack.append((n, boundary, 1))
                stack.append((n - 1, boundary, 0))
        elif not contains(disk, v[n - 1]):
            stack.append((n - 1, boundary + (n - 1,), 0))

    center, _, support = disk
    support_indices = order[list(support)]
    s = normalized[support_indices]
    # 在局部缩放坐标系中计算权重，而不是世界坐标。
    if len(s) == 1:
        weights = np.ones(1)
    elif len(s) == 2:
        weights = np.array([0.5, 0.5])
    else:
        local_scale = max(float(np.linalg.norm(s - center, axis=1).max()), 1e-300)
        weights = np.linalg.solve(np.vstack(((s - center).T / local_scale,
                                             np.ones(3))), [0, 0, 1])
    if not np.all(np.isfinite(weights)) or np.min(weights) < -1e-8:
        raise ArithmeticError("Disk center lies outside its support hull")
    weights = np.maximum(weights, 0)
    weights /= weights.sum()
    # 对任意 c，max ||s_i-c||² >= 支撑点的加权方差。
    mean = weights @ s
    lower = float(np.sqrt(np.sum(weights * np.sum((s - mean) ** 2, axis=1))))
    upper = float(np.linalg.norm(normalized - center, axis=1).max())
    gap = max(0.0, upper - lower) * scale
    if gap > 10 * tolerance:
        raise ArithmeticError("Enclosing disk failed upper/lower radius audit")
```

```
    return {
        "center": center * scale + origin,
        "radius": upper * scale,
        "lower_bound": lower * scale,
        "optimality_gap": gap,
        "support_indices": support_indices.tolist(),
        "support_weights": weights.tolist(),
    }
```

q1_reference.py

```
from __future__ import annotations

import argparse
import hashlib
import itertools
import json
from pathlib import Path

import numpy as np
from scipy.optimize import linprog

def cross(a, b):
    return float(a[0] * b[1] - a[1] * b[0])

def bearing_halfplanes(positions, bearings_deg, error_deg):
    positions = np.asarray(positions, dtype=float)
    bearings = np.asarray(bearings_deg, dtype=float)
    if (positions.ndim != 2 or positions.shape[1] != 2
            or len(positions) == 0 or bearings.shape != (len(positions),)
            or not np.all(np.isfinite(positions))
            or not np.all(np.isfinite(bearings))):
        raise ValueError("Require finite positions (n,2) and bearings (n,), n >= 1")
    if not np.isfinite(error_deg) or not 0 < error_deg < 90:
        raise ValueError("error_deg must lie strictly between 0 and 90")
    angles = np.deg2rad(bearings % 360)
    lower, upper = angles - np.deg2rad(error_deg), angles + np.deg2rad(error_deg)
    # cross(下射线, x-S) >= 0; cross(上射线, x-S) <= 0。
    a = np.empty((2 * len(positions), 2))
    a[0::2] = np.column_stack((np.sin(lower), -np.cos(lower)))
    a[1::2] = np.column_stack((-np.sin(upper), np.cos(upper)))
    b = np.einsum("ij,ij->i", a, np.repeat(positions, 2, axis=0))
    return a, b

def convex_hull(points, tolerance):
    unique = [np.asarray(p) for p in sorted(set(map(tuple, points)))]
    if len(unique) <= 2:
        return np.asarray(unique)

    def chain(seq):
        out = []
        for p in seq:
            while len(out) >= 2 and cross(out[-1] - out[-2], p - out[-1]) <= 0:
                out.pop()
            out.append(p)
        return out

    hull = chain(unique)[:-1] + chain(unique[::-1])[:-1]
    cleaned = []
```

```
    for p in hull:
        if not cleaned or np.linalg.norm(p - cleaned[-1]) > tolerance:
            cleaned.append(p)
    if len(cleaned) > 1 and np.linalg.norm(cleaned[0] - cleaned[-1]) <= tolerance:
        cleaned.pop()
    return np.asarray(cleaned)

def solve(positions, bearings_deg, error_deg, tolerance=1e-9, parallel_tol=1e-12):
    if not 1e-10 <= tolerance <= 1e-4 or not 0 < parallel_tol < 1:
        raise ValueError("Invalid numerical tolerances")
    positions = np.asarray(positions, dtype=float)
    # 先校验再平移，尤其要避免对空数组求均值。
    bearing_halfplanes(positions, bearings_deg, error_deg)
    origin = positions.mean(axis=0)
    scale = max(1.0, float(np.max(np.linalg.norm(positions - origin, axis=1))))
    local_positions = (positions - origin) / scale
    a, b = bearing_halfplanes(local_positions, bearings_deg, error_deg)
    options = {"primal_feasibility_tolerance": tolerance,
               "dual_feasibility_tolerance": tolerance}

    def lp(objective):
        result = linprog(objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2,
                         method="highs", options=options)
        if result.status not in (0, 2, 3):
            raise ArithmeticError(f"LP failed: {result.message}")
        return result

    result = {
        "geometry": "intersection_of_forward_angular_wedges",
        "bearing_error_deg": float(error_deg),
        "length_tolerance_m": float(tolerance * scale),
        "formal_interval_certificate": False,
    }
    feasibility = lp([0, 0])
    if feasibility.status == 2:
        return dict(result, status="empty", diameter_m=None,
                    same_diameter_disk_covers=None, vertices_m=[])
    for direction in ([1, 0], [-1, 0], [0, 1], [0, -1]):
        bound = lp(direction)
        if bound.status == 3:
            return dict(result, status="unbounded", diameter_m=None,
                        diameter_is_infinite=True,
                        same_diameter_disk_covers=None, vertices_m=[])
        if bound.status != 0:
            raise ArithmeticError("Inconsistent LP feasibility decisions")

    candidates = []
    skipped_pairs = 0
    for i, j in itertools.combinations(range(len(a)), 2):
        matrix = a[[i, j]]
        if abs(cross(matrix[0], matrix[1])) <= parallel_tol:
            skipped_pairs += 1
            continue
        point = np.linalg.solve(matrix, b[[i, j]])
        if np.max(a @ point - b) <= tolerance:
            candidates.append(point)
    if not candidates:
        raise ArithmeticError("Bounded LP but no resolved vertices; inspect conditioning")
    vertices = convex_hull(candidates, tolerance)
    # 独立线性规划支撑值用于防止极端点被静默遗漏。
    max_support_error = 0.0
```

```
    for direction in np.asarray([[1, 0], [-1, 0], [0, 1], [0, -1]]):
        support_error = abs(float(np.min(vertices @ direction)) - lp(direction).fun)
        max_support_error = max(max_support_error, support_error)
    if max_support_error > 10 * tolerance:
        raise ArithmeticError("Enumerated vertices disagree with LP support; ill-conditioned input
")

    pair = (0, 0)
    diameter_squared = 0.0
    for i, j in itertools.combinations(range(len(vertices)), 2):
        squared = float(np.sum((vertices[i] - vertices[j]) ** 2))
        if squared > diameter_squared:
            diameter_squared, pair = squared, (i, j)
    diameter = float(np.sqrt(diameter_squared))
    midpoint = (vertices[pair[0]] + vertices[pair[1]]) / 2
    distances = np.linalg.norm(vertices - midpoint, axis=1)
    farthest = int(np.argmax(distances))
    excess = float(distances[farthest] - diameter / 2)
    world_vertices = vertices * scale + origin
    return dict(
        result,
        status={1: "point", 2: "segment"}.get(len(vertices), "polygon"),
        vertices_m=world_vertices.tolist(),
        diameter_m=diameter * scale,
        diameter_endpoint_indices=list(pair),
        diameter_endpoints_m=world_vertices[list(pair)].tolist(),
        diameter_disk_center_m=(midpoint * scale + origin).tolist(),
        diameter_disk_radius_m=diameter * scale / 2,
        same_diameter_disk_covers=bool(excess <= tolerance),
        coverage_decision="within_numerical_tolerance" if excess <= tolerance else "does_not_cover
",
        maximum_distance_from_diameter_midpoint_m=float(max(distances) * scale),
        maximum_radius_excess_m=excess * scale,
        farthest_vertex_index=farthest,
        max_halfplane_residual_m=float(np.max(a @ vertices.T - b[:, None]) * scale),
        max_lp_support_error_m=max_support_error * scale,
        parallel_or_near_parallel_pairs_skipped=skipped_pairs,
    )

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("input", type=Path, help="JSON with positions_m and bearings_deg")
    parser.add_argument("--config", type=Path,
                        default=Path(__file__).resolve().parents[1] / "configs/q1.json")
    parser.add_argument("--output", type=Path)
    args = parser.parse_args()
    data = json.loads(args.input.read_text(encoding="utf-8"))
    config = json.loads(args.config.read_text(encoding="utf-8"))
    result = solve(data["positions_m"], data["bearings_deg"],
                   config["bearing_error_deg"],
                   config["normalized_feasibility_tolerance"],
                   config["parallel_determinant_tolerance"])
    result["provenance"] = {
        "input_sha256": hashlib.sha256(args.input.read_bytes()).hexdigest(),
        "config_sha256": hashlib.sha256(args.config.read_bytes()).hexdigest(),
        "solver_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        "input_description": data.get("description", "user-supplied bearings"),
    }
    encoded = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
```

```
        args.output.write_text(encoded, encoding="utf-8")
    else:
        print(encoded, end="")

if __name__ == "__main__":
    main()
```

q2.py

```
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

import numpy as np
from scipy.optimize import differential_evolution

import q1
from q2_geometry import evaluate, prior_cells, reading_interval, reception_margins, reflect_cells

def digest(path):
    return hashlib.sha256(Path(path).read_bytes().replace(b'\r\n', b'\n')).hexdigest()

def orient_audit(audit, side):
    result = dict(audit)
    if side == -1:
        result['worst_second_bearing_relative_deg'] *= -1
        result['second_bearing_interval_relative_deg'] = [-v for v in result['
   second_bearing_interval_relative_deg'][::-1]]
    return result

def validate_config(config, error):
    for key in ('minimum_reception_radius_m', 'maximum_reception_radius_m',
                'target_region_radius_m', 'near_radius_m', 'walking_speed_m_s',
                'geometry_margin_m', 'measurement_time_s'):
        if not np.isfinite(config[key]) or config[key] <= 0:
            raise ValueError('Require positive finite '+key)
    if not (0 < error < 10 and 2*error < config['minimum_measured_crossing_angle_deg'] < 45
            and config['near_radius_m'] < config['minimum_reception_radius_m']
            <= config['maximum_reception_radius_m']):
        raise ValueError('Unsupported physical or angle configuration')
    for key, minimum in (('prior_angular_cells', 1), ('training_bearing_samples', 3),
                         ('validation_bearing_samples', 3), ('search_population', 5),
                         ('search_iterations', 1)):
        if not isinstance(config[key], int) or isinstance(config[key], bool) or config[key] <
   minimum:
            raise ValueError('Invalid integer setting: '+key)
    budget = config.get('maximum_move_distance_m')
    if budget is not None and (not np.isfinite(budget) or budget <= 0):
        raise ValueError('Move budget must be positive and finite')

def candidate(point, cells, error, config):
    q = np.asarray(point, dtype=float)
    if q.shape != (2,) or not np.all(np.isfinite(q)):
        return None
    margin = config['geometry_margin_m']
    budget = config.get('maximum_move_distance_m')
```

```
    if budget is not None and np.linalg.norm(q) > budget:
        return None
    # 保持 q 在完整第一角域之外；上支/下支显式区分。
    if q[0] < 0 or q[1] <= q[0]*np.tan(np.deg2rad(error))+margin:
        return None
    slack, nearest = reception_margins(q, cells, config['minimum_reception_radius_m'])
    if slack < margin or nearest <= config['near_radius_m']+margin:
        return None
    try:
        bounds = reading_interval(q, cells, error)
    except ValueError:
        return None
    gamma = np.deg2rad(config['minimum_measured_crossing_angle_deg'])
    if bounds[0] < -np.pi+gamma or bounds[1] > -gamma:
        return None
    return {'interval': bounds, 'reception_margin_m': slack, 'nearest_possible_source_m': nearest}

def choose(first_position, first_bearing_deg, geometry_config, config):
    first = np.asarray(first_position, dtype=float)
    if first.shape != (2,) or not np.all(np.isfinite(first)) or not np.isfinite(first_bearing_deg)
  :
        raise ValueError('Require finite first_position_m[2] and first_bearing_deg')
    error = float(geometry_config['bearing_error_deg'])
    validate_config(config, error)
    cells, rotation = prior_cells(first, first_bearing_deg, error, config)
    R0, L = config['minimum_reception_radius_m'], float(cells[:, 3].max())
    # 若可能存在距离为 r 的源，安全 q 必须满足 |q| <= r + max(R0,r)。
    bound = float(np.min(cells[:, 2]+np.maximum(R0, cells[:, 2])))
    budget = config.get('maximum_move_distance_m')
    if budget is not None:
        bound = min(bound, budget)
    if bound <= config['near_radius_m']:
        raise ValueError('No supported candidate under this movement budget')
    finalists, histories, comparisons = [], [], []
    for side in (1, -1):
        branch_cells = reflect_cells(cells, side)
        evaluated = {}

        def objective(x):
            p = tuple(map(float, x))
            if p in evaluated:
                return evaluated[p][0]
            evidence = candidate(p, branch_cells, error, config)
            if evidence is None:
                return 1e8
            metric = evaluate(p, evidence['interval'], error, config['training_bearing_samples'])
            value = metric['sampled_worst_diameter_m']
            evaluated[p] = (value, evidence)
            return value

        # 包含原始固定规则、改进通用规则和短距离种子；
        # 显式种子候选在优化后仍保留资格。
        for scale in (R0, min(R0, 2*L/3), L, bound*.8, bound*.3):
            for xy in ((.8, .6), (.5, .5), (.6, .7), (.7983022, .6022571)):
                objective(np.asarray(xy)*scale)
        old = np.array([.8*R0, .6*R0])
        old_evidence = candidate(old, branch_cells, error, config)
        if old_evidence:
            comparisons.append({'side': side, 'local_position_m': (old*[1, side]).tolist(),
                                **orient_audit(evaluate(old, old_evidence['interval'], error,
                                                        config['validation_bearing_samples'], True
```

```
), side)})
    x0 = min(evaluated, key=lambda p: evaluated[p][0]) if evaluated else None
    opt = differential_evolution(objective, [(0., bound), (config['near_radius_m'], bound)],
                                 seed=config['search_seed'], popsize=config['search_population
'],
                                 maxiter=config['search_iterations'], tol=1e-7,
                                 polish=False, x0=x0)
    # 对入围点进行密集复核，而不是只验证一个粗网格胜者；
    # 始终保留原始固定点作为对照。
    shortlist = sorted(evaluated, key=lambda p: (evaluated[p][0], np.hypot(*p)))[:5]
    if tuple(old) in evaluated and tuple(old) not in shortlist:
        shortlist.append(tuple(old))
    for p in shortlist:
        evidence = evaluated[p][1]
        audit = evaluate(p, evidence['interval'], error, config['validation_bearing_samples'],
 True)
        finalists.append({'point': p, 'side': side, 'audit': audit, 'evidence': evidence})
    histories.append({'side': side, 'feasible_evaluations': len(evaluated),
                      'optimizer_success': bool(opt.success), 'optimizer_message': str(opt.
message),
                      'finalists_validated': len(shortlist)})
 if not finalists:
    raise ValueError('No supported direction-producing candidate; increase move budget or
inspect prior')
 selected = min(finalists, key=lambda row: (row['audit']['sampled_worst_diameter_m'],
                                            np.hypot(*row['point']), -row['side']))
 canonical = np.asarray(selected['point'])
 local = canonical*[1, selected['side']]
 actual_reading = selected['side']*selected['audit']['worst_second_bearing_relative_deg']

 # 包括线性规划和直径一致性检查。
 general = q1.solve([[0, 0], local], [0, actual_reading], error)
 discrepancy = abs(general['diameter_m']-selected['audit']['sampled_worst_diameter_m'])
 if discrepancy > 1e-5:
    raise ArithmeticError('Q1/Q2 diameter disagreement; inspect numerical geometry')
 world = first+rotation @ local
 audit = orient_audit(selected['audit'], selected['side'])
 return {'status': 'selected', 'strategy': 'disk_aware_sector_enclosure_minimax',
         'first_position_m': first.tolist(), 'first_bearing_deg': float(first_bearing_deg %
360),
         'second_position_m': world.tolist(), 'selected_local_m': local.tolist(),
         'selected_side': selected['side'], 'move_distance_m': float(np.linalg.norm(local)),
         'move_and_measure_time_s': float(np.linalg.norm(local)/config['walking_speed_m_s']+
config['measurement_time_s']),
         'maximum_move_distance_m': budget, 'same_channel_switch_time_s': 0.,
         'safe_region_margin_m': selected['evidence']['reception_margin_m'],
         'nearest_possible_source_m': selected['evidence']['nearest_possible_source_m'],
         'prior': {'target_disk_used_to_tighten': True, 'near_radius_used_to_tighten': True,
                   'enclosure': 'union_of_closed_annular_sectors',
                   'cells_rad_rad_m_m': cells.tolist(), 'maximum_range_bound_m': L,
                   'target_region_radius_m': config['target_region_radius_m']},
         'candidate_region': {'definition': 'reception_margins >= margin; nearest > near+margin
; separated upper/lower branch; crossing >= gamma; optional move budget',
                              'minimum_measured_crossing_angle_deg': config['
minimum_measured_crossing_angle_deg'],
                              'is_maximal_feasible_region': False},
         'selection_metric': '
sampled_minimax_of_Q1_angular_polygon_diameter_over_enclosed_readings',
         'audit': audit, 'q1_crosscheck_diameter_m': general['diameter_m'],
         'q1_crosscheck_difference_m': discrepancy, 'search_history': histories,
         'reference_candidates': comparisons, 'global_optimality_claimed': False,
```

```
            'formal_interval_certificate': False,
            'reception_guarantee_basis': 'analytic radial endpoint/kink and angular extrema over
   enclosing cells; explicit floating margin'}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('input', type=Path)
    configs = Path(__file__).resolve().parents[1]/'configs'
    parser.add_argument('--config', type=Path, default=configs/'q2.json')
    parser.add_argument('--geometry-config', type=Path, default=configs/'q1.json')
    parser.add_argument('--max-move', type=float, help='Optional movement budget in meters')
    parser.add_argument('--output', type=Path)
    args = parser.parse_args()
    data = json.loads(args.input.read_text(encoding='utf-8'))
    config = json.loads(args.config.read_text(encoding='utf-8'))
    geometry = json.loads(args.geometry_config.read_text(encoding='utf-8'))
    if args.max_move is not None:
        config['maximum_move_distance_m'] = args.max_move
    result = choose(data['first_position_m'], data['first_bearing_deg'], geometry, config)
    result['effective_config'] = config
    result['input_description'] = data.get('description', 'user-supplied first bearing')
    result['provenance'] = {'hash_scheme': 'sha256_text_lf', 'input_sha256': digest(args.input),
                            'config_sha256': digest(args.config), 'geometry_config_sha256': digest
   (args.geometry_config),
                            'sources': {name: digest(Path(__file__).with_name(name)) for name in
                                        ('q2.py', 'q2_geometry.py', 'q1.py', 'q1_geometry.py')}}
    encoded = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False)+'\n'
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(encoded, encoding='utf-8')
    else:
        print(encoded, end='')

if __name__ == '__main__':
    main()
```

q2_geometry.py

```
from __future__ import annotations

import itertools
import numpy as np
from scipy.optimize import minimize_scalar

def angular_dot_extrema(point, lower, upper):
    a, b = point
    lower, upper = np.asarray(lower), np.asarray(upper)
    ends = np.stack((a*np.cos(lower)+b*np.sin(lower),
                     a*np.cos(upper)+b*np.sin(upper)))
    lo, hi = ends.min(axis=0), ends.max(axis=0)
    phi, radius = np.arctan2(b, a), np.hypot(a, b)
    for shift in (-2*np.pi, 0., 2*np.pi):
        p = phi+shift
        hi = np.where((lower <= p) & (p <= upper), radius, hi)
        p += np.pi
        lo = np.where((lower <= p) & (p <= upper), -radius, lo)
    return lo, hi

def prior_cells(first, bearing_deg, error_deg, config):
```

```
    theta = np.deg2rad(bearing_deg % 360)
    rotation = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
    center = -rotation.T @ np.asarray(first, dtype=float)
    delta = np.deg2rad(error_deg)
    cuts = np.linspace(-delta, delta, config['prior_angular_cells']+1)
    _, hmax = angular_dot_extrema(center, cuts[:-1], cuts[1:])
    H = config['target_region_radius_m']
    C = float(np.dot(center, center)-H*H)
    maximum, near = config['maximum_reception_radius_m'], config['near_radius_m']
    padding = config['geometry_margin_m']
    cells = []
    for lo, hi, h in zip(cuts[:-1], cuts[1:], hmax):
        if C > 0 and (h <= 0 or h*h < C):
            continue
        root = np.sqrt(max(0., h*h-C))
        outer = h+root if h >= 0 else -C/(root-h)
        inner = C/(h+root) if C > 0 else 0.
        rlo, rhi = max(near, inner), min(maximum, outer)
        if rhi <= near or rlo > rhi:
            continue
        cells.append([float(lo), float(hi), max(0., float(rlo-padding)), float(rhi+padding)])
    if not cells:
        raise ValueError('First direction is inconsistent with the target disk and reception range
   ')
    return np.asarray(cells), rotation

def reflect_cells(cells, side):
    if side == 1:
        return cells
    result = cells.copy()
    result[:, 0], result[:, 1] = -cells[:, 1], -cells[:, 0]
    return result

def reception_margins(point, cells, minimum_radius):
    lower, upper, rlo, rhi = cells.T
    dotmin, dotmax = angular_dot_extrema(point, lower, upper)
    radii = np.stack((rlo, rhi, np.clip(minimum_radius, rlo, rhi)), axis=1)
    rho2 = float(np.dot(point, point))
    distances = np.sqrt(np.maximum(0., rho2+radii*radii-2*radii*dotmin[:, None]))
    slack = float(np.min(np.maximum(minimum_radius, radii)-distances))
    closest_r = np.clip(dotmax, rlo, rhi)
    nearest = float(np.sqrt(np.maximum(0., rho2+closest_r**2-2*closest_r*dotmax)).min())
    return slack, nearest

def reading_interval(point, cells, error_deg):
    a, b = point
    lower, upper, rlo, rhi = cells.T
    # 充分分离带使角分支在 (-pi,0) 内保持连续。
    max_y = np.maximum(rlo*np.sin(upper), rhi*np.sin(upper)).max()
    if b <= max_y:
        raise ValueError('Second point must be strictly above the prior enclosure')
    radii = np.repeat(np.stack((rlo, rhi), axis=1), 2, axis=1)
    angles = np.tile(np.stack((lower, upper), axis=1), (1, 2))
    values = list(np.arctan2(radii*np.sin(angles)-b, radii*np.cos(angles)-a).ravel())
    phi, rho = np.arctan2(b, a), np.hypot(a, b)
    for rs in (rlo, rhi):
        psi = np.arccos(np.clip(rs/rho, -1, 1))
        for alphas in (phi-psi, phi+psi):
            ok = (rs <= rho) & (lower <= alphas) & (alphas <= upper)
            values.extend(np.arctan2(rs[ok]*np.sin(alphas[ok])-b,
                                     rs[ok]*np.cos(alphas[ok])-a))
```

```
    delta = np.deg2rad(error_deg)
    return float(min(values)-delta), float(max(values)+delta)

def diameters(point, theta, error_deg):
    theta = np.atleast_1d(theta)
    delta, n = np.deg2rad(error_deg), len(theta)
    A = np.empty((n, 4, 2))
    A[:, 0] = [np.sin(-delta), -np.cos(-delta)]
    A[:, 1] = [-np.sin(delta), np.cos(delta)]
    A[:, 2, 0], A[:, 2, 1] = np.sin(theta-delta), -np.cos(theta-delta)
    A[:, 3, 0], A[:, 3, 1] = -np.sin(theta+delta), np.cos(theta+delta)
    rhs = np.zeros((n, 4))
    rhs[:, 2:] = A[:, 2:] @ np.asarray(point)
    vertices = np.empty((n, 6, 2))
    valid = np.empty((n, 6), dtype=bool)
    tol = 1e-11*max(1., np.linalg.norm(point))
    for k, (i, j) in enumerate(itertools.combinations(range(4), 2)):
        det = A[:, i, 0]*A[:, j, 1]-A[:, i, 1]*A[:, j, 0]
        if np.any(np.abs(det) < 1e-12):
            raise ArithmeticError('Unresolved parallel boundary')
        vertices[:, k, 0] = (rhs[:, i]*A[:, j, 1]-A[:, i, 1]*rhs[:, j])/det
        vertices[:, k, 1] = (A[:, i, 0]*rhs[:, j]-rhs[:, i]*A[:, j, 0])/det
        valid[:, k] = np.max(np.einsum('nij,nj->ni', A, vertices[:, k])-rhs, axis=1) <= tol
    if np.any(valid.sum(axis=1) < 1):
        raise ArithmeticError('Unresolved two-wedge intersection')
    result = np.zeros(n)
    for i, j in itertools.combinations(range(6), 2):
        ds = np.sum((vertices[:, i]-vertices[:, j])**2, axis=1)
        result = np.maximum(result, np.where(valid[:, i] & valid[:, j], ds, 0.))
    return np.sqrt(result)

def evaluate(point, interval, error_deg, samples=129, refine=False):
    low, high = interval
    delta = np.deg2rad(error_deg)
    origin = np.arctan2(-point[1], -point[0])
    # 即使训练阶段也纳入精确拓扑变化：均匀网格可能遗漏
    # 第二角域开始或停止包含原点时的尖点。
    angles = np.unique(np.r_[np.linspace(low, high, samples),
                             [t for t in (origin-delta, origin+delta) if low < t < high]])
    ds = diameters(point, angles, error_deg)
    k = int(ds.argmax())
    best, reading = float(ds[k]), float(angles[k])
    if refine:
        peaks = np.flatnonzero((ds[1:-1] >= ds[:-2]) & (ds[1:-1] >= ds[2:]))+1
        for i in peaks:
            opt = minimize_scalar(lambda t: -diameters(point, [t], error_deg)[0],
                                  bounds=(angles[i-1], angles[i+1]), method='bounded',
                                  options={'xatol': 1e-12})
            if opt.success and -opt.fun > best:
                best, reading = float(-opt.fun), float(opt.x)
    return {'sampled_worst_diameter_m': best, 'worst_second_bearing_relative_deg': float(np.
   rad2deg(reading)),
            'second_bearing_interval_relative_deg': np.rad2deg(interval).tolist(),
            'bearing_samples': int(samples), 'topology_changes_included': True,
            'local_peak_refinement': bool(refine), 'formal_continuous_certificate': False}
```

q2_reference.py

```
from __future__ import annotations
```

```
import argparse
import hashlib
import itertools
import json
from pathlib import Path

import numpy as np
from scipy.optimize import minimize_scalar

import q1

def safe_disk_centers(error_deg, minimum_radius):
    delta = np.deg2rad(error_deg)
    return np.array([[0., 0.], [minimum_radius * np.cos(delta), minimum_radius * np.sin(delta)],
                     [minimum_radius * np.cos(delta), -minimum_radius * np.sin(delta)]])

def reception_safe(point, error_deg, minimum_radius, tolerance=1e-8):
    point = np.asarray(point, dtype=float)
    return bool(np.all(np.linalg.norm(safe_disk_centers(error_deg, minimum_radius) - point,
                                      axis=1) <= minimum_radius + tolerance))

def bearing_interval(point, error_deg, maximum_radius):
    a, b = point
    delta = np.deg2rad(error_deg)
    if not (0 < a < maximum_radius * np.cos(delta)
            and b > maximum_radius * np.sin(delta)
            and np.hypot(a, b) < maximum_radius):
        raise ValueError("Point must be above the full prior sector and inside radius L")
    return (float(np.arctan2(-b, -a) - delta),
            float(np.arctan2(maximum_radius * np.sin(delta) - b,
                             maximum_radius * np.cos(delta) - a) + delta))

def candidate(point, error_deg, config):
    point = np.asarray(point, dtype=float)
    if point.shape != (2,) or not np.all(np.isfinite(point)):
        return False
    if not reception_safe(point, error_deg, config["minimum_reception_radius_m"]):
        return False
    try:
        low, high = bearing_interval(point, error_deg, config["maximum_reception_radius_m"])
    except ValueError:
        return False
    gamma = np.deg2rad(config["minimum_measured_crossing_angle_deg"])
    return bool(low >= -np.pi + gamma and high <= -gamma)

def two_wedge_diameters(point, second_bearings_rad, error_deg):
    theta = np.atleast_1d(np.asarray(second_bearings_rad, dtype=float))
    delta = np.deg2rad(error_deg)
    n = len(theta)
    a = np.empty((n, 4, 2))
    a[:, 0] = [np.sin(-delta), -np.cos(-delta)]
    a[:, 1] = [-np.sin(delta), np.cos(delta)]
    a[:, 2, 0], a[:, 2, 1] = np.sin(theta - delta), -np.cos(theta - delta)
    a[:, 3, 0], a[:, 3, 1] = -np.sin(theta + delta), np.cos(theta + delta)
    b = np.zeros((n, 4))
    b[:, 2:] = a[:, 2:] @ np.asarray(point)
    vertices = np.empty((n, 6, 2))
    valid = np.empty((n, 6), dtype=bool)
    scale = max(1., float(np.linalg.norm(point)))
    for k, (i, j) in enumerate(itertools.combinations(range(4), 2)):
```

```
        det = a[:, i, 0] * a[:, j, 1] - a[:, i, 1] * a[:, j, 0]
        if np.any(np.abs(det) < 1e-12):
            raise ArithmeticError("Parallel boundary outside supported candidate geometry")
        vertices[:, k, 0] = (b[:, i] * a[:, j, 1] - a[:, i, 1] * b[:, j]) / det
        vertices[:, k, 1] = (a[:, i, 0] * b[:, j] - b[:, i] * a[:, j, 0]) / det
        residual = np.einsum("nij,nj->ni", a, vertices[:, k]) - b
        valid[:, k] = np.max(residual, axis=1) <= 1e-9 * scale
    if np.any(np.sum(valid, axis=1) < 1):
        raise ArithmeticError("No feasible vertices; unsupported or ill-conditioned geometry")
    diameters_squared = np.zeros(n)
    for i, j in itertools.combinations(range(6), 2):
        distances = np.sum((vertices[:, i] - vertices[:, j]) ** 2, axis=1)
        diameters_squared = np.maximum(diameters_squared,
                                       np.where(valid[:, i] & valid[:, j], distances, 0.))
    return np.sqrt(diameters_squared)

def evaluate(point, error_deg, maximum_radius, samples, refine=False):
    low, high = bearing_interval(point, error_deg, maximum_radius)
    bearings = np.linspace(low, high, samples)
    diameters = two_wedge_diameters(point, bearings, error_deg)
    index = int(np.argmax(diameters))
    best_theta, best_diameter = float(bearings[index]), float(diameters[index])
    if refine:
        # 细化采样局部极大值；这是数值对抗检查，
        # 不能证明不存在更窄的未采样峰值。
        peaks = np.flatnonzero((diameters[1:-1] >= diameters[:-2])
                               & (diameters[1:-1] >= diameters[2:])) + 1
        for i in peaks:
            opt = minimize_scalar(lambda t: -two_wedge_diameters(point, [t], error_deg)[0],
                                  bounds=(bearings[i - 1], bearings[i + 1]),
                                  method="bounded", options={"xatol": 1e-12})
            if opt.success and -opt.fun > best_diameter:
                best_theta, best_diameter = float(opt.x), float(-opt.fun)
    return {
        "second_bearing_interval_relative_deg": np.rad2deg([low, high]).tolist(),
        "sampled_worst_diameter_m": best_diameter,
        "worst_second_bearing_relative_deg": float(np.rad2deg(best_theta)),
        "bearing_samples": samples,
        "local_peak_refinement": refine,
    }

def choose(first_position, first_bearing_deg, geometry_config, config):
    first = np.asarray(first_position, dtype=float)
    if first.shape != (2,) or not np.all(np.isfinite(first)) or not np.isfinite(first_bearing_deg)
   :
        raise ValueError("Require finite first_position_m[2] and first_bearing_deg")
    error = float(geometry_config["bearing_error_deg"])
    radius, maximum = config["minimum_reception_radius_m"], config["maximum_reception_radius_m"]
    gamma = config["minimum_measured_crossing_angle_deg"]
    if not (0 < error < 45 and 2 * error < gamma < 90
            and 0 < radius < maximum and config["walking_speed_m_s"] > 0):
        raise ValueError("Unsupported physical/angle configuration")
    steps = config["candidate_grid_steps_m"]
    if (not steps or any(not np.isfinite(s) or s <= 0 for s in steps)
            or any(b >= a for a, b in zip(steps, steps[1:]))
            or config["training_bearing_samples"] < 3
            or config["validation_bearing_samples"] < 3):
        raise ValueError("Invalid grid settings")
    evaluated = {}
    history = []
    best = None
```

```
    for level, step in enumerate(steps):
        if level == 0:
            xs = ys = np.arange(step, radius + 1e-8, step)
        else:
            span = steps[level - 1]
            xs = np.arange(max(step, best[0] - span), min(radius, best[0] + span) + step / 2, step
)
            ys = np.arange(max(step, best[1] - span), min(radius, best[1] + span) + step / 2, step
)
        for a in xs:
            for b in ys:
                p = (float(a), float(b))
                if p in evaluated or not candidate(p, error, config):
                    continue
                metrics = evaluate(p, error, maximum, config["training_bearing_samples"])
                evaluated[p] = metrics["sampled_worst_diameter_m"]
        if not evaluated:
            raise ValueError("No candidate found; adjust explicit angle/grid settings")
        best = min(evaluated, key=lambda p: (evaluated[p], np.hypot(*p), p))
        history.append({"step_m": step, "cumulative_candidates": len(evaluated),
                        "selected_local_m": list(best), "sampled_worst_diameter_m": evaluated[best
]})

    theta = np.deg2rad(first_bearing_deg % 360)
    rotation = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
    local = np.array(best)
    selected = first + rotation @ local
    reflected = first + rotation @ (local * [1, -1])
    audit = evaluate(best, error, maximum, config["validation_bearing_samples"], refine=True)
    duration = np.linalg.norm(local) / config["walking_speed_m_s"] + config["measurement_time_s"]
    # 用于同口径保守比较的简单前向/侧向候选点。
    reference = [0.5 * radius, 0.5 * radius]
    comparison = None
    if candidate(reference, error, config):
        comparison = dict(local_position_m=reference,
                          **evaluate(reference, error, maximum, config["validation_bearing_samples
"], True))
    return {
        "status": "selected", "first_position_m": first.tolist(),
        "first_bearing_deg": float(first_bearing_deg % 360),
        "second_position_m": selected.tolist(), "symmetric_second_position_m": reflected.tolist(),
        "selected_local_m": local.tolist(),
        "move_distance_m": float(np.linalg.norm(local)),
        "move_and_measure_time_s": float(duration),
        "same_channel_switch_time_s": 0.,
        "safe_region_local_disk_centers_m": safe_disk_centers(error, radius).tolist(),
        "safe_region_disk_radius_m": radius,
        "minimum_measured_crossing_angle_deg": gamma,
        "safe_region_margin_m": float(radius - np.max(np.linalg.norm(safe_disk_centers(error,
radius) - local, axis=1))),
        "selection_metric": "sampled_minimax_of_Q1_angular_polygon_diameter",
        "prior": {"radial_interval_m": [0., maximum], "angular_error_deg": error,
                  "target_disk_used_to_tighten": False, "near_radius_used_to_tighten": False},
        "global_optimality_claimed": False, "formal_interval_certificate": False,
        "grid_history": history, "audit": audit, "reference_candidate": comparison,
        "reception_guarantee_basis": "analytical three-disk region; proof in q2-solution.md",
    }

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("input", type=Path)
```

```
    configs = Path(__file__).resolve().parents[1] / "configs"
    parser.add_argument("--config", type=Path, default=configs / "q2.json")
    parser.add_argument("--geometry-config", type=Path, default=configs / "q1.json")
    parser.add_argument("--output", type=Path)
    args = parser.parse_args()
    data = json.loads(args.input.read_text(encoding="utf-8"))
    config = json.loads(args.config.read_text(encoding="utf-8"))
    geometry = json.loads(args.geometry_config.read_text(encoding="utf-8"))
    result = choose(data["first_position_m"], data["first_bearing_deg"], geometry, config)
    result["provenance"] = {key: hashlib.sha256(path.read_bytes()).hexdigest() for key, path in {
        "input_sha256": args.input, "config_sha256": args.config,
        "geometry_config_sha256": args.geometry_config, "solver_sha256": Path(__file__),
        "q1_solver_sha256": Path(q1.__file__),
    }.items()}
    result["input_description"] = data.get("description", "user-supplied first bearing")
    encoded = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(encoded, encoding="utf-8")
    else:
        print(encoded, end="")

if __name__ == "__main__":
    main()
```

plot_q2.m

```
function plot_q2(outputDir)
% 候选区域填色仅为展示网格，不是可行性或最优性证明。
base = fileparts(mfilename('fullpath'));
if nargin < 1, outputDir = fullfile(base, 'results'); end
if ~isfolder(outputDir), mkdir(outputDir); end
cfg = jsondecode(fileread(fullfile(base, '..', 'configs', 'q2.json')));
geo = jsondecode(fileread(fullfile(base, '..', 'configs', 'q1.json')));
names = {'origin', 'edge', 'oblique'};
labels = {'Central reading', 'Outward edge reading', 'Oblique edge reading'};
data = cell(1,3);
for k = 1:3
    data{k} = jsondecode(fileread(fullfile(base, 'results', ['q2-' names{k} '.json'])));
end
f = figure('Visible','off','Color','w','Position',[100 100 1450 820]);
tiledlayout(2,3,'TileSpacing','compact','Padding','compact');
for k=1:3
    r=data{k}; cells=r.prior.cells_rad_rad_m_m;
    nexttile(k); hold on; box on;
    extent=max([cells(:,4); abs(r.selected_local_m(:))*1.4]);
    xs=linspace(0,max(30,extent),55);
    ys=linspace(-max(30,extent),max(30,extent),91);
    [xx,yy]=meshgrid(xs,ys); allowed=false(size(xx));
    for n=1:numel(xx)
        q=[xx(n),abs(yy(n))]; side=sign(yy(n));
        if side==0, continue; end
        c=cells;
        if side<0, c(:,1:2)=[-cells(:,2),-cells(:,1)]; end
        allowed(n)=candidateForDisplay(q,c,cfg,geo.bearing_error_deg);
    end
    scatter(xx(allowed),yy(allowed),4,[.80 .88 .96],'filled','HandleVisibility','off');
    for j=1:size(cells,1)
        aa=linspace(cells(j,1),cells(j,2),20);
```

```
        rr=[cells(j,3)*ones(1,20),cells(j,4)*ones(1,20)];
        av=[aa fliplr(aa)];
        patch(rr.*cos(av),rr.*sin(av),[.95 .72 .26],...
            'EdgeColor','none','FaceAlpha',.7,'HandleVisibility','off');
    end
    plot(0,0,'ko','MarkerFaceColor','k','DisplayName','First point');
    q=r.selected_local_m;
    plot([0 q(1)],[0 q(2)],'-','Color',[.10 .45 .35],'LineWidth',1.5,'HandleVisibility','off');
    plot(q(1),q(2),'p','MarkerSize',12,'MarkerFaceColor',[.10 .55 .40],...
        'MarkerEdgeColor',[.10 .40 .30],'DisplayName','Adaptive second point');
    xlim([0 max(30,extent)]); ylim([-max(30,extent) max(30,extent)]);
    axis equal; grid on; xlabel('Local forward / m'); ylabel('Local lateral / m');
    title(sprintf('%s\nD = %.3f m; move + measure = %.2f s',labels{k},...
        r.audit.sampled_worst_diameter_m,r.move_and_measure_time_s));
    if k==1, legend('Location','southoutside'); end
end
old=zeros(3,1); improved=zeros(3,1); durations=zeros(3,1);
for k=1:3
    old(k)=min([data{k}.reference_candidates.sampled_worst_diameter_m]);
    improved(k)=data{k}.audit.sampled_worst_diameter_m;
    durations(k)=data{k}.move_and_measure_time_s;
end
nexttile(4); bar([old improved]); grid on;
xticklabels({'Center','Edge','Oblique'}); ylabel('Worst angular polygon diameter / m');
title('Same-prior numerical comparison'); legend({'Fixed (800, +/-600)','Adaptive'},'Location','
   northoutside');
nexttile(5); bar([205*ones(3,1) durations]); grid on;
xticklabels({'Center','Edge','Oblique'}); ylabel('Move + second measurement / s');
title('Movement cost'); legend({'Fixed','Adaptive'},'Location','northoutside');
nexttile(6); axis off;
text(0,.95,{'Blue dots: candidates in plotted window', 'Orange: conservative prior enclosure',...
    'Both candidate sides are evaluated.', '', 'Reception: continuous cell extrema',...
    'Selection and diameter: numerical audit', 'No global optimality certificate',...
    'Constructed cases; no simulator scores'},'Units','normalized','FontSize',11,'
   VerticalAlignment','top');
sgtitle('Q2: disk-aware second observation','FontSize',16);
exportgraphics(f,fullfile(outputDir,'q2-adaptive.png'),'Resolution',150);
exportgraphics(f,fullfile(outputDir,'q2-adaptive.pdf'),'ContentType','vector');
savefig(f,fullfile(outputDir,'q2-adaptive.fig'));
close(f);
end

function yes=candidateForDisplay(q,c,cfg,error)
% 与 q2_geometry 相同的解析单元约束，仅用于显示。
yes=false; delta=deg2rad(error); margin=cfg.geometry_margin_m;
if q(2)<=q(1)*tan(delta)+margin, return; end
if ~isempty(cfg.maximum_move_distance_m) && norm(q)>cfg.maximum_move_distance_m, return; end
phi=atan2(q(2),q(1)); rho=norm(q); lo=c(:,1); hi=c(:,2);
ends=[q(1)*cos(lo)+q(2)*sin(lo),q(1)*cos(hi)+q(2)*sin(hi)];
dmin=min(ends,[],2); dmax=max(ends,[],2);
inside=lo<=phi & phi<=hi; dmax(inside)=rho;
rr=[c(:,3),c(:,4),max(c(:,3),min(cfg.minimum_reception_radius_m,c(:,4)))];
dd=sqrt(max(0,rho^2+rr.^2-2*rr.*dmin));
slack=max(cfg.minimum_reception_radius_m,rr)-dd;
if min(slack,[],'all')<margin, return; end
closest=max(c(:,3),min(dmax,c(:,4)));
near=sqrt(max(0,rho^2+closest.^2-2*closest.*dmax));
if min(near)<=cfg.near_radius_m+margin, return; end
if q(2)<=max(max(c(:,3).*sin(hi),c(:,4).*sin(hi))), return; end
angles=[];
for j=1:size(c,1)
```

```
    for r=c(j,3:4)
        aa=c(j,1:2);
        if r<=rho
            extra=phi+[-1 1]*acos(r/rho);
            aa=[aa extra(extra>=c(j,1)&extra<=c(j,2))]; %#ok<AGROW>
        end
        angles=[angles atan2(r*sin(aa)-q(2),r*cos(aa)-q(1))]; %#ok<AGROW>
    end
end
gamma=deg2rad(cfg.minimum_measured_crossing_angle_deg);
yes=min(angles)-delta>=-pi+gamma && max(angles)+delta<=-gamma;
end
```

robot.py

```
# 可移植正式入口：默认离线检查，--run 才连接模拟器

import argparse
import datetime
import hashlib
import json
import math
import os
from pathlib import Path
import sys
import time
import traceback
import urllib.request
import urllib.error
import io

sys.dont_write_bytecode = True
for key in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS'):
    os.environ[key] = '1'
HERE = Path(__file__).resolve().parent
STUDY = HERE / 'vendor'
ARCHIVE = STUDY / 'frozen-strategy-20260912-heuristics-stop'
POLICY = ARCHIVE / 'selected-policy'
BASE = STUDY / 'frozen/contests/2026-B'
sha = lambda p: hashlib.sha256(p.read_bytes()).hexdigest()

def save(path, value):
    with path.open('x', encoding='utf-8') as f:
        json.dump(value, f, ensure_ascii=False, indent=2)
        f.write('\n')
        f.flush()
        os.fsync(f.fileno())

def prepare():
    manifest = json.loads((HERE / 'SOURCE_SHA256.json').read_text(encoding='utf-8'))
    checked = {}
    for rel, expected in manifest.items():
        if sha(STUDY / rel) != expected:
            raise RuntimeError('Frozen source hash mismatch: ' + rel)
        checked[rel] = expected
    sys.path[:0] = [str(POLICY), str(BASE / 'src')]
    import controller
    from policy import solve
    from protocol import ProtocolConfig, SimulatorClient
```

```
    assert (
        sha(POLICY / 'policy.py')
        == '4af80777ba07f16c905241de2f11a739a12c153ce34354a5a80b75f1bd003275'
    )
    return (
        checked,
        controller.observation_from_reply,
        solve,
        ProtocolConfig,
        SimulatorClient,
    )

def go_round(value):
    whole, fraction = divmod(abs(value), 1.0)
    rounded = int(whole) + (fraction >= 0.5)
    return -rounded if value < 0 else rounded

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        '--check',
        action='store_true',
        help='Offline source and import validation (default).',
    )
    parser.add_argument(
        '--run',
        action='store_true',
        help='Send live HTTP commands to an already-ready simulator session.',
    )
    parser.add_argument('--output', type=Path)
    parser.add_argument('--problem', type=int, choices=(3, 4))
    parser.add_argument('--case-id')
    parser.add_argument('--mode', choices=('practice', 'formal'), default='practice')
    parser.add_argument('--robot-id')
    args = parser.parse_args()
    checked, observation, solve, Config, Client = prepare()
    binding = dict(
        policy_sha256=sha(POLICY / 'policy.py'),
        source_sha256=checked,
        adapter_sha256=sha(Path(__file__)),
        optimization_resumed=False,
        variants={'3': 'tenth-v1', '4': 'tenth-v1'},
    )
    if args.check or not args.run:
        print(f'CHECK PASS: {len(checked)} frozen source hashes checked; no HTTP calls')
        return
    if not args.output or not args.problem or not args.case_id or not args.robot_id:
        parser.error('--run requires --output, --problem, --case-id, --robot-id')
    if args.robot_id.strip() != args.robot_id:
        parser.error('--robot-id must not contain leading/trailing whitespace')
    out = args.output.resolve()
    if out.exists():
        parser.error(
            '--output must be a new directory; existing records are never overwritten'
        )
    ready = dict(mode=args.mode, problem=args.problem, case_id=args.case_id)
    out.mkdir(parents=True)
    problem = ready['problem']
    variant = binding['variants'][str(problem)]
    save(
```

```
        out / 'metadata.json',
        dict(
            binding,
            problem=problem,
            variant=variant,
            case_id=ready['case_id'],
            mode=args.mode,
            started_utc=datetime.datetime.now(datetime.timezone.utc).isoformat(),
        ),
    )
    wire = (out / 'wire.jsonl').open('x', encoding='utf-8')
    attempts = (out / 'http-attempts.jsonl').open('x', encoding='utf-8')

    def emit(f, value):
        value = dict(
            value, recorded_utc=datetime.datetime.now(datetime.timezone.utc).isoformat()
        )
        f.write(json.dumps(value, ensure_ascii=False, allow_nan=False) + '\n')
        f.flush()
        os.fsync(f.fileno())

    opener = urllib.request.build_opener(urllib.request.ProxyHandler({})).open

    def logged_open(request, timeout):
        payload = json.loads(request.data)
        emit(
            attempts,
            dict(
                phase='send',
                path=request.selector,
                payload=dict(payload, robot_id='<redacted>'),
            ),
        )
        try:
            with opener(request, timeout=timeout) as response:
                status = response.status
                body = response.read()
            emit(
                attempts,
                dict(
                    phase='response',
                    request_id=payload['request_id'],
                    status=status,
                    body=body.decode('utf-8', errors='replace'),
                ),
            )
            buffered = io.BytesIO(body)
            buffered.status = status
            return buffered
        except urllib.error.HTTPError as exc:
            body = exc.read()
            emit(
                attempts,
                dict(
                    phase='response',
                    request_id=payload['request_id'],
                    status=exc.code,
                    body=body.decode('utf-8', errors='replace'),
                ),
            )
            raise urllib.error.HTTPError(
```

```
                exc.url, exc.code, exc.msg, exc.hdrs, io.BytesIO(body)
            ) from exc
        except Exception as exc:
            emit(
                attempts,
                dict(phase='error', request_id=payload['request_id'], error=repr(exc)),
            )
            raise

    class Recorded(Client):
        def _post(self, path, payload):
            assert path in ('/enter', '/measure', '/clear', '/exit')
            begin = time.perf_counter()
            emit(
                wire,
                dict(
                    phase='send',
                    path=path,
                    payload=dict(payload, robot_id='<redacted>'),
                ),
            )
            try:
                reply = super()._post(path, payload)
            except Exception as exc:
                emit(wire, dict(phase='error', path=path, error=repr(exc)))
                raise
            emit(
                wire,
                dict(
                    phase='response',
                    path=path,
                    reply=reply,
                    http_status=self.last_http_status,
                    elapsed_s=time.perf_counter() - begin,
                ),
            )
            if reply.get('accepted') is not True:
                raise RuntimeError('Simulator rejected action')
            return reply

    client = Recorded(Config(robot_id=args.robot_id), opener=logged_open)
    started = time.perf_counter()
    state = None
    error = None
    exit_error = None
    cleared = set()
    actions = 0
    deadline = None
    max_virtual = None
    try:
        entered = client.enter()
        deadline = time.perf_counter() + entered['remaining_real_duration_s'] - 5
        max_virtual = entered['max_virtual_duration_s']

        def action(kind, p, c):
            nonlocal actions
            if time.perf_counter() >= deadline:
                raise TimeoutError('official remaining real budget')
            if actions >= 10000 or client.virtual_time_s >= max_virtual:
                raise TimeoutError('official action/virtual budget')
            p = tuple(go_round(float(v) * 1e6) / 1e6 for v in p)
```

```
            if kind == 'measure':
                reply = client.measure(*p, c)
            elif kind == 'clear':
                reply = client.clear(*p, c)
            else:
                raise ValueError(kind)
            actions += 1
            if kind == 'clear' and reply.get('clear_result') == 'success':
                assert c not in cleared
                cleared.add(c)
            if actions % 100 == 0 or (
                kind == 'clear' and reply.get('clear_result') == 'success'
            ):
                print(
                    json.dumps(
                        dict(
                            problem=problem,
                            actions=actions,
                            cleared=len(cleared),
                            virtual_s=client.virtual_time_s,
                        )
                    ),
                    flush=True,
                )
            return observation('/' + kind, reply)

        state = solve(problem, action, variant)
    except Exception:
        error = traceback.format_exc()
    finally:
        if (
            client.session_open
            and (deadline is None or time.perf_counter() < deadline)
            and (max_virtual is None or client.virtual_time_s < max_virtual)
        ):
            try:
                client.exit()
            except Exception:
                exit_error = traceback.format_exc()
        wire.close()
        attempts.close()
        result = dict(
            problem=problem,
            variant=variant,
            case_id=ready['case_id'],
            mode=args.mode,
            state=state,
            error=error,
            exit_error=exit_error,
            actions=actions,
            cleared_channels=sorted(cleared),
            cleared_count=len(cleared),
            virtual_time_s=client.virtual_time_s,
            tc_s=client.virtual_time_s / len(cleared) if cleared else None,
            wall_s=time.perf_counter() - started,
            session_open=client.session_open,
            total_sources=None,
            full_clearance_verified=False,
        )
        save(out / 'result.json', result)
        print('RESULT', json.dumps(result, ensure_ascii=False), flush=True)
```

```
    if error or exit_error or client.session_open:
        raise SystemExit(1)

if __name__ == '__main__':
    main()
```

angular.py

```
# 环形连续覆盖

import math

class AngularCover:
    def __init__(self):
        self.points = []

    @staticmethod
    def interval(p):
        r = math.hypot(*p)
        if r < 1:
            return None
        bounds = [(s * s + r * r - 999.9**2) / (2 * s * r) for s in (1000, 1800)]
        if max(bounds) >= 1:
            return None
        alpha = math.acos(max(-1, max(bounds)))
        theta = math.atan2(p[1], p[0]) % (2 * math.pi)
        return theta - alpha, theta + alpha

    def gaps(self, extra=()):
        intervals = []
        for p in self.points + list(extra):
            bounds = self.interval(p)
            if bounds is None:
                continue
            a, b = bounds
            for shift in (-2 * math.pi, 0, 2 * math.pi):
                lo = max(0, a + shift)
                hi = min(2 * math.pi, b + shift)
                if lo < hi:
                    intervals.append((lo, hi))
        gaps = []
        end = 0
        for lo, hi in sorted(intervals):
            if lo > end + 1e-9:
                gaps.append((end, lo))
            end = max(end, hi)
        if end < 2 * math.pi - 1e-9:
            gaps.append((end, 2 * math.pi))
        if len(gaps) > 1 and gaps[0][0] == 0 and gaps[-1][1] == 2 * math.pi:
            first = gaps.pop(0)
            last = gaps.pop()
            gaps.append((last[0], first[1] + 2 * math.pi))
        return gaps

    def gain(self, p):
        return sum(b - a for a, b in self.gaps()) - sum(
            b - a for a, b in self.gaps([p])
        )
```

```
    def add(self, p):
        self.points.append(p)

    def pending(self, position):
        result = []
        r = 1250
        alpha = self.interval((r, 0))[1]
        for a, b in self.gaps():
            n = math.ceil((b - a) / (2 * alpha - 1e-5))
            for i in range(n):
                t = a + (i + 0.5) * (b - a) / n
                result.append((round(r * math.cos(t), 6), round(r * math.sin(t), 6)))
        return result
```

core.py

```
# 解析几何与精确有理数工具

from __future__ import annotations

from dataclasses import dataclass
from functools import lru_cache
import math
import time

ERROR = 1.01  # 包含两位小数示向度取整
EPS = 1e-7
POINTS = ((0.0, 0.0),) + tuple(
    (1500 * math.cos(k * math.pi / 3), 1500 * math.sin(k * math.pi / 3))
    for k in range(6)
)

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def axes(angle):
    t = math.radians(angle)
    return (math.cos(t), math.sin(t)), (-math.sin(t), math.cos(t))

def world(x, y, u, v):
    return (x * u[0] + y * v[0], x * u[1] + y * v[1])

def clip(poly, normal, offset):
    if not poly:
        return ()
    result = []
    a = poly[-1]
    fa = dot(normal, a) - offset - EPS
    for b in poly:
        fb = dot(normal, b) - offset - EPS
        if (fa <= 0) != (fb <= 0):
            t = fa / (fa - fb)
            result.append((a[0] + t * (b[0] - a[0]), a[1] + t * (b[1] - a[1])))
        if fb <= 0:
            result.append(b)
        a, fa = b, fb
    return tuple(result)

def square(q, radius):
```

```
    x, y = q
    return (
        (x - radius, y - radius),
        (x + radius, y - radius),
        (x + radius, y + radius),
        (x - radius, y + radius),
    )

def in_square(poly, q, radius):
    for n in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        poly = clip(poly, n, dot(n, q) + radius)
    return poly

def wedge(poly, q, bearing, half=ERROR):
    if not 0 < half < 90:
        raise ValueError('wedge half angle must lie in (0, 90)')
    lower, _ = axes(bearing - half)
    upper, _ = axes(bearing + half)
    for n in ((lower[1], -lower[0]), (-upper[1], upper[0])):
        poly = clip(poly, n, dot(n, q))
    return poly

def initial_polygon(q, observation):
    if observation['kind'] == 'near':
        return square(q, 5)
    if observation['kind'] != 'direction':
        raise ValueError('initialization requires a confirmed positive observation')
    u, v = axes(observation['bearing'])
    # 投影界适用于距离≤1500且角度误差≤ERROR
    s = 1500 * math.sin(math.radians(ERROR)) + EPS
    x, y = dot(q, u), dot(q, v)
    poly = tuple(
        world(a, b, u, v)
        for a, b in (
            (x - EPS, y - s),
            (x + 1500 + EPS, y - s),
            (x + 1500 + EPS, y + s),
            (x - EPS, y + s),
        )
    )
    return in_square(wedge(poly, q, observation['bearing']), (0, 0), 1800)

def posterior(poly, q, observation):
    kind = observation['kind']
    if kind == 'direction':
        return in_square(wedge(poly, q, observation['bearing']), q, 1500)
    if kind == 'near':
        return in_square(poly, q, 5)
    if kind == 'no_signal':
        # 安全但宽松：不声称实现圆形排除
        return poly
    raise ValueError('unknown/rejected receipts are not observations')

def contains(poly, point, tol=1e-5):
    return bool(poly) and all(
        (b[0] - a[0]) * (point[1] - a[1]) - (b[1] - a[1]) * (point[0] - a[0]) >= -tol
        for a, b in zip(poly, poly[1:] + poly[:1])
    )

def principal(poly):
    if not poly:
```

```
        raise ValueError('empty feasible enclosure')
    a, b = max(((a, b) for a in poly for b in poly), key=lambda ab: math.dist(*ab))
    return math.degrees(math.atan2(b[1] - a[1], b[0] - a[0])) % 180

@dataclass(frozen=True)
class Cover:
    points: tuple
    bound_s: float
    cell_radius_m: float

def cover_cost(points, start, anchor):
    # 执行直到首次成功，再在锚点同频道检测
    path = (start,) + tuple(points) + (anchor,)
    return (
        sum(math.dist(a, b) for a, b in zip(path, path[1:])) / 5
        + 3 * len(points)
        + 2
        + 5
    )

def cover(poly, start, anchor):
    if not poly:
        raise ValueError('empty enclosure cannot be a completed source')
    best = None
    for angle in (principal(poly), 0.0):
        u, v = axes(angle)
        xs, ys = [dot(p, u) for p in poly], [dot(p, v) for p in poly]
        lo, hi, bot, top = min(xs) - EPS, max(xs) + EPS, min(ys) - EPS, max(ys) + EPS
        w, h = hi - lo, top - bot
        center = world((lo + hi) / 2, (bot + top) / 2, u, v)
        radius = math.hypot(w, h) / 2
        if radius < 20 - 1e-5:
            plan = Cover((center,), cover_cost((center,), start, anchor), radius)
            if best is None or plan.bound_s < best.bound_s:
                best = plan
            continue
        nx, ny = max(1, math.ceil(w / 25)), max(1, math.ceil(h / 25))
        if nx * ny > 1000:
            continue
        # 四种蛇形朝向，顺序并列时用稳定编号
        for flipx in (False, True):
            for flipy in (False, True):
                cells = []
                for i in range(nx):
                    ii = nx - 1 - i if flipx else i
                    for j in range(ny):
                        jj = ny - 1 - j if (bool(i % 2) != flipy) else j
                        cells.append(
                            world(
                                lo + (ii + 0.5) * w / nx,
                                bot + (jj + 0.5) * h / ny,
                                u,
                                v,
                            )
                        )
                cost = cover_cost(cells, start, anchor)
                if best is None or cost < best.bound_s - 1e-9:
                    best = Cover(tuple(cells), cost, math.hypot(w / nx, h / ny) / 2)
    if best is None:
        raise ValueError('enclosure outside intended detected-source strip scope')
    return best
```

```
def initial_cover(anchor, first):
    poly = initial_polygon(anchor, first)
    generic = cover(poly, anchor, anchor)
    if first['kind'] == 'near':
        return generic
    u, v = axes(first['bearing'])
    cells = []
    for k in range(60):
        for y in ((-25, 0, 25) if k % 2 == 0 else (25, 0, -25)):
            p = world(12.5 + 25 * k, y, u, v)
            cells.append((anchor[0] + p[0], anchor[1] + p[1]))
    strip = Cover(tuple(cells), cover_cost(cells, anchor, anchor), math.sqrt(2) * 12.5)
    return generic if generic.bound_s < strip.bound_s else strip

def candidates(poly, position, group='union'):
    u, v = axes(principal(poly))
    xs, ys = [dot(p, u) for p in poly], [dot(p, v) for p in poly]
    lo, hi, mid = min(xs), max(xs), (min(ys) + max(ys)) / 2
    half = (hi - lo) / 2
    geometric = [
        world(lo + f * (hi - lo), mid + s * scale * max(half, 5), u, v)
        for f in (0.5, 0.25, 0.75)
        for scale in (1.0, 0.5)
        for s in (-1, 1)
    ]
    fixed = [
        (
            position[0]
            + r
            * (math.cos(math.radians(a)) * u[0] + s * math.sin(math.radians(a)) * v[0]),
            position[1]
            + r
            * (math.cos(math.radians(a)) * u[1] + s * math.sin(math.radians(a)) * v[1]),
        )
        for r in (50, 100, 200, 400, 800)
        for a in (30, 60)
        for s in (-1, 1)
    ]
    interleaved = [
        item
        for i in range(max(len(fixed), len(geometric)))
        for family in (geometric, fixed)
        for item in family[i : i + 1]
    ]
    pool = (
        fixed if group == 'fixed' else geometric if group == 'geometry' else interleaved
    )
    # 量化坐标才是实际候选，不只是排序键
    return tuple(
        dict.fromkeys(
            (round(p[0], 6), round(p[1], 6))
            for p in pool
            if math.dist(p, position) > 1e-4
        )
    )

def branches(poly, q, anchor, bin_deg=10, deadline=math.inf):
    if 360 % bin_deg or not 0 < bin_deg < 90:
        raise ValueError('bin width must divide 360 and be below 90')
    result = {}
```

```
    for index in range(int(360 / bin_deg)):
        if time.perf_counter() >= deadline:
            raise TimeoutError
        p = wedge(poly, q, (index + 0.5) * bin_deg, bin_deg / 2 + ERROR)
        if p:
            result[('direction', index)] = cover(p, q, anchor)
    p = in_square(poly, q, 5)
    if p:
        result[('near', 0)] = cover(p, q, anchor)

    # 未保证接收时，在 no_signal 分支保留原 K
    if max(math.dist(q, p) for p in poly) > 1000 - 1e-5:
        result[('no_signal', 0)] = cover(poly, q, anchor)
    return result

def choose_measure(
    poly, position, anchor, incumbent, group='union', bin_deg=10, budget_s=math.inf
):
    begin = time.perf_counter()
    deadline = begin + budget_s
    selected = None
    best = incumbent.bound_s
    evaluated = 0
    timed_out = False
    for q in candidates(poly, position, group):
        if time.perf_counter() >= deadline:
            timed_out = True
            break
        try:
            children = branches(poly, q, anchor, bin_deg, deadline)
        except TimeoutError:
            timed_out = True
            break  # 不使用未完整评估的反馈树
        if time.perf_counter() >= deadline:
            timed_out = True
            break
        evaluated += 1
        value = (
            math.dist(position, q) / 5 + 5 + max(c.bound_s for c in children.values())
        )
        if value < best - 1e-6:
            best = value
            selected = (q, children, value)
    return selected, {
        'evaluated': evaluated,
        'timed_out': timed_out,
        'wall_s': time.perf_counter() - begin,
        'old_bound_s': incumbent.bound_s,
        'selected_bound_s': best,
    }

def localize(anchor, channel, first, group='union', budget_s=math.inf, max_extra=3):
    poly = initial_polygon(anchor, first)
    position = anchor
    incumbent = initial_cover(anchor, first)
    initial_bound = incumbent.bound_s
    spent = 0.0
    decisions = []
    for _ in range(max_extra if group != 'none' else 0):
        selected, meta = choose_measure(
            poly, position, anchor, incumbent, group, budget_s=budget_s
```

```
        )
        decisions.append(meta)
        if selected is None:
            break
        q, children, value = selected
        obs = yield {
            'kind': 'measure',
            'position': q,
            'channel': channel,
            'purpose': 'localize',
            'enclosure': poly,
        }
        spent += math.dist(position, q) / 5 + 5
        key = (
            ('direction', int((obs['bearing'] % 360) // 10))
            if obs['kind'] == 'direction'
            else (obs['kind'], 0)
        )
        if key not in children:
            raise ValueError('feedback not covered by predicted branches')
        poly = posterior(poly, q, obs)
        inherited = children[key]  # 保留精确认证的后续，不只保留标量
        fresh = cover(poly, q, anchor)
        incumbent = fresh if fresh.bound_s <= inherited.bound_s else inherited
        position = q
        assert spent + incumbent.bound_s <= initial_bound + 1e-5
    for point in incumbent.points:
        obs = yield {
            'kind': 'clear',
            'position': point,
            'channel': channel,
            'purpose': 'cover',
            'enclosure': poly,
        }
        spent += math.dist(position, point) / 5 + (5 if obs['kind'] == 'success' else 3)
        position = point
        if obs['kind'] == 'success':
            break
        if obs['kind'] != 'no_target_in_range':
            raise ValueError('unconfirmed clear cannot advance the fallback')
    else:
        raise ValueError('finite cover exhausted without success')
    # 显式支付并执行返回，不虚构免费移动
    obs = yield {
        'kind': 'measure',
        'position': anchor,
        'channel': channel,
        'purpose': 'return',
    }
    if obs['kind'] != 'no_signal':
        raise ValueError('cleared channel returned unexpected feedback')
    spent += math.dist(position, anchor) / 5 + 5
    assert spent <= initial_bound + 1e-5
    return {'initial_bound_s': initial_bound, 'actual_s': spent, 'decisions': decisions}

def block_channels(current, channels, end):
    middle = sorted(set(channels) - {end})
    if current in middle:
        middle.remove(current)
        middle.insert(0, current)
    return tuple(middle + [end])
```

```
def route_cost(route, points, start, channel):
    cost = 0.0
    for i, channels in route:
        cost += math.dist(start, points[i]) / 5
        for c in channels:
            cost += 5 + int(c != channel)
            channel = c
        start = points[i]
    return cost

def schedule(
    required,
    start=(0.0, 0.0),
    channel=1,
    points=POINTS,
    mode='exact',
    budget_s=math.inf,
    incumbent_route=None,
):
    start_clock = time.perf_counter()
    required = tuple(tuple(sorted(s)) for s in required)
    n = len(points)
    mask = sum(1 << i for i in range(n) if required[i])
    fallback, p, c = [], start, channel
    todo = [i for i in range(n) if required[i]]
    while todo:
        i = (
            todo[0]
            if mode == 'fixed'
            else min(todo, key=lambda i: (round(math.dist(p, points[i]), 8), i))
        )
        order = (
            ((c,) + tuple(x for x in required[i] if x != c))
            if c in required[i]
            else required[i]
        )
        fallback.append((i, order))
        todo.remove(i)
        p, c = points[i], order[-1]
    if incumbent_route is not None:
        retained = tuple(
            (i, tuple(c for c in cs if c in required[i])) for i, cs in incumbent_route
        )
        retained = tuple((i, cs) for i, cs in retained if cs)
        expected = sorted((i, c) for i, cs in enumerate(required) for c in cs)
        if sorted((i, c) for i, cs in retained for c in cs) != expected:
            raise ValueError('incumbent task multiset mismatch')
        if route_cost(retained, points, start, channel) <= route_cost(
            fallback, points, start, channel
        ):
            fallback = list(retained)
    if mode != 'exact' or budget_s <= 0:
        return tuple(fallback), {
            'exact': False,
            'timed_out': mode == 'exact',
            'wall_s': time.perf_counter() - start_clock,
        }
    choices = {}
    locations = tuple(points) + (start,)
```

```
    @lru_cache(None)
    def value(mask, last, c):
        if time.perf_counter() - start_clock >= budget_s:
            raise TimeoutError
        if not mask:
            return 0.0
        best = (math.inf, 0, 0)
        for i in range(n):
            if not mask & (1 << i):
                continue
            cs = required[i]
            for end in cs:
                switches = len(cs) - 1 + int(c not in cs or (c == end and len(cs) > 1))
                score = (
                    math.dist(locations[last], points[i]) / 5 + 5 * len(cs) + switches
                )
                candidate = (score + value(mask ^ (1 << i), i, end), i, end)
                if candidate < best:
                    best = candidate
        choices[mask, last, c] = best[1:]
        return best[0]

    try:
        value(mask, n, channel)
    except TimeoutError:
        return tuple(fallback), {
            'exact': False,
            'timed_out': True,
            'wall_s': time.perf_counter() - start_clock,
        }
    route, last, c = [], n, channel
    while mask:
        i, end = choices[mask, last, c]
        route.append((i, block_channels(c, required[i], end)))
        mask ^= 1 << i
        last, c = i, end
    assert (
        route_cost(route, points, start, channel)
        <= route_cost(fallback, points, start, channel) + 1e-7
    )
    return tuple(route), {
        'exact': True,
        'timed_out': False,
        'wall_s': time.perf_counter() - start_clock,
    }

def controller(mode='exact', group='union', budget_s=math.inf):
    required = [set(range(1, 21)) for _ in POINTS]
    cleared, excluded = set(), set()
    position, channel = (0.0, 0.0), 1
    cached_route = None
    virtual_s = 0.0
    metrics = {
        'planning': [],
        'sources': [],
        'responsibility_history': [],
        'global_ledger_s': [],
    }
    while any(required):
        route, meta = schedule(
            required,
```

```
            position,
            channel,
            mode=mode,
            budget_s=budget_s,
            incumbent_route=cached_route,
        )
        cached_route = list(route)
        ledger = (
            virtual_s
            + route_cost(route, POINTS, position, channel)
            + (16 - len(cleared)) * 1746
        )
        assert ledger <= 30569 + 1e-5
        metrics['global_ledger_s'].append(ledger)
        metrics['planning'].append(meta)
        i, order = route[0]
        discovered = False
        for c in order:
            obs = yield {
                'kind': 'measure',
                'position': POINTS[i],
                'channel': c,
                'purpose': 'discovery',
            }
            if obs['kind'] not in ('direction', 'near', 'no_signal'):
                raise ValueError('unconfirmed discovery receipt')
            virtual_s += math.dist(position, POINTS[i]) / 5 + 5 + int(c != channel)
            position, channel = POINTS[i], c
            required[i].remove(c)
            cached_route[0] = (i, tuple(x for x in cached_route[0][1] if x != c))
            if not cached_route[0][1]:
                cached_route.pop(0)
            if obs['kind'] in ('direction', 'near'):
                for duties in required:
                    duties.discard(c)
                info = yield from localize(position, c, obs, group, budget_s)
                metrics['sources'].append(info)
                virtual_s += info['actual_s']
                cleared.add(c)
                discovered = True
            elif not any(c in duties for duties in required):
                excluded.add(c)
            metrics['responsibility_history'].append(sum(map(len, required)))
            if discovered:
                break
    assert cleared.isdisjoint(excluded) and cleared | excluded == set(range(1, 21))
    return {
        'completed': True,
        'controller_virtual_s': virtual_s,
        'cleared_channels': sorted(cleared),
        'excluded_channels': sorted(excluded),
        **metrics,
    }
```

improvement.py

```
# 覆盖改进

from dataclasses import dataclass
```

```
import math
import time

import core as base

@dataclass(frozen=True)
class Cell:
    kind: str
    lo: float
    hi: float
    poly: tuple
    plan: base.Cover

def check_deadline(deadline):
    if time.perf_counter() >= deadline:
        raise TimeoutError

def coarse_tree(poly, q, anchor, deadline=math.inf):
    cells = []
    for lo in range(0, 360, 10):
        check_deadline(deadline)
        p = base.wedge(poly, q, lo + 5, 5 + base.ERROR)
        if p:
            cells.append(Cell('direction', lo, lo + 10, p, base.cover(p, q, anchor)))
    check_deadline(deadline)
    p = base.in_square(poly, q, 5)
    if p:
        cells.append(Cell('near', 0, 0, p, base.cover(p, q, anchor)))
    check_deadline(deadline)
    if max(math.dist(q, p) for p in poly) > 1000 - 1e-5:
        cells.append(Cell('no_signal', 0, 0, poly, base.cover(poly, q, anchor)))
    check_deadline(deadline)
    if not cells:
        raise ValueError('empty feedback tree')
    return tuple(cells)

def tree_bound(cells):
    return max(c.plan.bound_s for c in cells)

def response_plan(cells, obs):
    kind = obs.get('kind')
    if kind == 'direction':
        bearing = obs.get('bearing')
        if not isinstance(bearing, (int, float)) or not math.isfinite(bearing):
            raise ValueError('invalid bearing')
        bearing %= 360
        matches = [c for c in cells if c.kind == kind and c.lo <= bearing < c.hi]
    else:
        matches = [c for c in cells if c.kind == kind]
    if len(matches) != 1:
        raise ValueError('feedback not covered uniquely by saved tree')
    return matches[0].plan

def refine_once(cells, q, anchor, min_width_deg=1.0, deadline=math.inf):
    if not 0 < min_width_deg <= 10:
        raise ValueError('min_width_deg must lie in (0, 10]')
    check_deadline(deadline)
    i = max(range(len(cells)), key=lambda i: cells[i].plan.bound_s)
    parent = cells[i]
    if parent.kind != 'direction' or parent.hi - parent.lo <= min_width_deg:
        return cells, False
```

```
    mid = (parent.lo + parent.hi) / 2
    children = []
    for lo, hi in ((parent.lo, mid), (mid, parent.hi)):
        check_deadline(deadline)
        p = base.wedge(parent.poly, q, (lo + hi) / 2, (hi - lo) / 2 + base.ERROR)
        if p:
            fresh = base.cover(p, q, anchor)
            plan = fresh if fresh.bound_s <= parent.plan.bound_s else parent.plan
            children.append(Cell('direction', lo, hi, p, plan))
    check_deadline(deadline)
    if not children:
        raise ValueError('nonempty parent lost both feedback children')
    result = cells[:i] + tuple(children) + cells[i + 1 :]
    assert tree_bound(result) <= tree_bound(cells) + 1e-8
    return result, True

def choose_measure(
    poly,
    position,
    anchor,
    incumbent,
    group='union',
    budget_s=math.inf,
    refine=True,
    min_width_deg=1.0,
    max_splits=512,
):
    if math.isnan(budget_s) or budget_s < 0 or max_splits < 0:
        raise ValueError('invalid planning budget')
    if not 0 < min_width_deg <= 10:
        raise ValueError('invalid feedback resolution')
    start = time.perf_counter()
    deadline = start + budget_s
    candidates = []
    selected = None
    best = incumbent.bound_s
    splits = 0
    timed_out = False

    def consider(q, cells):
        nonlocal best, selected
        score = math.dist(position, q) / 5 + 5 + tree_bound(cells)
        if score < best - 1e-6:
            best = score
            selected = (q, cells, score)

    for q in base.candidates(poly, position, group):
        try:
            cells = coarse_tree(poly, q, anchor, deadline)
        except TimeoutError:
            timed_out = True
            break
        candidates.append([q, cells, True])
        consider(q, cells)
    coarse_best = best
    if refine and not timed_out:
        try:
            while splits < max_splits and any(item[2] for item in candidates):
                for item in candidates:
                    if not item[2] or splits >= max_splits:
                        continue
```

```
                    cells, changed = refine_once(
                        item[1], item[0], anchor, min_width_deg, deadline
                    )
                    item[1], item[2] = cells, changed
                    if changed:
                        splits += 1
                        consider(item[0], cells)
            except TimeoutError:
                timed_out = True
        assert best <= coarse_best + 1e-8
        return selected, {
            'evaluated': len(candidates),
            'splits': splits,
            'timed_out': timed_out,
            'wall_s': time.perf_counter() - start,
            'old_bound_s': incumbent.bound_s,
            'coarse_bound_s': coarse_best,
            'selected_bound_s': best,
            'split_limit_reached': refine and splits >= max_splits,
            'selected_cells': len(selected[1]) if selected else 0,
        }

def localize(
    anchor,
    channel,
    first,
    group='union',
    budget_s=math.inf,
    refine=True,
    return_to_anchor=False,
    max_extra=3,
):
    poly = base.initial_polygon(anchor, first)
    position = anchor
    incumbent = base.initial_cover(anchor, first)
    initial_bound = incumbent.bound_s
    spent = 0.0
    decisions = []
    for _ in range(max_extra if group != 'none' else 0):
        selected, meta = choose_measure(
            poly, position, anchor, incumbent, group, budget_s, refine
        )
        decisions.append(meta)
        if selected is None:
            break
        q, cells, _ = selected
        obs = yield {
            'kind': 'measure',
            'position': q,
            'channel': channel,
            'purpose': 'localize',
            'enclosure': poly,
        }
        inherited = response_plan(cells, obs)
        spent += math.dist(position, q) / 5 + 5
        poly = base.posterior(poly, q, obs)
        fresh = base.cover(poly, q, anchor)
        incumbent = fresh if fresh.bound_s <= inherited.bound_s else inherited
        position = q
        assert spent + incumbent.bound_s <= initial_bound + 1e-5
    for point in incumbent.points:
```

```
        obs = yield {
            'kind': 'clear',
            'position': point,
            'channel': channel,
            'purpose': 'cover',
            'enclosure': poly,
        }
        if obs['kind'] not in ('success', 'no_target_in_range'):
            raise ValueError('unconfirmed clear cannot advance fallback')
        spent += math.dist(position, point) / 5 + (5 if obs['kind'] == 'success' else 3)
        position = point
        if obs['kind'] == 'success':
            break
    else:
        raise ValueError('finite cover exhausted without success')
    # 仅是潜在值，不伪造或计入返回值
    exit_credit = math.dist(position, anchor) / 5 + 5
    assert spent + exit_credit <= initial_bound + 1e-5
    if return_to_anchor:  # 析因实验对照，不是默认
        obs = yield {
            'kind': 'measure',
            'position': anchor,
            'channel': channel,
            'purpose': 'return',
        }
        if obs['kind'] != 'no_signal':
            raise ValueError('unexpected return response')
        spent += exit_credit
        position, exit_credit = anchor, 0.0
    return {
        'initial_bound_s': initial_bound,
        'actual_s': spent,
        'position': position,
        'anchor': anchor,
        'exit_credit_s': exit_credit,
        'decisions': decisions,
    }

def controller(
    mode='exact', group='union', budget_s=math.inf, refine=True, return_to_anchor=False
):
    required = [set(range(1, 21)) for _ in base.POINTS]
    cleared, excluded = set(), set()
    position, channel = (0.0, 0.0), 1
    cached_route = None
    virtual_s = 0.0
    metrics = {
        'planning': [],
        'sources': [],
        'responsibility_history': [],
        'global_ledger_s': [],
        'planning_positions': [],
    }
    while any(required):
        route, meta = base.schedule(
            required,
            position,
            channel,
            mode=mode,
            budget_s=budget_s,
            incumbent_route=cached_route,
```

```
        )
        cached_route = list(route)
        ledger = (
            virtual_s
            + base.route_cost(route, base.POINTS, position, channel)
            + (16 - len(cleared)) * 1746
        )
        saved_measurements = 0 if return_to_anchor else 5 * len(cleared)
        assert ledger <= 30569 - saved_measurements + 1e-5
        metrics['global_ledger_s'].append(ledger)
        metrics['planning_positions'].append(position)
        metrics['planning'].append(meta)
        i, order = route[0]
        discovered = False
        for c in order:
            obs = yield {
                'kind': 'measure',
                'position': base.POINTS[i],
                'channel': c,
                'purpose': 'discovery',
            }
            if obs['kind'] not in ('direction', 'near', 'no_signal'):
                raise ValueError('unconfirmed discovery receipt')
            virtual_s += math.dist(position, base.POINTS[i]) / 5 + 5 + int(c != channel)
            position, channel = base.POINTS[i], c
            required[i].remove(c)
            cached_route[0] = (i, tuple(x for x in cached_route[0][1] if x != c))
            if not cached_route[0][1]:
                cached_route.pop(0)
            if obs['kind'] in ('direction', 'near'):
                for duties in required:
                    duties.discard(c)
                info = yield from localize(
                    position, c, obs, group, budget_s, refine, return_to_anchor
                )
                metrics['sources'].append(info)
                virtual_s += info['actual_s']
                position = info['position']  # 实际成功清除点
                cleared.add(c)
                discovered = True
            elif not any(c in duties for duties in required):
                excluded.add(c)
            metrics['responsibility_history'].append(sum(map(len, required)))
            if discovered:
                break
    assert cleared.isdisjoint(excluded) and cleared | excluded == set(range(1, 21))
    assert virtual_s <= 30569 - (0 if return_to_anchor else 5 * len(cleared)) + 1e-5
    return {
        'completed': True,
        'controller_virtual_s': virtual_s,
        'cleared_channels': sorted(cleared),
        'excluded_channels': sorted(excluded),
        **metrics,
    }

def run(generator, env, max_actions=5000):
    observed_costs = []
    local_start = None
    pending = False
    try:
        action = next(generator)
```

```
        for _ in range(max_actions):
            if action['purpose'] == 'discovery' and pending:
                observed_costs.append(env.time_s - local_start)
                pending = False
            obs = env.step(action)
            if action['purpose'] == 'discovery' and obs['kind'] in (
                'near',
                'direction',
            ):
                local_start, pending = env.time_s, True
            action = generator.send(obs)
    except StopIteration as end:
        if pending:
            observed_costs.append(env.time_s - local_start)
        result = end.value
        assert result['completed']
        assert set(result['cleared_channels']) == set(env.sources) == env.cleared
        assert set(result['excluded_channels']).isdisjoint(env.sources)
        assert abs(result['controller_virtual_s'] - env.time_s) < 1e-5
        assert len(observed_costs) == len(result['sources'])
        for actual, source in zip(observed_costs, result['sources']):
            assert abs(actual - source['actual_s']) < 1e-5
            assert (
                actual + source.get('exit_credit_s', 0)
                <= source['initial_bound_s'] + 1e-5
            )
        result.update(total_s=env.time_s, actions=len(env.trace), trace=env.trace)
        for field, key, value in [
            ('clear_calls', 'kind', 'clear'),
            ('return_calls', 'purpose', 'return'),
            ('extra_measure_calls', 'purpose', 'localize'),
            ('discovery_calls', 'purpose', 'discovery'),
        ]:
            result[field] = sum(row[key] == value for row in env.trace)
        return result
    raise AssertionError('action limit exhausted')
```

certified.py

```
# 覆盖证书

from fractions import Fraction as F
from functools import lru_cache
from math import factorial, isqrt

SCALE = 10**15

def rat(x):
    return x if isinstance(x, F) else F(x)

def outward(a, b):
    return F(a * SCALE // 1, SCALE), F(-((-b * SCALE) // 1), SCALE)

def add(a, b):
    return outward(a[0] + b[0], a[1] + b[1])

def mul(a, b):
    p = [x * y for x in a for y in b]
    return outward(min(p), max(p))
```

```
def neg(a):
    return -a[1], -a[0]

@lru_cache(None)
def pi_interval():
    def atan(n):
        value = sum(
            (F((-1) ** k, (2 * k + 1) * n ** (2 * k + 1)) for k in range(32)), F(0)
        )
        tail = F(1, 65 * n**65)
        return value, value + tail  # 32 项交错和最终为负

    a, b = atan(5), atan(239)
    return 16 * a[0] - 4 * b[1], 16 * a[1] - 4 * b[0]

@lru_cache(maxsize=8192)
def trig(degrees):
    degree = (rat(degrees) + 180) % 360 - 180
    x = mul((degree / 180, degree / 180), pi_interval())
    x2 = mul(x, x)
    sin, cos = (F(0), F(0)), (F(0), F(0))
    power = (F(1), F(1))
    for k in range(19):
        cc = F((-1) ** k, factorial(2 * k))
        ss = F((-1) ** k, factorial(2 * k + 1))
        cos = add(cos, mul(power, (cc, cc)))
        sin = add(sin, mul(mul(power, x), (ss, ss)))
        power = mul(power, x2)
    # |x|<4，用 36/37 阶泰勒余项的共同上界
    error = F(4**37, factorial(37))
    return (
        (max(F(-1), sin[0] - error), min(F(1), sin[1] + error)),
        (max(F(-1), cos[0] - error), min(F(1), cos[1] + error)),
    )

def sqrt_bounds(x, scale=10**9):
    x = rat(x)
    if x < 0:
        raise ValueError('negative squared distance')
    k = isqrt(x.numerator * scale * scale // x.denominator)
    lo = F(k, scale)
    hi = lo if lo * lo == x else F(k + 1, scale)
    return lo, hi

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def dist2(a, b):
    return sum((rat(x) - rat(y)) ** 2 for x, y in zip(a, b))

def clip(poly, n, offset):
    if not poly:
        return ()
    out = []
    a = poly[-1]
    fa = dot(n, a) - offset
    for b in poly:
        fb = dot(n, b) - offset
        if (fa <= 0) != (fb <= 0):
            t = fa / (fa - fb)
```

```
            out.append(tuple(a[j] + t * (b[j] - a[j]) for j in (0, 1)))
        if fb <= 0:
            out.append(b)
        a, fa = b, fb
    return tuple(out)

def square(q=(0, 0), r=1800):
    x, y = map(rat, q)
    r = rat(r)
    return ((x - r, y - r), (x + r, y - r), (x + r, y + r), (x - r, y + r))

def rectangle_clip(poly, q, r):
    for n in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        poly = clip(poly, n, dot(n, q) + r)
    return poly

def interval_halfplane(poly, q, nx, ny, bound=0):
    n = ((nx[0] + nx[1]) / 2, (ny[0] + ny[1]) / 2)
    slack = (nx[1] - nx[0]) / 2 * (1800 + abs(q[0])) + (ny[1] - ny[0]) / 2 * (
        1800 + abs(q[1])
    )
    return clip(poly, n, dot(n, q) + bound + slack)

def wedge(poly, q, bearing, half=F(201, 200)):
    lower_s, lower_c = trig(rat(bearing) - half)
    upper_s, upper_c = trig(rat(bearing) + half)
    poly = interval_halfplane(poly, q, lower_s, neg(lower_c))
    return interval_halfplane(poly, q, neg(upper_s), upper_c)

def positive_polygon(poly, q, obs):
    q = tuple(map(rat, q))
    if obs['kind'] == 'near':
        return rectangle_clip(poly, q, 5)
    if obs['kind'] != 'direction':
        raise ValueError('positive observation required')
    bearing = F(str(obs['bearing']))
    poly = wedge(poly, q, bearing)
    s, c = trig(bearing)
    poly = interval_halfplane(poly, q, c, s, 1500)
    poly = interval_halfplane(poly, q, neg(c), neg(s), 0)
    transverse = 1500 * trig(F(201, 200))[0][1]
    poly = interval_halfplane(poly, q, neg(s), c, transverse)
    poly = interval_halfplane(poly, q, s, neg(c), transverse)
    return rectangle_clip(poly, q, 1500)

def certified_cover(poly, bearing=0, failed=(), allow_empty=False):
    if not poly:
        raise ValueError('empty geometry is inconsistency, never completion')
    s, c = trig(F(str(bearing)))
    sn, cs = sum(s) / 2, sum(c) / 2
    u, v = (cs, sn), (-sn, cs)
    scale = cs * cs + sn * sn
    xs, ys = [dot(u, p) for p in poly], [dot(v, p) for p in poly]
    lo, hi, bot, top = min(xs), max(xs), min(ys), max(ys)
    nx, ny = max(1, -(-(hi - lo) // 25)), max(1, -(-(top - bot) // 25))

    def world(x, y):
        return ((cs * x - sn * y) / scale, (sn * x + cs * y) / scale)

    cells, skipped = [], 0
    for i in range(nx):
```

```
        for j in (range(ny) if i % 2 == 0 else range(ny - 1, -1, -1)):
            xl, xh = lo + i * (hi - lo) / nx, lo + (i + 1) * (hi - lo) / nx
            yl, yh = bot + j * (top - bot) / ny, bot + (j + 1) * (top - bot) / ny
            vertices = tuple(
                world(x, y) for x, y in ((xl, yl), (xh, yl), (xh, yh), (xl, yh))
            )
            if any(all(dist2(z, q) <= 400 for z in vertices) for q in failed):
                skipped += 1
                continue
            center = tuple(float(x) for x in world((xl + xh) / 2, (yl + yh) / 2))
            assert all(dist2(z, center) <= 400 for z in vertices)
            cells.append((center, vertices))
    if not cells and not allow_empty:
        raise ValueError('all cells excluded: inconsistent observations')
    return cells, {
        'cells': len(cells),
        'excluded_cells': skipped,
        'exact_rational_cell_coverage': True,
        'report_enclosure_model': 'closed_1p01',
        'report_enclosure_half_deg': 1.01,
    }

def q4_grid():
    return tuple(
        (float(x), float(y))
        for i, x in enumerate(range(-1800, 1801, 720))
        for y in (range(-1800, 1801, 720) if i % 2 == 0 else range(1800, -1801, -720))
    )

def verify_q4_grid():
    points = set(q4_grid())
    assert len(points) == 36
    assert (3600 // 720) ** 2 == 25
    for x in range(-1800, 1800, 720):
        for y in range(-1800, 1800, 720):
            assert {(x, y), (x + 720, y), (x, y + 720), (x + 720, y + 720)} <= points
    assert 5 * 720**2 < 4 * 1000**2
    assert 5 * 900**2 > 4 * 1000**2
    return {
        'points': 36,
        'max_required_distance_squared_m2': 648000,
        'receive_radius_squared_m2': 1000000,
        'proof': 'reflected source is in the right triangle conv{(0,0),(s,0),(0,s)}; '
        'every closed halfplane through it holds a cell corner within 5s^2/4',
        'exact_integer_checks': True,
    }
```

joint.py

```
# 联合路线与清除规划

from dataclasses import dataclass, field
from fractions import Fraction as F
from pathlib import Path
import math
import sys

sys.path.append(str(Path(__file__).resolve().parent.parent))
import core
```

```
import improvement
import certified as cert
from physical import Belief
from q4_pair import symmetric_pair

def action(kind, q, channel, purpose, cell=None):
    a = {
        'kind': kind,
        'position': tuple(map(float, q)),
        'channel': channel,
        'purpose': purpose,
    }
    if cell is not None:
        a['cell'] = cell
    return a

def cost(plan, position, channel, active=0):
    value = 2 * active
    for a in plan:
        value += math.dist(position, a['position']) / 5
        if a['kind'] == 'measure':
            value += 5 + int(channel != a['channel'])
            channel = a['channel']
        else:
            value += 3
        position = a['position']
    return value

def exact_cost(plan, position, channel, active=0):
    lo = hi = F(2 * active)
    for a in plan:
        length = cert.sqrt_bounds(cert.dist2(position, a['position']))
        fee = 3
        if a['kind'] == 'measure':
            fee = 5 + int(channel != a['channel'])
            channel = a['channel']
        lo += length[0] / 5 + fee
        hi += length[1] / 5 + fee
        position = a['position']
    return lo, hi

def promote(plan, index):
    return [plan[index]] + plan[:index] + plan[index + 1 :]

def difference_bounds(new, old, position, channel):
    terms = {}
    fees = 0
    for route, sign in ((new, 1), (old, -1)):
        p, ch = position, channel
        for a in route:
            d2 = cert.dist2(p, a['position'])
            terms[d2] = terms.get(d2, 0) + sign
            if a['kind'] == 'measure':
                fees += sign * (5 + int(ch != a['channel']))
                ch = a['channel']
            else:
                fees += sign * 3
            p = a['position']
    lo = hi = F(fees)
    for d2, coefficient in terms.items():
        if coefficient:
```

```
            a, b = cert.sqrt_bounds(d2)
            lo += min(coefficient * a, coefficient * b) / 5
            hi += max(coefficient * a, coefficient * b) / 5
    return lo, hi

def travel_delta(plan, index, position):
    if index == 0:
        return F(0), F(0)
    q = plan[index]['position']
    before = plan[index - 1]['position']
    old_edges = [(position, plan[0]['position']), (before, q)]
    new_edges = [(position, q), (q, plan[0]['position'])]
    if index + 1 < len(plan):
        after = plan[index + 1]['position']
        old_edges.append((q, after))
        new_edges.append((before, after))
    lower = upper = F(0)
    for a, b in new_edges:
        lo, hi = cert.sqrt_bounds(cert.dist2(a, b))
        lower += lo
        upper += hi
    for a, b in old_edges:
        lo, hi = cert.sqrt_bounds(cert.dist2(a, b))
        lower -= hi
        upper -= lo
    return lower / 5, upper / 5

@dataclass
class SourceTask:
    belief: Belief
    bearing: float
    extra: int = 0
    measurement_tried_version: int = -1
    attempts: int = 0

@dataclass
class Controller:
    q4: bool = True
    joint: bool = True
    measurements: bool = True
    failed_pruning: bool = True
    depth: int = 1
    beam: int = 4
    position: tuple = (0.0, 0.0)
    channel: int = 1
    spent: float = 0.0
    tasks: dict = field(default_factory=dict)
    cleared: set = field(default_factory=set)
    excluded: set = field(default_factory=set)
    pending: dict = None
    saved_branches: object = None
    forced: object = None
    metrics: dict = field(
        default_factory=lambda: {
            'reorders': 0,
            'interleavings': 0,
            'measure_guards': 0,
            'measure_rejects': 0,
            'pruned_cells': 0,
            'certified_covers': 0,
            'max_active': 0,
```

```
            'ledger_checks': 0,
        }
    )

    def __post_init__(self):
        if self.depth not in (1, 2, 3) or self.beam < 1:
            raise ValueError('invalid lookahead configuration')
        points = cert.q4_grid() if self.q4 else core.POINTS
        self.plan = []
        current = 1
        for q in points:
            channels = [current] + [c for c in range(1, 21) if c != current]
            self.plan.extend(action('measure', q, c, 'discovery') for c in channels)
            current = channels[-1]
        self.initial_search_bound = float(exact_cost(self.plan, self.position, 1)[1])
        self.service_reserve = 0.0
        self.history = []
        self.beliefs = {c: Belief(self.q4) for c in range(1, 21)}

    def cover_plan(self, belief, bearing, channel):
        cells, meta = cert.certified_cover(
            belief.poly, bearing, belief.failed if self.failed_pruning else ()
        )
        if self.failed_pruning:
            kept = [
                (center, vertices)
                for center, vertices in cells
                if not belief.cell_impossible(vertices)
            ]
            self.metrics['pruned_cells'] += len(cells) - len(kept)
            cells = kept
            if not cells:
                raise ValueError('physical constraints exclude entire source cover')
        self.metrics['certified_covers'] += 1
        return [
            action('clear', center, channel, 'cover', vertices)
            for center, vertices in cells
        ]

    def candidates(self, plan, position):
        # 搜索有固定完整顺序，清除单元在源间独立
        # 源内同理，只有成功清除才移除同组
        cs = [i for i, a in enumerate(plan) if a['kind'] == 'clear']
        nearest = sorted(
            cs, key=lambda i: (math.dist(position, plan[i]['position']), i)
        )[:12]
        first_search = next(
            (i for i, a in enumerate(plan) if a['purpose'] == 'discovery'), 0
        )
        return list(dict.fromkeys([0, first_search] + [i for i in nearest if i]))

    def reorder(self):
        if not self.joint or len(self.plan) < 2:
            return
        # Beam 节点保留完整尾部，不使用真实源或预测
        # 采用确定性全失败路线代理和成功预留
        nodes = [(0.0, [], self.plan, self.position, self.channel)]
        best_score = cost(self.plan, self.position, self.channel)
        best = self.plan
        for _ in range(self.depth):
            expanded = []
```

```
            for prefix_cost, prefix, tail, p, ch in nodes:
                if not tail:
                    continue
                for i in self.candidates(tail, p):
                    route = promote(tail, i)
                    a, rest = route[0], route[1:]
                    step = cost([a], p, ch)
                    ch2 = a['channel'] if a['kind'] == 'measure' else ch
                    score = prefix_cost + step + cost(rest, a['position'], ch2)
                    full = prefix + route
                    if score < best_score - 1e-6 or (
                        abs(score - best_score) < 1e-8
                        and full[0]['purpose'] == 'discovery'
                        and best[0]['purpose'] != 'discovery'
                    ):
                        best_score, best = score, full
                    expanded.append(
                        (
                            score,
                            prefix_cost + step,
                            prefix + [a],
                            rest,
                            a['position'],
                            ch2,
                        )
                    )
            nodes = [
                (v, p, t, q, c)
                for _, v, p, t, q, c in sorted(expanded, key=lambda n: n[0])[
                    : self.beam
                ]
            ]
        if best is not self.plan:
            # 完整精确有理距离界验证启发式改进
            if difference_bounds(best, self.plan, self.position, self.channel)[1] <= 0:
                self.plan = best
                self.metrics['reorders'] += 1

    def try_measure(self, c):
        task = self.tasks[c]
        if (
            not self.measurements
            or task.attempts >= 3
            or task.extra >= 3
            or task.measurement_tried_version == task.belief.version
        ):
            return None
        task.measurement_tried_version = task.belief.version
        task.attempts += 1
        if sum(a['channel'] == c and a['kind'] == 'clear' for a in self.plan) <= 1:
            return None
        poly = tuple(tuple(map(float, p)) for p in task.belief.poly)
        incumbent = core.cover(poly, self.position, self.position)
        # 仅快速提议，下面完整有理树才是权威
        proposal, _ = improvement.choose_measure(
            poly, self.position, self.position, incumbent, max_splits=64
        )
        if proposal is None:
            if self.q4:
                return self.try_pair(
                    c,
```

```
                    exact_cost(self.plan, self.position, self.channel, len(self.tasks))[
                        0
                    ],
                )
            return None
        q = proposal[0]
        rest = [
            a for a in self.plan if not (a['kind'] == 'clear' and a['channel'] == c)
        ]
        branches = {}
        worst = F(0)
        for lo in range(0, 360, 10):
            p = cert.wedge(
                task.belief.poly,
                tuple(map(cert.rat, q)),
                F(lo + 5),
                F(5) + task.belief.report_half,
            )
            if p:
                cells, _ = cert.certified_cover(
                    p,
                    lo + 5,
                    task.belief.failed if self.failed_pruning else (),
                    allow_empty=True,
                )
                if not cells:
                    continue
                tail = [
                    action('clear', center, c, 'cover', vertices)
                    for center, vertices in cells
                ] + rest
                branches[('direction', lo // 10)] = tail
        near = cert.rectangle_clip(task.belief.poly, tuple(map(cert.rat, q)), 5)
        if near:
            cells, _ = cert.certified_cover(near, 0)
            branches[('near', 0)] = [
                action('clear', center, c, 'cover', vertices)
                for center, vertices in cells
            ] + rest

        if self.q4 or any(cert.dist2(p, q) > 1000**2 for p in task.belief.poly):
            branches[('no_signal', 0)] = list(self.plan)
        m = action('measure', q, c, 'localize')
        for tail in branches.values():
            worst = max(
                worst,
                exact_cost([m] + tail, self.position, self.channel, len(self.tasks))[1],
            )
        current_lower = exact_cost(
            self.plan, self.position, self.channel, len(self.tasks)
        )[0]
        if branches and worst < current_lower:
            self.saved_branches = branches
            self.metrics['measure_guards'] += 1
            return m
        self.metrics['measure_rejects'] += 1
        if self.q4:
            return self.try_pair(c, current_lower)
        return None

    def try_pair(self, c, current_lower):
```

```
        task = self.tasks[c]
        positive = next(
            (
                q
                for q, o, a in task.belief.events
                if a == 'measure' and o['kind'] in ('near', 'direction')
            ),
            None,
        )
        if positive is None or task.extra > 1:
            return None
        rest = [
            a for a in self.plan if not (a['kind'] == 'clear' and a['channel'] == c)
        ]
        for offset in ((0, 200), (200, 0), (0, 100), (100, 0)):
            pair = symmetric_pair(task.belief.poly, positive, offset)
            if not pair['certified']:
                continue

            def positive_tails(q):
                tails = {}
                for lo in range(0, 360, 2):
                    p = cert.wedge(
                        task.belief.poly,
                        tuple(map(cert.rat, q)),
                        F(lo + 1),
                        F(1) + task.belief.report_half,
                    )
                    if not p:
                        continue
                    cells, _ = cert.certified_cover(
                        p, lo + 1, task.belief.failed, allow_empty=True
                    )
                    if cells:
                        tails[('direction', lo, lo + 2)] = [
                            action('clear', center, c, 'cover', vertices)
                            for center, vertices in cells
                        ] + rest
                p = cert.rectangle_clip(task.belief.poly, tuple(map(cert.rat, q)), 5)
                if p:
                    cells, _ = cert.certified_cover(
                        p, 0, task.belief.failed, allow_empty=True
                    )
                    if cells:
                        tails[('near', 0)] = [
                            action('clear', center, c, 'cover', vertices)
                            for center, vertices in cells
                        ] + rest
                return tails

            first = action('measure', pair['plus'], c, 'localize')
            second = action('measure', pair['minus'], c, 'localize')
            first_tails = positive_tails(pair['plus'])
            second_tails = positive_tails(pair['minus'])
            if not first_tails or not second_tails:
                continue
            values = [
                exact_cost(
                    [first] + tail, self.position, self.channel, len(self.tasks)
                )[1]
                for tail in first_tails.values()
```

```
            ]
            values += [
                exact_cost(
                    [first, second] + tail, self.position, self.channel, len(self.tasks)
                )[1]
                for tail in second_tails.values()
            ]
            if max(values) < current_lower:
                first_tails[('no_signal', 0)] = {
                    'action': second,
                    'branches': second_tails,
                }
                self.saved_branches = first_tails
                self.metrics['paired_measure_guards'] = (
                    self.metrics.get('paired_measure_guards', 0) + 1
                )
                return first
        return None

    def next_action(self):
        if self.pending is not None:
            return self.pending  # 调用方在收到回执前不得创建新动作
        if self.forced is not None:
            self.pending = self.forced['action']
            self.saved_branches = self.forced['branches']
            self.forced = None
            return self.pending
        if not self.plan:
            return None
        self.reorder()
        chosen = self.plan[0]
        if chosen['kind'] == 'clear':
            chosen = self.try_measure(chosen['channel']) or chosen
        self.pending = chosen
        return chosen

    def accept(self, observation):
        if self.pending is None:
            raise ValueError('no pending action')
        a = self.pending
        if observation.get('kind') not in (
            ('near', 'direction', 'no_signal')
            if a['kind'] == 'measure'
            else ('success', 'no_target_in_range')
        ):
            raise ValueError('unknown is not confirmed feedback')
        c, q = a['channel'], a['position']
        self.spent += cost([a], self.position, self.channel) + (
            2 if observation['kind'] == 'success' else 0
        )
        if a['kind'] == 'measure':
            self.channel = c
        if self.history and self.history[-1]['channel'] != c and c in self.tasks:
            self.metrics['interleavings'] += 1
        self.position = q
        if a['purpose'] == 'localize':
            if observation['kind'] == 'direction':
                bearing = observation['bearing'] % 360
                matches = [
                    k
                    for k in self.saved_branches
```

```
                    if k[0] == 'direction'
                    and (
                        (len(k) == 2 and k[1] == int(bearing // 10))
                        or (len(k) == 3 and k[1] <= bearing < k[2])
                    )
                ]
                if len(matches) != 1:
                    raise ValueError('direction not uniquely covered')
                key = matches[0]
            else:
                key = (observation['kind'], 0)
            if key not in self.saved_branches:
                raise ValueError('observation outside saved tree')
            child = self.saved_branches[key]
            if isinstance(child, dict):
                self.forced = child
            else:
                self.plan = child
            task = self.tasks[c]
            task.belief.observe(q, observation)
            task.extra += 1
            self.saved_branches = None
        else:
            assert self.plan[0] == a
            self.plan.pop(0)
            if a['purpose'] == 'discovery':
                belief = self.beliefs[c]
                belief.observe(q, observation)
                if observation['kind'] == 'no_signal':
                    if not any(x['channel'] == c for x in self.plan):
                        self.excluded.add(c)
                else:
                    bearing = observation.get('bearing', 0.0)
                    task = SourceTask(belief, bearing)
                    self.tasks[c] = task
                    self.plan = [x for x in self.plan if x['channel'] != c]
                    clear = self.cover_plan(belief, bearing, c)
                    reserve = exact_cost(
                        clear + [action('measure', q, c, 'accounting-only')], q, c, 1
                    )[1]
                    self.service_reserve += float(reserve)
                    self.plan = clear + self.plan
            else:
                task = self.tasks[c]
                task.belief.observe(q, observation, 'clear')
                if observation['kind'] == 'success':
                    self.cleared.add(c)
                    del self.tasks[c]
                    self.plan = [x for x in self.plan if x['channel'] != c]
                elif self.failed_pruning:
                    retained = []
                    for x in self.plan:
                        if (
                            x['kind'] == 'clear'
                            and x['channel'] == c
                            and all(cert.dist2(p, q) <= 400 for p in x['cell'])
                        ):
                            self.metrics['pruned_cells'] += 1
                        else:
                            retained.append(x)
                    self.plan = retained
```

```
        self.history.append(
            {
                'kind': a['kind'],
                'position': q,
                'channel': c,
                'purpose': a['purpose'],
                'observation': dict(observation),
            }
        )
        self.pending = None
        self.metrics['max_active'] = max(self.metrics['max_active'], len(self.tasks))
        self.metrics['ledger_checks'] += 1
        remaining = cost(self.plan, self.position, self.channel, len(self.tasks))
        if self.forced is not None:
            remaining = max(
                float(
                    exact_cost(
                        [self.forced['action']] + tail,
                        self.position,
                        self.channel,
                        len(self.tasks),
                    )[1]
                )
                for tail in self.forced['branches'].values()
            )
        ledger = self.spent + remaining
        assert ledger <= self.initial_search_bound + self.service_reserve + 1e-4

    def completed(self):
        return (
            self.pending is None
            and self.forced is None
            and not self.plan
            and not self.tasks
            and self.cleared | self.excluded == set(range(1, 21))
        )
```

optimization.py

```
# 清除位置优化

from fractions import Fraction as F
import heapq
import itertools

import certified as cert

def center_bounds(vertices, box):
    center = ((box[0] + box[1]) / 2, (box[2] + box[3]) / 2)
    upper = max(cert.dist2(p, center) for p in vertices)
    lower = max(
        sum(
            (max(lo - x, F(0), x - hi)) ** 2
            for x, lo, hi in ((p[0], box[0], box[1]), (p[1], box[2], box[3]))
        )
        for p in vertices
    )
    return lower, upper, center
```

```
def clear_center(vertices, tolerance_m=F(1, 100), max_splits=10000):
    vertices = tuple(tuple(map(cert.rat, p)) for p in vertices)
    if not vertices or tolerance_m <= 0 or max_splits < 0:
        raise ValueError('invalid center optimization inputs')
    box = (
        min(p[0] for p in vertices),
        max(p[0] for p in vertices),
        min(p[1] for p in vertices),
        max(p[1] for p in vertices),
    )
    counter = itertools.count()
    lower, best, point = center_bounds(vertices, box)
    heap = [(lower, next(counter), box)]
    splits = 0
    while heap:
        lower = heap[0][0]
        gap = cert.sqrt_bounds(best)[1] - cert.sqrt_bounds(lower)[0]
        if gap <= tolerance_m or splits >= max_splits:
            break
        _, _, box = heapq.heappop(heap)
        axis = 0 if box[1] - box[0] >= box[3] - box[2] else 2
        mid = (box[axis] + box[axis + 1]) / 2
        for low, high in ((box[axis], mid), (mid, box[axis + 1])):
            child = list(box)
            child[axis : axis + 2] = [low, high]
            lb, ub, q = center_bounds(vertices, child)
            if ub < best:
                best, point = ub, q
            if lb <= best:
                heapq.heappush(heap, (lb, next(counter), tuple(child)))
        splits += 1
    lower = min(best, heap[0][0]) if heap else best
    command = tuple(map(float, point))
    actual_upper2 = max(cert.dist2(command, p) for p in vertices)
    lo, hi = cert.sqrt_bounds(lower)[0], cert.sqrt_bounds(actual_upper2)[1]
    return {
        'point': command,
        'radius_lower': float(lo),
        'radius_upper': float(hi),
        'gap_upper_m': float(hi - lo),
        'splits': splits,
        'epsilon_optimal_for_polygon': hi - lo <= tolerance_m,
        'guaranteed_clear_actual_command': actual_upper2 <= 400,
        'scope': 'continuous clear-center for supplied rational polygon only',
        'full_continuous_policy_optimal': False,
    }

def finite_minimax(state, actions, transition, terminal, depth):
    best = terminal(state)
    policy = ('fallback',)
    visited = 1
    if depth == 0:
        return best, policy, visited
    for action in actions(state):
        branches = list(transition(state, action))
        if not branches:
            raise ValueError('action lacks complete feedback branches')
        score = F(0)
        children = []
        for label, child, cost in branches:
            value, subpolicy, nodes = finite_minimax(
```

```
                child, actions, transition, terminal, depth - 1
            )
            visited += nodes
            score = max(score, cost + value)
            children.append((label, subpolicy))
        if score < best:
            best, policy = score, (action, tuple(children))
    return best, policy, visited

def depth_witness():
    fallback = {0: F(100), 1: F(100), 2: F(5)}

    def actions(s):
        return ('measure',) if s < 2 else ()

    def transition(s, a):
        return [('left', s + 1, F(10)), ('right', s + 1, F(10))]

    return [
        {
            'depth': d,
            'value_s': float(
                finite_minimax(0, actions, transition, fallback.__getitem__, d)[0]
            ),
        }
        for d in (0, 1, 2, 3)
    ]
```

physical.py

```
# 物理模型与信念

from dataclasses import dataclass, field
from fractions import Fraction as F
import math

import certified as cert

def interval(x):
    return cert.rat(x), cert.rat(x)

def sub(a, b):
    return cert.add(a, cert.neg(b))

def sq(a):
    return (
        (F(0) if a[0] <= 0 <= a[1] else min(a[0] ** 2, a[1] ** 2)),
        max(a[0] ** 2, a[1] ** 2),
    )

def le(a, b=(F(0), F(0))):
    if a[0] > b[1]:
        return -1
    return 1 if a[1] <= b[0] else 0

def gt(a, b=(F(0), F(0))):
    return -le(a, b)

def conjunction(values):
```

```
    return min(values)

def disjunction(values):
    return max(values)

def trig_range(a):
    mid = (a[0] + a[1]) / 2
    s, c = cert.trig(mid)
    radius = (a[1] - a[0]) / 2 * cert.pi_interval()[1] / 180
    return tuple((max(F(-1), t[0] - radius), min(F(1), t[1] + radius)) for t in (s, c))

@dataclass
class Belief:
    report_half = F(201, 200)
    q4: bool = True
    events: list = field(default_factory=list)
    measured: dict = field(default_factory=dict)
    poly: tuple = field(default_factory=cert.square)
    failed: list = field(default_factory=list)
    version: int = 0
    inconsistent: bool = False

    def observe(self, q, obs, action='measure'):
        q = tuple(map(cert.rat, q))
        kind = obs.get('kind')
        allowed = (
            ('direction', 'near', 'no_signal')
            if action == 'measure'
            else ('success', 'no_target_in_range')
        )
        if kind not in allowed:
            raise ValueError('unknown receipt is not physical evidence')
        if kind == 'direction' and not math.isfinite(obs.get('bearing', math.nan)):
            raise ValueError('invalid direction')
        if action == 'measure':
            signature = (kind, F(str(obs['bearing'])) if kind == 'direction' else None)
            if q in self.measured and self.measured[q] != signature:
                self.inconsistent = True
                raise ValueError('spatially fixed measurement contradiction')
            self.measured[q] = signature
        self.events.append((q, dict(obs), action))
        if kind in ('near', 'direction'):
            self.poly = cert.positive_polygon(self.poly, q, obs)
        if kind == 'no_target_in_range':
            self.failed.append(q)
        self.version += 1
        if not self.poly:
            self.inconsistent = True
            raise ValueError('empty certified enclosure')

    def classify(self, box, directional):
        if self.inconsistent:
            return -1
        x, y, radius, angle = box
        values = [le(cert.add(sq(x), sq(y)), interval(1800**2))]
        sin, cos = trig_range(angle) if directional else (None, None)
        for q, obs, action in self.events:
            dx, dy = sub(interval(q[0]), x), sub(interval(q[1]), y)
            d2 = cert.add(sq(dx), sq(dy))
            if action == 'clear':
                values.append(
```

```
                    gt(d2, interval(400))
                    if obs['kind'] == 'no_target_in_range'
                    else le(d2, interval(400))
                )
                continue
            distance_ok = le(d2, sq(radius))
            visibility = (
                cert.add(cert.mul(cos, dx), cert.mul(sin, dy))
                if directional
                else interval(1)
            )
            visible = le(cert.neg(visibility))
            received = min(distance_ok, visible)
            if obs['kind'] == 'no_signal':
                values.append(-received)
            else:
                values.append(received)
                values.append(
                    le(d2, interval(25))
                    if obs['kind'] == 'near'
                    else gt(d2, interval(25))
                )
                if obs['kind'] == 'direction':
                    angle_value = F(str(obs['bearing']))
                    ls, lc = cert.trig(angle_value - self.report_half)
                    us, uc = cert.trig(angle_value + self.report_half)
                    # 探测点到源向量为 -(dx,dy)
                    values.append(
                        le(cert.add(cert.mul(cert.neg(ls), dx), cert.mul(lc, dy)))
                    )
                    values.append(
                        le(cert.add(cert.mul(us, dx), cert.mul(cert.neg(uc), dy)))
                    )
            if -1 in values:
                return -1
        return min(values)

    def roots(self):
        x = (min(p[0] for p in self.poly), max(p[0] for p in self.poly))
        y = (min(p[1] for p in self.poly), max(p[1] for p in self.poly))
        return [(False, (x, y, (F(1000), F(1500)), (F(0), F(0))))] + (
            [(True, (x, y, (F(1000), F(1500)), (F(0), F(360))))] if self.q4 else []
        )

    def pave(self, max_splits=128, widths=(25, 25, 25, 5)):
        pending = self.roots()
        kept, rejected, splits = [], 0, 0
        while pending:
            kind, box = pending.pop()
            state = self.classify(box, kind)
            if state == -1:
                rejected += 1
                continue
            axis = max(range(4), key=lambda j: (box[j][1] - box[j][0]) / widths[j])
            if (
                state == 1
                or box[axis][1] - box[axis][0] <= widths[axis]
                or splits >= max_splits
            ):
                kept.append((kind, box, state))
                continue
```

```
            mid = sum(box[axis]) / 2
            for limits in ((box[axis][0], mid), (mid, box[axis][1])):
                child = list(box)
                child[axis] = limits
                pending.append((kind, tuple(child)))
            splits += 1
        return kept, {
            'splits': splits,
            'rejected': rejected,
            'remaining': len(kept),
            'unresolved': sum(s == 0 for _, _, s in kept),
            'exact_set_solved': all(s == 1 for _, _, s in kept),
            'certified_outer_enclosure': True,
        }

    def truth_not_rejected(self, point, radius, direction=None):
        box = tuple(interval(z) for z in (*point, radius, direction or 0))
        return self.classify(box, direction is not None) != -1

    def cell_impossible(self, vertices):
        x = (min(v[0] for v in vertices), max(v[0] for v in vertices))
        y = (min(v[1] for v in vertices), max(v[1] for v in vertices))
        if cert.add(sq(x), sq(y))[0] > 1800**2:
            return True
        for q, obs, action in self.events:
            d2 = cert.add(sq(sub(x, interval(q[0]))), sq(sub(y, interval(q[1]))))
            if (
                action == 'clear'
                and obs['kind'] == 'no_target_in_range'
                and d2[1] <= 400
            ):
                return True
            if action == 'measure':
                if obs['kind'] == 'no_signal' and not self.q4 and d2[1] <= 1000**2:
                    return True
                if obs['kind'] in ('near', 'direction') and d2[0] > 1500**2:
                    return True
                if obs['kind'] == 'near' and d2[0] > 25:
                    return True
        return False
```

q4__pair.py

```


from fractions import Fraction as F
import math

import certified as cert

def symmetric_pair(poly, anchor, offset):
    anchor = tuple(map(cert.rat, anchor))
    # 先选命令浮点数，确保精确对称性经过转换后仍成立
    plus = tuple(float(anchor[j] + cert.rat(offset[j])) for j in (0, 1))
    minus = tuple(float(2 * anchor[j] - cert.rat(plus[j])) for j in (0, 1))
    exact = all(cert.rat(plus[j]) + cert.rat(minus[j]) == 2 * anchor[j] for j in (0, 1))
    received_range = all(
        cert.dist2(p, q) <= 1000**2 for p in poly for q in (plus, minus)
    )
```

```
    return {
        'plus': plus,
        'minus': minus,
        'certified': exact and received_range,
        'exact_command_symmetry': exact,
        'guaranteed_range': received_range,
        'forbidden_feedback_pair': (
            ['no_signal', 'no_signal'] if exact and received_range else None
        ),
    }

def audit():
    # 场地东边界附近，首个正回执的外多边形较短
    anchor = (1600.0, 0.0)
    poly = cert.positive_polygon(
        cert.square(), anchor, {'kind': 'direction', 'bearing': 0}
    )
    good = symmetric_pair(poly, anchor, (0, 100))
    long_poly = cert.positive_polygon(
        cert.square(), (0, 0), {'kind': 'direction', 'bearing': 0}
    )
    bad = symmetric_pair(long_poly, (0, 0), (0, 100))
    assert good['certified'] and not bad['certified']
    return {
        'short_prior': good,
        'long_prior': bad,
        'scope': 'conditional Q4 two-measurement visibility, not global policy optimality',
    }
```

refinement.py

```
# 精确支撑投影

from dataclasses import dataclass
from fractions import Fraction as F
import heapq
import itertools
import math

import certified as cert
from joint import Controller, action, exact_cost, difference_bounds
from optimization import clear_center
import core

def count_supports(domains, minimum=10, maximum=16):
    domains = tuple(frozenset(d) for d in domains)
    if not 0 <= minimum <= maximum <= len(domains):
        raise ValueError('invalid source-count bounds')
    if any(not d or not d <= {0, 1} for d in domains):
        raise ValueError('invalid or empty existence domain')
    lower, upper = sum(min(d) for d in domains), sum(max(d) for d in domains)
    if lower > maximum or upper < minimum:
        raise ValueError('source count inconsistent with confirmed observations')
    return tuple(
        frozenset(
            v
            for v in d
            if lower - min(d) + v <= maximum and upper - max(d) + v >= minimum
        )
```

```
        for d in domains
    )

def point_box_distance2(p, box):
    return sum(
        max(lo - cert.rat(x), F(0), cert.rat(x) - hi) ** 2
        for x, (lo, hi) in zip(p, box)
    )

def route_center(poly, start, following=None, max_splits=128, tolerance_s=F(1, 100)):
    poly = tuple(tuple(map(cert.rat, v)) for v in poly)
    if not poly or max_splits < 0 or tolerance_s <= 0:
        raise ValueError('invalid continuous query')
    domain = tuple(
        (max(v[j] - 20 for v in poly), min(v[j] + 20 for v in poly)) for j in (0, 1)
    )
    if any(lo > hi for lo, hi in domain):
        return {'status': 'empty', 'reason': 'disjoint necessary coordinate intervals'}
    if any(cert.dist2(a, b) > 1600 for a in poly for b in poly):
        return {'status': 'empty', 'reason': 'diameter witness exceeds 40 m'}

    def feasible(q):
        return all(cert.dist2(q, v) <= 400 for v in poly)

    def lower(box):
        return (
            cert.sqrt_bounds(point_box_distance2(start, box))[0]
            + (
                cert.sqrt_bounds(point_box_distance2(following, box))[0]
                if following is not None
                else 0
            )
        ) / 5

    def upper(q):
        return (
            cert.sqrt_bounds(cert.dist2(start, q))[1]
            + (
                cert.sqrt_bounds(cert.dist2(q, following))[1]
                if following is not None
                else 0
            )
        ) / 5

    candidates = [
        tuple(map(float, start)),
        tuple(float((a + b) / 2) for a, b in domain),
    ]
    if following is not None:
        candidates.append(tuple(map(float, following)))
    # 半径中心只是可行种子，不是行程目标
    seed = clear_center(poly, max_splits=min(max_splits, 128))
    if seed['guaranteed_clear_actual_command']:
        candidates.append(seed['point'])
    best_q, best = None, None
    for q in candidates:
        if feasible(q) and (best is None or upper(q) < best):
            best_q, best = q, upper(q)
    serial = itertools.count()
    heap = [(lower(domain), next(serial), domain)]
    splits, excluded = 0, 0
```

```
    while heap and splits < max_splits:
        if best is not None and best - heap[0][0] <= tolerance_s:
            break
        lb, _, box = heapq.heappop(heap)
        if best is not None and lb > best:
            continue
        axis = max(range(2), key=lambda j: box[j][1] - box[j][0])
        mid = sum(box[axis]) / 2
        if box[axis][0] == box[axis][1]:
            heapq.heappush(heap, (lb, next(serial), box))
            break
        for limits in ((box[axis][0], mid), (mid, box[axis][1])):
            child = list(box)
            child[axis] = limits
            child = tuple(child)
            if any(point_box_distance2(v, child) > 400 for v in poly):
                excluded += 1
                continue
            q = tuple(float(sum(x) / 2) for x in child)
            if feasible(q) and (best is None or upper(q) < best):
                best_q, best = q, upper(q)
            bound = lower(child)
            if best is None or bound <= best:
                heapq.heappush(heap, (bound, next(serial), child))
        splits += 1
    if best is None:
        return {
            'status': 'undetermined' if heap else 'empty',
            'splits': splits,
            'unresolved_boxes': len(heap),
            'reason': 'no feasible command found within search',
        }
    lb = min(best, heap[0][0]) if heap else best
    return {
        'status': 'feasible',
        'point': best_q,
        'lower_s': str(lb),
        'upper_s': str(best),
        'gap_s': float(best - lb),
        'epsilon_optimal_for_clear_travel': best - lb <= tolerance_s,
        'splits': splits,
        'excluded_boxes': excluded,
        'scope': 'one guaranteed clear with fixed preceding and following positions',
    }

def prune_physical_cells(belief, cells, budget=32):
    boxes, meta = belief.pave(budget)
    if not boxes:
        raise ValueError('physical evidence inconsistent: no source state remains')
    spatial = [(b[0], b[1]) for _, b, _ in boxes]
    kept = []
    for a in cells:
        bounds = tuple(
            (min(v[j] for v in a['cell']), max(v[j] for v in a['cell'])) for j in (0, 1)
        )
        if any(
            all(x[0] <= y[1] and y[0] <= x[1] for x, y in zip(bounds, b))
            for b in spatial
        ):
            kept.append(a)
    if not kept:
```

```
        raise ValueError('no certified source cell remains')
    return kept, meta

@dataclass
class ReviewedController(Controller):
    use_count: bool = True
    use_center: bool = True
    use_discovery_dp: bool = True
    use_physical_queries: bool = False
    q4_pair_only: bool = True
    center_splits: int = 64
    minimum_sources: int = 10
    maximum_sources: int = 16

    def __post_init__(self):
        super().__post_init__()
        self.domains = tuple(frozenset((0, 1)) for _ in range(20))
        self.center_versions = {}
        self.discovery_dirty = True
        self.fault = None
        self.metrics.update(
            count_exclusions=0,
            count_forced_present=0,
            center_queries=0,
            center_adoptions=0,
            discovery_dp_queries=0,
            discovery_dp_adoptions=0,
            physical_queries=0,
            physical_cell_exclusions=0,
            dominated_q4_single_skipped=0,
        )

    def cover_plan(self, belief, bearing, channel):
        cells = super().cover_plan(belief, bearing, channel)
        if self.use_physical_queries:
            kept, _ = prune_physical_cells(belief, cells)
            self.metrics['physical_queries'] += 1
            self.metrics['physical_cell_exclusions'] += len(cells) - len(kept)
            return kept
        return cells

    def update_count(self):
        if not self.use_count:
            return
        domains = list(self.domains)
        for c in self.cleared | set(self.tasks):
            domains[c - 1] = domains[c - 1] & {1}
        for c in self.excluded:
            domains[c - 1] = domains[c - 1] & {0}
        contracted = count_supports(domains, self.minimum_sources, self.maximum_sources)
        new_excluded = {
            c for c, d in enumerate(contracted, 1) if d == {0}
        } - self.excluded
        self.metrics['count_exclusions'] += len(new_excluded)
        self.metrics['count_forced_present'] += sum(
            a == {0, 1} and b == {1} for a, b in zip(domains, contracted)
        )
        self.domains = contracted
        self.excluded.update(new_excluded)
        self.plan = [a for a in self.plan if a['channel'] not in new_excluded]
```

```
    def discovery_dp(self):
        if (
            not self.use_discovery_dp
            or self.q4
            or self.tasks
            or not self.discovery_dirty
        ):
            return
        self.discovery_dirty = False
        if not self.plan:
            return
        required = [set() for _ in core.POINTS]
        for a in self.plan:
            if a['purpose'] != 'discovery':
                return
            required[core.POINTS.index(a['position'])].add(a['channel'])
        route, meta = core.schedule(required, self.position, self.channel)
        self.metrics['discovery_dp_queries'] += 1
        proposal = [
            action('measure', core.POINTS[i], c, 'discovery')
            for i, cs in route
            for c in cs
        ]
        if difference_bounds(proposal, self.plan, self.position, self.channel)[1] <= 0:
            self.plan = proposal
            self.metrics['discovery_dp_adoptions'] += 1

    def adopt_center(self, channel):
        task = self.tasks[channel]
        if (
            not self.use_center
            or self.center_versions.get(channel) == task.belief.version
        ):
            return
        self.center_versions[channel] = task.belief.version
        poly = task.belief.poly
        # 廉价必要条件避免对条带做连续求解
        if any(cert.dist2(a, b) > 1600 for a in poly for b in poly):
            return
        rest = [
            a
            for a in self.plan
            if not (a['kind'] == 'clear' and a['channel'] == channel)
        ]
        following = rest[0]['position'] if rest else None
        result = route_center(poly, self.position, following, self.center_splits)
        self.metrics['center_queries'] += 1
        if result['status'] != 'feasible':
            return
        proposal = [action('clear', result['point'], channel, 'cover', poly)] + rest
        if difference_bounds(proposal, self.plan, self.position, self.channel)[1] <= 0:
            self.plan = proposal
            self.metrics['center_adoptions'] += 1

    def try_measure(self, c):
        if not self.q4 or not self.q4_pair_only:
            return super().try_measure(c)
        task = self.tasks[c]
        if (
            not self.measurements
            or task.attempts >= 3
```

```
            or task.extra >= 3
            or task.measurement_tried_version == task.belief.version
        ):
            return None
        task.measurement_tried_version = task.belief.version
        task.attempts += 1
        if sum(a['channel'] == c and a['kind'] == 'clear' for a in self.plan) <= 1:
            return None
        # 在归档的 no_signal 尾部不变时，单次测量至少
        # 多花 5 秒：行程和切频满足三角不等式
        # 该定理不排斥不同后验尾部之后的感知
        self.metrics['dominated_q4_single_skipped'] += 1
        return self.try_pair(
            c, exact_cost(self.plan, self.position, self.channel, len(self.tasks))[0]
        )

    def next_action(self):
        if self.fault:
            raise ValueError(self.fault)
        if self.pending is not None or self.forced is not None:
            return super().next_action()
        self.discovery_dp()
        if not self.plan:
            return None
        self.reorder()
        if self.plan[0]['kind'] == 'clear':
            self.adopt_center(self.plan[0]['channel'])
        chosen = self.plan[0]
        if chosen['kind'] == 'clear':
            chosen = self.try_measure(chosen['channel']) or chosen
        self.pending = chosen
        return chosen

    def accept(self, observation):
        # 归档接受在验证数值示向度前先改账本
        # 先校验畸形回执，矛盾时安全失败
        if observation.get('kind') == 'direction':
            bearing = observation.get('bearing')
            if (
                isinstance(bearing, bool)
                or not isinstance(bearing, (int, float))
                or not math.isfinite(bearing)
            ):
                raise ValueError('invalid direction receipt; state unchanged')
        if self.fault:
            raise ValueError(self.fault)
        a = self.pending
        if a is None:
            raise ValueError('no pending action')
        try:
            super().accept(observation)
            self.update_count()
        except ValueError as exc:
            # 未知回执可重试，不消耗待处理动作
            if observation.get('kind') not in ('unknown', None):
                self.fault = str(exc)
            raise
        if observation['kind'] in ('success', 'near', 'direction') or (
            a['purpose'] == 'discovery'
            and not any(
                x['purpose'] == 'discovery' and x['position'] == a['position']
```

```
                for x in self.plan
            )
        ):
            self.discovery_dirty = True

    def completed(self):
        return not self.fault and super().completed()
```

completion.py

```
# 完成判定

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class CompletionDecision:
    status: str
    residual_channels: tuple
    evidence: tuple
    state_version: int
    fault: str | None

def decide_completion(
    *,
    cleared,
    excluded,
    active_channels,
    pending=False,
    fault=None,
    state_version=0,
    minimum_sources=10,
    maximum_sources=16,
    no_active_proof=False,
):
    cleared = set(map(int, cleared))
    excluded = set(map(int, excluded))
    active = set(map(int, active_channels))
    residual = tuple(sorted(set(range(1, 21)) - cleared - excluded))
    if fault:
        return CompletionDecision(
            'undetermined', residual, ('fault',), state_version, str(fault)
        )
    if pending:
        return CompletionDecision(
            'undetermined', residual, ('pending',), state_version, None
        )
    if not 1 <= minimum_sources <= maximum_sources <= 20:
        return CompletionDecision(
            'undetermined', residual, ('invalid_source_bounds',), state_version, None
        )
    if not residual:
        return CompletionDecision(
            'completed', (), ('all_channels_cleared_or_excluded',), state_version, None
        )
    if len(cleared) >= maximum_sources and not active:
        return CompletionDecision(
            'completed', (), ('maximum_source_count_cleared',), state_version, None
```

```
        )
    if no_active_proof and not active:
        return CompletionDecision(
            'completed', (), ('no_active_source_proof',), state_version, None
        )
    return CompletionDecision(
        'incomplete', residual, ('residual_responsibility',), state_version, None
    )
```

controller.py

```
# 正式控制器：协议、账本、完成判定与动作执行

from __future__ import annotations

from dataclasses import asdict
from fractions import Fraction as F
import hashlib
from pathlib import Path
import json
import math
import sys
import time

BASE = Path(__file__).resolve().parents[1]
RESEARCH = BASE / 'reviews/r02/b-dispatch-contract/first3/continuation'
REMAINING = RESEARCH / 'remaining'
for path in (RESEARCH, REMAINING):
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))

import joint as joint_module
from physical import Belief as ResearchBelief
from refinement import ReviewedController
import certified as cert
from completion import decide_completion
from deadline import planning_limit_s
from ledger import ActionLedger
from results import build_production_record, validate_run_record

def observation_from_reply(path, reply):
    if reply.get('accepted') is not True:
        raise ValueError('cannot convert rejected reply')
    if path == '/measure':
        kind = reply.get('measure_result')
        if kind not in ('no_signal', 'near', 'direction'):
            raise ValueError('invalid measure result')
        observation = {'kind': kind}
        if kind == 'direction':
            observation['bearing'] = reply['svd_deg']
        return observation
    if path == '/clear':
        kind = reply.get('clear_result')
        if kind not in ('success', 'no_target_in_range'):
            raise ValueError('invalid clear result')
        return {'kind': kind}
    raise ValueError('path is not a physical action')

def _positive_polygon_1p01(poly, q, observation):
```

```
    q = tuple(map(cert.rat, q))
    if observation['kind'] == 'near':
        return cert.rectangle_clip(poly, q, 5)
    if observation['kind'] != 'direction':
        raise ValueError('positive observation required')
    bearing = F(str(observation['bearing']))
    poly = cert.wedge(poly, q, bearing, half=F(101, 100))
    s, c = cert.trig(bearing)
    poly = cert.interval_halfplane(poly, q, c, s, 1500)
    poly = cert.interval_halfplane(poly, q, cert.neg(c), cert.neg(s), 0)
    transverse = 1500 * cert.trig(F(101, 100))[0][1]
    poly = cert.interval_halfplane(poly, q, cert.neg(s), c, transverse)
    poly = cert.interval_halfplane(poly, q, s, cert.neg(c), transverse)
    return cert.rectangle_clip(poly, q, 1500)

class SpecBelief(ResearchBelief):

    report_half = F(101, 100)

    def observe(self, q, obs, action='measure'):
        q = tuple(map(cert.rat, q))
        kind = obs.get('kind')
        allowed = (
            ('direction', 'near', 'no_signal')
            if action == 'measure'
            else ('success', 'no_target_in_range')
        )
        if kind not in allowed:
            raise ValueError('unknown receipt is not physical evidence')
        if kind == 'direction' and not math.isfinite(obs.get('bearing', math.nan)):
            raise ValueError('invalid direction')
        if action == 'measure':
            signature = (kind, F(str(obs['bearing'])) if kind == 'direction' else None)
            if q in self.measured and self.measured[q] != signature:
                self.inconsistent = True
                raise ValueError('spatially fixed measurement contradiction')
            self.measured[q] = signature
        self.events.append((q, dict(obs), action))
        if kind in ('near', 'direction'):
            self.poly = _positive_polygon_1p01(self.poly, q, obs)
        if kind == 'no_target_in_range':
            self.failed.append(q)
        self.version += 1
        if not self.poly:
            self.inconsistent = True
            raise ValueError('empty certified enclosure')

class AlgorithmController:

    def __init__(
        self,
        *,
        q4=False,
        use_discovery_dp=True,
        minimum_sources=10,
        maximum_sources=16,
        three_switches=False,
    ):
        self.q4 = bool(q4)
        original_belief = joint_module.Belief
        joint_module.Belief = SpecBelief
```

```
        try:
            self.base = ReviewedController(
                q4=self.q4,
                joint=three_switches,
                measurements=three_switches,
                use_count=True,
                use_center=False,
                use_discovery_dp=use_discovery_dp,
                use_physical_queries=three_switches,
                minimum_sources=minimum_sources,
                maximum_sources=maximum_sources,
            )
        finally:
            joint_module.Belief = original_belief
        self.plan_id = hashlib.sha256(
            json.dumps(
                self.base.plan, default=str, sort_keys=True, separators=(',', ':')
            ).encode()
        ).hexdigest()

    def next_action(self):
        raw = self.base.next_action()
        if raw is None:
            return None
        payload = {
            'state_version': len(self.base.history),
            'plan_id': self.plan_id,
            'kind': raw['kind'],
            'channel': raw['channel'],
            'position': [f'{float(v):.6f}' for v in raw['position']],
        }
        action_id = hashlib.sha256(
            json.dumps(payload, sort_keys=True, separators=(',', ':')).encode()
        ).hexdigest()
        return dict(
            raw,
            action_id=action_id,
            state_version=len(self.base.history),
            plan_id=self.plan_id,
            branch_id=None,
        )

    def accept_reply(self, path, reply):
        observation = observation_from_reply(path, reply)
        self.base.accept(observation)
        return observation

    def completed(self):
        return self.completion_decision().status == 'completed'

    def completion_decision(self):
        return decide_completion(
            cleared=self.base.cleared,
            excluded=self.base.excluded,
            active_channels=self.base.tasks,
            pending=self.base.pending is not None,
            fault=getattr(self.base, 'fault', None),
            state_version=len(self.base.history),
        )

    def snapshot(self):
```

```
        return {
            'position': self.base.position,
            'channel': self.base.channel,
            'spent': self.base.spent,
            'cleared': sorted(self.base.cleared),
            'excluded': sorted(self.base.excluded),
            'tasks': sorted(self.base.tasks),
            'plan_remaining': len(self.base.plan),
            'completion': asdict(self.completion_decision()),
            'completed': self.completed(),
            'optimization_switches': {
                'measurements': self.base.measurements,
                'joint': self.base.joint,
                'use_physical_queries': self.base.use_physical_queries,
                'use_center': self.base.use_center,
                'use_discovery_dp': self.base.use_discovery_dp,
            },
            'optimization_metrics': dict(self.base.metrics),
        }

def run_episode(
    client,
    *,
    q4=False,
    use_discovery_dp=True,
    max_actions=10000,
    minimum_sources=10,
    maximum_sources=16,
    session_id=None,
    case_id=None,
    mode='local',
    spec_path=None,
    config_paths=(),
    source_paths=(),
    clock=time.monotonic,
    three_switches=True,
):
    # 2026-09-12 起三项优化（额外测向/跨源点级交错/精确物理剪枝）为生产默认;
    # 显式传 three_switches=False 可回退到旧默认用于对照。
    controller = AlgorithmController(
        q4=q4,
        use_discovery_dp=use_discovery_dp,
        minimum_sources=minimum_sources,
        maximum_sources=maximum_sources,
        three_switches=three_switches,
    )
    started = clock()
    enter = client.enter()
    if enter.get('accepted') is not True:
        return {
            'schema_version': '2026-B-run/1',
            'contest': '2026-B',
            'problem': 4 if q4 else 3,
            'command': ['run_episode'],
            'records': [],
            'summary': {'completed': False, 'stop_reason': 'enter_rejected'},
        }
    deadline = started + float(enter['remaining_real_duration_s'])
    ledger = ActionLedger(
        session_id or f"{client.config.robot_id}-{4 if q4 else 3}",
        virtual_time_s=float(enter['virtual_time_s']),
```

```
    )
    records = []
    stop_reason = 'plan_done'
    while len(records) < max_actions:
        remaining_real = deadline - clock()
        if remaining_real <= 0:
            stop_reason = 'deadline'
            break
        if remaining_real <= 20:
            stop_reason = 'deadline_reserve'
            break
        action = controller.next_action()
        if action is None:
            stop_reason = 'complete' if controller.completed() else 'plan_done'
            break
        path = '/measure' if action['kind'] == 'measure' else '/clear'
        prefix = 'measure' if path == '/measure' else 'clear'
        request_id = client.next_request_id(prefix)
        if path == '/measure':
            payload = client.measure_payload(
                request_id, *action['position'], action['channel']
            )
        else:
            payload = client.clear_payload(
                request_id, *action['position'], action['channel']
            )
        ledger.begin(path, request_id, payload, action_id=action.get('action_id'))
        try:
            if path == '/measure':
                reply = client.measure(
                    *action['position'], action['channel'], request_id=request_id
                )
            else:
                reply = client.clear(
                    *action['position'], action['channel'], request_id=request_id
                )
        except Exception as exc:
            ledger.unknown(str(exc))
            stop_reason = 'undetermined'
            break
        ledger.pending.http_status = client.last_http_status
        if reply.get('accepted') is not True:
            receipt = ledger.reject(path, request_id, payload, reply)
            records.append(
                {
                    'request_id': client.last_request_id,
                    'action': path,
                    'action_id': receipt['action_id'],
                    'payload_hash': receipt['payload_hash'],
                    'state_version_before': receipt['state_version_before'],
                    'state_version_after': receipt['state_version_after'],
                    'accepted': False,
                    'result_state': receipt['result_state'],
                    'virtual_time_s': reply.get('virtual_time_s', 0.0),
                    'real_timestamp_ms': reply.get('real_timestamp_ms', 0.0),
                    'response': reply,
                }
            )
            stop_reason = 'rejected'
            break
        receipt = ledger.commit(path, request_id, payload, reply)
```

```
        records.append(
            {
                'request_id': client.last_request_id,
                'action': path,
                'action_id': receipt['action_id'],
                'payload_hash': receipt['payload_hash'],
                'position': [payload['position']['x'], payload['position']['y']],
                'channel': payload['channel'],
                'state_version_before': receipt['state_version_before'],
                'state_version_after': receipt['state_version_after'],
                'accepted': True,
                'result_state': receipt['result_state'],
                'virtual_time_s': reply.get('virtual_time_s', 0.0),
                'real_timestamp_ms': reply.get('real_timestamp_ms', 0.0),
                'planning_limit_s': planning_limit_s(remaining_real),
                'response': reply,
            }
        )
        controller.accept_reply(path, reply)
        if controller.completed():
            stop_reason = 'complete'
            break
    exit_error = None
    if client.session_open:
        try:
            exit_reply = client.exit()
            records.append(
                {
                    'request_id': client.last_request_id,
                    'action': '/exit',
                    'accepted': exit_reply.get('accepted') is True,
                    'virtual_time_s': exit_reply.get('virtual_time_s', 0.0),
                    'real_timestamp_ms': exit_reply.get('real_timestamp_ms', 0.0),
                    'response': exit_reply,
                }
            )
        except Exception as exc:
            exit_error = repr(exc)
    summary = {
        'completed': controller.completed(),
        'stop_reason': stop_reason,
        'virtual_time_s': client.virtual_time_s,
        'program_wall_s': clock() - started,
        'remaining_real_duration_s': client.remaining_real_duration_s,
        'state': controller.snapshot(),
        'ledger': ledger.snapshot(),
        'exit_error': exit_error,
    }
    if spec_path is None:
        record = {
            'schema_version': '2026-B-run/1',
            'contest': '2026-B',
            'problem': 4 if q4 else 3,
            'command': ['run_episode', 'q4' if q4 else 'q3'],
            'records': records,
            'summary': summary,
        }
        validate_run_record(record)
        return record
    return build_production_record(
        problem=4 if q4 else 3,
```

```
        command=['run_episode', 'q4' if q4 else 'q3'],
        records=records,
        summary=summary,
        case_id=case_id or f'q{4 if q4 else 3}-unknown',
        mode=mode,
        spec_path=spec_path,
        config_paths=config_paths,
        source_paths=source_paths,
    )
```

deadline.py

```
# 时间预算

from __future__ import annotations

import multiprocessing as mp
import time

def planning_limit_s(remaining_real_s, reserve_s=20.0):
    if remaining_real_s is None:
        return None
    if remaining_real_s <= reserve_s:
        return 0.0
    return min(0.2, max(0.0, remaining_real_s - reserve_s))

def _worker(function, args, queue):
    try:
        queue.put(('ok', function(*args)))
    except Exception as exc:
        queue.put(('error', repr(exc)))

def run_with_hard_deadline(function, args=(), timeout_s=0.2):
    if timeout_s <= 0:
        return {'status': 'skipped_zero_budget', 'result': None, 'wall_s': 0.0}
    queue = mp.Queue()
    process = mp.Process(target=_worker, args=(function, args, queue))
    started = time.monotonic()
    process.start()
    process.join(timeout_s)
    wall = time.monotonic() - started
    if process.is_alive():
        process.terminate()
        process.join(1.0)
        return {'status': 'timed_out', 'result': None, 'wall_s': wall}
    if queue.empty():
        return {'status': 'worker_failed', 'result': None, 'wall_s': wall}
    status, payload = queue.get()
    return {'status': status, 'result': payload, 'wall_s': wall}
```

ledger.py

```
# 动作账本

from __future__ import annotations

from dataclasses import asdict, dataclass
```

```
import hashlib
import json
import math
import time

def payload_hash(payload):
    encoded = json.dumps(
        payload,
        ensure_ascii=False,
        sort_keys=True,
        separators=(',', ':'),
        allow_nan=False,
    ).encode('utf-8')
    return hashlib.sha256(encoded).hexdigest()

@dataclass
class PendingAction:
    session_id: str
    request_id: str
    path: str
    payload_hash: str
    payload: dict
    action_id: str | None
    sent_monotonic_s: float
    attempts: int = 1
    http_status: int | None = None
    accepted: bool | None = None

class ActionLedger:

    def __init__(
        self, session_id, *, position=(0.0, 0.0), channel=1, virtual_time_s=0.0
    ):
        self.session_id = str(session_id)
        self.position = tuple(map(float, position))
        self.channel = int(channel)
        self.virtual_time_s = float(virtual_time_s)
        self.cleared = set()
        self.excluded = set()
        self.history = []
        self.state_version = 0
        self.pending = None
        self.fault = None
        self.timing_anomaly = False
        self.max_timing_delta_s = 0.0

    def begin(
        self, path, request_id, payload, *, action_id=None, sent_monotonic_s=None
    ):
        if self.fault:
            raise ValueError(f'ledger fault: {self.fault}')
        digest = payload_hash(payload)
        if self.pending is not None:
            same = (
                self.pending.path == path
                and self.pending.request_id == request_id
                and self.pending.payload_hash == digest
            )
            if not same:
                raise ValueError('another action is pending')
            self.pending.attempts += 1
```

```
            return self.pending
        if path not in ('/measure', '/clear'):
            raise ValueError('ledger only owns physical measure/clear actions')
        if payload.get('request_id') != request_id:
            raise ValueError('request_id does not match payload')
        self.pending = PendingAction(
            session_id=self.session_id,
            request_id=request_id,
            path=path,
            payload_hash=digest,
            payload=dict(payload),
            action_id=action_id,
            sent_monotonic_s=(
                time.monotonic()
                if sent_monotonic_s is None
                else float(sent_monotonic_s)
            ),
        )
        return self.pending

    def _check_pending(self, path, request_id, payload):
        if self.pending is None:
            raise ValueError('no pending action')
        digest = payload_hash(payload)
        if (
            self.pending.path != path
            or self.pending.request_id != request_id
            or self.pending.payload_hash != digest
        ):
            raise ValueError('reply does not match pending action')

    def _expected_cost(self, path, payload, result_state):
        position = (float(payload['position']['x']), float(payload['position']['y']))
        move = math.dist(self.position, position) / 5
        if path == '/measure':
            return (
                move + 5 + int(self.channel != payload['channel']),
                position,
                payload['channel'],
            )
        if result_state == 'success':
            return move + 5, position, self.channel
        return move + 3, position, self.channel

    def commit(self, path, request_id, payload, reply, *, received_monotonic_s=None):
        if reply.get('accepted') is not True:
            raise ValueError('commit requires accepted=true')
        self._check_pending(path, request_id, payload)
        self.pending.accepted = True
        if path == '/measure':
            result_state = reply.get('measure_result')
            if result_state not in ('no_signal', 'near', 'direction'):
                raise ValueError('invalid measure_result')
        else:
            result_state = reply.get('clear_result')
            if result_state not in ('success', 'no_target_in_range'):
                raise ValueError('invalid clear_result')
        expected, position, channel = self._expected_cost(path, payload, result_state)
        server_delta = float(reply['virtual_time_s']) - self.virtual_time_s
        tolerance = 2 * (len(self.history) + 2) * 10**-6
        delta = abs(server_delta - expected)
```

```
        self.max_timing_delta_s = max(self.max_timing_delta_s, delta)
        if delta > tolerance:
            self.timing_anomaly = True
        self.position = position
        self.channel = channel
        self.virtual_time_s = float(reply['virtual_time_s'])
        if path == '/clear' and result_state == 'success':
            self.cleared.add(payload['channel'])
        self.state_version += 1
        receipt = {
            'session_id': self.session_id,
            'request_id': request_id,
            'payload_hash': self.pending.payload_hash,
            'action_id': self.pending.action_id,
            'http_status': self.pending.http_status,
            'accepted': True,
            'result_state': result_state,
            'raw_response': dict(reply),
            'sent_monotonic_s': self.pending.sent_monotonic_s,
            'received_monotonic_s': (
                time.monotonic()
                if received_monotonic_s is None
                else float(received_monotonic_s)
            ),
            'state_version_before': self.state_version - 1,
            'state_version_after': self.state_version,
            'expected_cost_s': expected,
            'server_cost_s': server_delta,
            'timing_delta_s': delta,
        }
        self.history.append(receipt)
        self.pending = None
        return receipt

    def reject(self, path, request_id, payload, reply, *, received_monotonic_s=None):
        self._check_pending(path, request_id, payload)
        self.pending.accepted = False
        if reply.get('accepted') is not False:
            raise ValueError('reject requires accepted=false')
        receipt = {
            'session_id': self.session_id,
            'request_id': request_id,
            'payload_hash': self.pending.payload_hash,
            'action_id': self.pending.action_id,
            'http_status': self.pending.http_status,
            'accepted': False,
            'result_state': 'rejected',
            'raw_response': dict(reply),
            'sent_monotonic_s': self.pending.sent_monotonic_s,
            'received_monotonic_s': (
                time.monotonic()
                if received_monotonic_s is None
                else float(received_monotonic_s)
            ),
            'state_version_before': self.state_version,
            'state_version_after': self.state_version,
        }
        self.pending = None
        self.history.append(receipt)
        return receipt
```

```
    def unknown(self, reason):
        if self.pending is None:
            raise ValueError('no pending action')
        self.pending.attempts += 1
        return {
            'state_version': self.state_version,
            'request_id': self.pending.request_id,
            'payload_hash': self.pending.payload_hash,
            'reason': str(reason),
            'pending': True,
        }

    def pending_json(self):
        if self.pending is None:
            return None
        return json.dumps(asdict(self.pending), ensure_ascii=False, sort_keys=True)

    def snapshot(self):
        return {
            'session_id': self.session_id,
            'state_version': self.state_version,
            'position_m': self.position,
            'channel': self.channel,
            'virtual_time_s': self.virtual_time_s,
            'cleared_channels': sorted(self.cleared),
            'excluded_channels': sorted(self.excluded),
            'pending': None if self.pending is None else asdict(self.pending),
            'fault': self.fault,
            'timing_anomaly': self.timing_anomaly,
            'max_timing_delta_s': self.max_timing_delta_s,
        }
```

protocol.py

```
# 机器狗 HTTP 协议客户端

from __future__ import annotations

from dataclasses import dataclass
import hashlib
import json
import math
import time
import urllib.error
import urllib.request

DEFAULT_BASE_URL = 'http://127.0.0.1:2026'
ARENA_ID = 'default'

class ProtocolError(RuntimeError):
    pass

def _finite_number(value):
    return (
        not isinstance(value, bool)
        and isinstance(value, (int, float))
        and math.isfinite(value)
    )
```

```
def _valid_text(value, *, limit, field):
    if not isinstance(value, str) or not 1 <= len(value.encode('utf-8')) <= limit:
        raise ValueError(
            f'{field} must be nonempty UTF-8 text with at most {limit} bytes'
        )
    if any(ord(char) < 32 or ord(char) == 127 for char in value):
        raise ValueError(f'{field} contains control characters')

def _valid_position(x, y):
    for name, value in (('x', x), ('y', y)):
        if (
            isinstance(value, bool)
            or not isinstance(value, (int, float))
            or not math.isfinite(value)
        ):
            raise ValueError(f'position.{name} must be finite')
        if abs(value) > 2_000_000:
            raise ValueError(f'position.{name} exceeds 2000000 m')

def _valid_channel(channel):
    if (
        isinstance(channel, bool)
        or not isinstance(channel, int)
        or not 1 <= channel <= 20
    ):
        raise ValueError('channel must be an integer in 1..20')

@dataclass(frozen=True)
class ProtocolConfig:
    robot_id: str
    base_url: str = DEFAULT_BASE_URL
    timeout_s: float = 5.0
    retries: int = 2
    backoff_s: float = 0.05
    user_agent: str = 'cumcm-2026-b/0.1'

    def __post_init__(self):
        _valid_text(self.robot_id, limit=64, field='robot_id')
        if not isinstance(self.base_url, str) or not self.base_url.startswith(
            ('http://127.0.0.1:', 'http://localhost:')
        ):
            raise ValueError('base_url must target the local simulator')
        if not math.isfinite(self.timeout_s) or self.timeout_s <= 0:
            raise ValueError('timeout_s must be positive')
        if type(self.retries) is not int or self.retries < 0:
            raise ValueError('retries must be a nonnegative integer')
        if not math.isfinite(self.backoff_s) or self.backoff_s < 0:
            raise ValueError('backoff_s must be nonnegative')

class SimulatorClient:

    def __init__(self, config, opener=None):
        self.config = config
        self.opener = opener or urllib.request.urlopen
        self.session_open = False
        self.virtual_time_s = 0.0
        self.remaining_real_duration_s = None
        self.detector_channel = 1
        self.last_path = None
        self.last_request_id = None
        self.last_payload = None
```

```
        self.last_payload_sha256 = None
        self.last_http_status = None
        self._request_counter = 0

    def next_request_id(self, prefix):
        if not isinstance(prefix, str) or not prefix:
            raise ValueError('request prefix must be nonempty')
        self._request_counter += 1
        return f'{prefix}-{self._request_counter}'

    def _request_id(self, prefix):
        return self.next_request_id(prefix)

    def _payload(self, request_id, **fields):
        return {
            'arena_id': ARENA_ID,
            'robot_id': self.config.robot_id,
            'request_id': request_id,
            **fields,
        }

    def _post(self, path, payload):
        encoded = json.dumps(payload, ensure_ascii=False, allow_nan=False).encode(
            'utf-8'
        )
        self.last_path = path
        self.last_request_id = payload.get('request_id')
        self.last_payload = dict(payload)
        self.last_payload_sha256 = hashlib.sha256(encoded).hexdigest()
        for attempt in range(self.config.retries + 1):
            request = urllib.request.Request(
                self.config.base_url + path,
                data=encoded,
                headers={
                    'Content-Type': 'application/json',
                    'User-Agent': self.config.user_agent,
                },
                method='POST',
            )
            try:
                with self.opener(request, timeout=self.config.timeout_s) as response:
                    status = response.status
                    body = response.read()
            except urllib.error.HTTPError as exc:
                status = exc.code
                body = exc.read()
            except (urllib.error.URLError, TimeoutError, ConnectionError) as exc:
                if attempt < self.config.retries:
                    time.sleep(self.config.backoff_s * (attempt + 1))
                    continue
                raise ProtocolError(f'{path} transport failed: {exc}') from exc
            try:
                reply = json.loads(body.decode('utf-8'))
            except (UnicodeDecodeError, json.JSONDecodeError) as exc:
                raise ProtocolError(f'{path} returned non-JSON response') from exc
            self.last_http_status = status
            if status == 200:
                self._validate_common(reply)
                return reply
            if status in (429, 500, 503) and attempt < self.config.retries:
                time.sleep(self.config.backoff_s * (attempt + 1))
```

```
                continue
            raise ProtocolError(f'{path} HTTP {status}: {reply}')
        raise ProtocolError('retry loop exhausted')

    @staticmethod
    def _validate_common(reply):
        if not isinstance(reply, dict) or type(reply.get('accepted')) is not bool:
            raise ProtocolError('reply must contain boolean accepted')
        for key in ('real_timestamp_ms', 'virtual_time_s'):
            value = reply.get(key)
            if (
                isinstance(value, bool)
                or not isinstance(value, (int, float))
                or not math.isfinite(value)
            ):
                raise ProtocolError(f'reply field {key} must be finite')

    @staticmethod
    def _accepted(reply):
        return reply.get('accepted') is True

    def enter(self):
        reply = self._post('/enter', self._payload(self._request_id('enter')))
        if self._accepted(reply):
            for key in (
                'max_virtual_duration_s',
                'max_real_duration_s',
                'remaining_real_duration_s',
            ):
                if not _finite_number(reply.get(key)):
                    raise ProtocolError(f'enter reply requires finite {key}')
            self.session_open = True
            self.virtual_time_s = float(reply['virtual_time_s'])
            self.remaining_real_duration_s = float(reply['remaining_real_duration_s'])
        return reply

    def measure_payload(self, request_id, x, y, channel):
        _valid_position(x, y)
        _valid_channel(channel)
        return self._payload(request_id, position={'x': x, 'y': y}, channel=channel)

    def measure(self, x, y, channel, request_id=None):
        if not self.session_open:
            raise ProtocolError('measure requires a successful enter')
        request_id = request_id or self._request_id('measure')
        reply = self._post('/measure', self.measure_payload(request_id, x, y, channel))
        if self._accepted(reply):
            result = reply.get('measure_result')
            if result not in ('no_signal', 'near', 'direction'):
                raise ProtocolError('invalid measure_result')
            if result == 'direction':
                bearing = reply.get('svd_deg')
                if (
                    isinstance(bearing, bool)
                    or not isinstance(bearing, (int, float))
                    or not math.isfinite(bearing)
                ):
                    raise ProtocolError('direction reply requires finite svd_deg')
            self.virtual_time_s = float(reply['virtual_time_s'])
            self.detector_channel = channel
        return reply
```

```
    def clear_payload(self, request_id, x, y, channel):
        _valid_position(x, y)
        _valid_channel(channel)
        return self._payload(request_id, position={'x': x, 'y': y}, channel=channel)

    def clear(self, x, y, channel, request_id=None):
        if not self.session_open:
            raise ProtocolError('clear requires a successful enter')
        request_id = request_id or self._request_id('clear')
        reply = self._post('/clear', self.clear_payload(request_id, x, y, channel))
        if self._accepted(reply):
            if reply.get('clear_result') not in ('success', 'no_target_in_range'):
                raise ProtocolError('invalid clear_result')
            self.virtual_time_s = float(reply['virtual_time_s'])
        return reply

    def exit(self):
        if not self.session_open:
            raise ProtocolError('exit requires a successful enter')
        reply = self._post('/exit', self._payload(self._request_id('exit')))
        if self._accepted(reply):
            if reply.get('exit_reason') not in ('user_exit',):
                raise ProtocolError('invalid exit_reason')
            self.virtual_time_s = float(reply['virtual_time_s'])
            self.session_open = False
        return reply
```

results.py

```
# 结果汇总

from __future__ import annotations

import hashlib
import importlib.metadata as importlib_metadata
import json
import math
from pathlib import Path
import platform
import sys

def sha256_text_lf(path):
    return hashlib.sha256(Path(path).read_bytes().replace(b'\r\n', b'\n')).hexdigest()

def _finite(value):
    return (
        not isinstance(value, bool)
        and isinstance(value, (int, float))
        and math.isfinite(value)
    )

def validate_run_record(record):
    if not isinstance(record, dict):
        raise ValueError('run record must be a mapping')
    required = {'schema_version', 'contest', 'problem', 'command', 'records', 'summary'}
    missing = required - set(record)
    if missing:
        raise ValueError(f'missing run fields: {sorted(missing)}')
```

```
    if not isinstance(record['records'], list):
        raise ValueError('records must be a list')
    for index, item in enumerate(record['records']):
        if not isinstance(item, dict):
            raise ValueError(f'records[{index}] must be a mapping')
        for key in (
            'request_id',
            'action',
            'accepted',
            'virtual_time_s',
            'real_timestamp_ms',
        ):
            if key not in item:
                raise ValueError(f'records[{index}] missing {key}')
        if type(item['accepted']) is not bool:
            raise ValueError(f'records[{index}].accepted must be boolean')
        for key in ('virtual_time_s', 'real_timestamp_ms'):
            if not _finite(item[key]):
                raise ValueError(f'records[{index}].{key} must be finite')
    if not isinstance(record['summary'], dict):
        raise ValueError('summary must be a mapping')
    return record

def write_json(path, record):
    validate_run_record(record)
    destination = Path(path)
    destination.parent.mkdir(parents=True, exist_ok=True)
    text = json.dumps(record, ensure_ascii=False, indent=2, allow_nan=False) + '\n'
    destination.write_text(text, encoding='utf-8', newline='\n')
    return destination

def load_json(path):
    return json.loads(Path(path).read_text(encoding='utf-8'))

def environment_snapshot():
    packages = {}
    for name in ('numpy', 'scipy', 'z3-solver'):
        try:
            packages[name] = importlib_metadata.version(name)
        except importlib_metadata.PackageNotFoundError:
            packages[name] = None
    return {
        'python': sys.version.split()[0],
        'platform': platform.platform(),
        'packages': packages,
    }

def file_bindings(paths):
    return {Path(path).as_posix(): sha256_text_lf(path) for path in paths}

def build_production_record(
    *,
    problem,
    command,
    records,
    summary,
    case_id,
    mode,
    spec_path,
    config_paths,
    source_paths,
```

```
):
    if mode not in ('local', 'practice', 'official'):
        raise ValueError('invalid run mode')
    record = {
        'schema_version': '2026-B-production-run/1',
        'contest': '2026-B',
        'problem': int(problem),
        'command': list(command),
        'records': records,
        'summary': summary,
        'case_id': str(case_id),
        'mode': mode,
        'metadata': {
            'environment': environment_snapshot(),
            'spec_sha256': sha256_text_lf(spec_path),
            'config_sha256': file_bindings(config_paths),
            'source_sha256': file_bindings(source_paths),
        },
    }
    validate_run_record(record)
    return record

def validate_production_record(record):
    validate_run_record(record)
    for key in ('case_id', 'mode', 'metadata'):
        if key not in record:
            raise ValueError(f'production record missing {key}')
    if record['mode'] not in ('local', 'practice', 'official'):
        raise ValueError('invalid production mode')
    metadata = record['metadata']
    for key in ('environment', 'spec_sha256', 'config_sha256', 'source_sha256'):
        if key not in metadata:
            raise ValueError(f'production metadata missing {key}')
    return record
```

correlated_bearing.py

```
import math
import numpy as np

def noise_weights(p):
    x, y = np.asarray(p) / 150
    ix, iy = math.floor(x), math.floor(y)
    tx, ty = x - ix, y - iy
    sx, sy = tx * tx * (3 - 2 * tx), ty * ty * (3 - 2 * ty)
    return {
        (ix, iy): (1 - sx) * (1 - sy),
        (ix + 1, iy): sx * (1 - sy),
        (ix, iy + 1): (1 - sx) * sy,
        (ix + 1, iy + 1): sx * sy,
    }

def estimate(poly, history, problem, geometry, mode=0):
    mid, rad, u, v, width = geometry
    positive = [
        (p, o)
        for k, p, o in history
```

```
        if k == 'measure' and o['kind'] in ('direction', 'near')
    ]
    bearings = [(p, o['bearing']) for p, o in positive if o['kind'] == 'direction']
    if len(bearings) < 2:
        return geometry
    polygon = np.asarray(poly)
    u = np.asarray(u)
    v = np.asarray(v)
    xx = polygon @ u
    yy = polygon @ v
    nx, ny = 161, 17
    xs = np.linspace(xx.min(), xx.max(), nx)
    ys = np.linspace(yy.min(), yy.max(), ny)
    points = (xs[:, None, None] * u + ys[None, :, None] * v).reshape(-1, 2)
    keep = (points * points).sum(axis=1) <= 1770**2
    for a, b in zip(polygon, np.roll(polygon, -1, axis=0)):
        e = b - a
        d = points - a
        keep &= e[0] * d[:, 1] - e[1] * d[:, 0] >= -1e-6
    failures = [p for k, p, o in history if k == 'clear' and o['kind'] != 'success']
    for p in failures:
        keep &= np.linalg.norm(points - p, axis=1) > 20
    points = points[keep]
    if not len(points):
        return geometry
    bp = np.array([p for p, b in bearings])
    observed = np.array([b for p, b in bearings])
    d = points[:, None, :] - bp
    predicted = np.degrees(np.arctan2(d[:, :, 1], d[:, :, 0]))
    residual = (predicted - observed + 180) % 360 - 180
    w = [noise_weights(p) for p in bp]
    covariance = np.array(
        [
            [sum(value * b.get(key, 0) for key, value in a.items()) / 3 for b in w]
            for a in w
        ]
    )
    if mode == 1:
        covariance = np.diag(np.diag(covariance))
    covariance += np.eye(len(w)) * (0.01**2 / 12 + 1e-6)
    inverse = np.linalg.inv(covariance)
    loglike = -0.5 * np.einsum('ni,ij,nj->n', residual, inverse, residual)
    # 融合未知接收半径和共同朝向，使用所有
    # 发现前后的正负回执
    posdelta = np.array([p for p, o in positive])[None, :, :] - points[:, None, :]
    posdist = np.linalg.norm(posdelta, axis=2)
    minimum = np.maximum(1000, posdist.max(axis=1))
    negatives = [
        p for k, p, o in history if k == 'measure' and o['kind'] == 'no_signal'
    ]
    nd = (
        np.array(negatives)[None, :, :] - points[:, None, :]
        if negatives
        else np.empty((len(points), 0, 2))
    )
    nr = np.linalg.norm(nd, axis=2)
    if problem == 3:
        maximum = (
            np.minimum(1500, nr.min(axis=1))
            if negatives
            else np.full(len(points), 1500.0)
```

```
        )
        likelihood = np.maximum(0, maximum - minimum) / 500
    else:
        angles = np.arctan2(posdelta[:, :, 1], posdelta[:, :, 0]) % (2 * math.pi)
        negangles = np.arctan2(-nd[:, :, 1], -nd[:, :, 0]) % (2 * math.pi)
        likelihood = np.zeros(len(points))
        for f in (0.0625, 0.1875, 0.3125, 0.4375, 0.5625, 0.6875, 0.8125, 0.9375):
            receive = minimum + f * np.maximum(0, 1500 - minimum)
            inside = nr <= receive[:, None]
            joined = np.concatenate(
                [angles, np.where(inside, negangles, np.inf)], axis=1
            )
            joined.sort(axis=1)
            counts = np.isfinite(joined).sum(axis=1)
            probability = np.zeros(len(points))
            for n in np.unique(counts):
                ix = counts == n
                a = joined[ix, :n]
                gaps = np.diff(
                    np.concatenate([a, a[:, :1] + 2 * math.pi], axis=1), axis=1
                )
                probability[ix] = np.maximum(0, gaps.max(axis=1) - math.pi) / (
                    2 * math.pi
                )
            # 公开源数先验下的边缘定向比例
            likelihood += (0.46 * (~inside.any(axis=1)) + 0.54 * probability) / 8
        likelihood *= np.maximum(0, 1500 - minimum) / 500
    loglike += np.log(np.maximum(likelihood, 1e-300))
    if likelihood.max() <= 0:
        return geometry
    weights = np.exp(loglike - loglike.max())
    weights /= weights.sum()
    center = weights @ points
    distance = np.linalg.norm(points - center, axis=1)
    order = np.argsort(distance)
    confidence = {2: 0.95, 3: 0.999, 4: 0.8, 5: 0.5}.get(mode, 0.999)
    credible = distance[
        order[
            min(len(order) - 1, np.searchsorted(np.cumsum(weights[order]), confidence))
        ]
    ]
    padding = math.hypot((xs[-1] - xs[0]) / (nx - 1), (ys[-1] - ys[0]) / (ny - 1))
    radius = float(credible + padding) if mode >= 2 else float(distance.max() + padding)
    return tuple(center), radius, tuple(u), tuple(v), float(min(width, 2 * radius))
```

coverage.py

```
# 保守单元覆盖证书

import math
import numpy as np

class Coverage:
    def __init__(self, problem, step=80):
        self.problem = problem
        self.radius = step / math.sqrt(2) + 1e-5
        axis = np.arange(-1800, 1800 + step, step)
        self.cells = np.array(
            [
```

```
                (x, y)
                for x in axis
                for y in axis
                if math.hypot(x, y) <= 1800 + self.radius
            ]
        )
        self.points = []
        self.mask = np.zeros(len(self.cells), dtype=bool)

    def covered(self, points):
        if not points:
            return np.zeros(len(self.cells), dtype=bool)
        delta = np.array(points)[None, :, :] - self.cells[:, None, :]
        dist = np.linalg.norm(delta, axis=2)
        valid = dist < 1000 - self.radius
        if self.problem == 3:
            return valid.any(axis=1)
        angles = np.arctan2(delta[:, :, 1], delta[:, :, 0]) % (2 * math.pi)
        # 观测者在单元内时，不能保证整单元覆盖
        valid &= dist > self.radius
        errors = np.arcsin(np.minimum(1, self.radius / np.maximum(dist, 1e-10)))
        angles = np.where(valid, angles, np.inf)
        order = np.argsort(angles, axis=1)
        angles = np.take_along_axis(angles, order, axis=1)
        errors = np.take_along_axis(errors, order, axis=1)
        count = valid.sum(axis=1)
        result = np.zeros(len(self.cells), dtype=bool)
        for n in np.unique(count):
            if n < 3:
                continue
            rows = count == n
            a = angles[rows, :n]
            e = errors[rows, :n]
            gaps = np.roll(a, -1, axis=1) - a
            gaps[:, -1] += 2 * math.pi
            result[rows] = (gaps + e + np.roll(e, -1, axis=1) < math.pi - 1e-8).all(
                axis=1
            )
        return result

    def add(self, p):
        self.points.append(p)
        self.mask = self.covered(self.points)

    @property
    def complete(self):
        return bool(self.mask.all())

    def gain(self, p):
        if self.problem == 3:
            return int(np.count_nonzero(self.covered(self.points + [p]) & ~self.mask))
        # 同时奖励部分角度覆盖和单元完成
        delta = self.cells[~self.mask] - p
        return int(np.count_nonzero(np.linalg.norm(delta, axis=1) < 1000 - self.radius))
```

discovery.py

```
# 环形发现网格
```

```
import math
import numpy as np

def ring_grid(outer=12, inner=8, r_outer=1865, r_inner=999, phase=0):
    return [(0.0, 0.0)] + [
        (
            round(r * math.cos(2 * math.pi * k / n + a), 6),
            round(r * math.sin(2 * math.pi * k / n + a), 6),
        )
        for n, r, a in [(outer, r_outer, 0), (inner, r_inner, phase)]
        for k in range(n)
    ]

def certified_cells(cells, radius, points):
    delta = np.array(points)[None, :, :] - cells[:, None, :]
    dist = np.linalg.norm(delta, axis=2)
    valid = (dist < 1000 - radius - 1e-5) & (dist > radius)
    angles = np.where(
        valid, np.arctan2(delta[:, :, 1], delta[:, :, 0]) % (2 * math.pi), np.inf
    )
    errors = np.arcsin(np.minimum(1, radius / np.maximum(dist, 1e-10)))
    order = np.argsort(angles, axis=1)
    angles = np.take_along_axis(angles, order, axis=1)
    errors = np.take_along_axis(errors, order, axis=1)
    count = valid.sum(axis=1)
    result = np.zeros(len(cells), dtype=bool)
    for n in np.unique(count):
        if n < 3:
            continue
        rows = count == n
        a = angles[rows, :n]
        e = errors[rows, :n]
        gaps = np.roll(a, -1, axis=1) - a
        gaps[:, -1] += 2 * math.pi
        result[rows] = (gaps + e + np.roll(e, -1, axis=1) < math.pi - 1e-9).all(axis=1)
    return result

def certify(points, min_side=0.1):
    cells = np.array([[0.0, 0.0]])
    side = 3600.0
    levels = []
    while len(cells):
        mask = certified_cells(cells, side / math.sqrt(2), points)
        uncertain = cells[~mask]
        levels.append(
            dict(
                side_m=side,
                cells=len(cells),
                certified=int(mask.sum()),
                uncertain=len(uncertain),
            )
        )
        if not len(uncertain):
            return dict(covered=True, formal_interval_proof=False, levels=levels)
        if side <= min_side:
            return dict(
                covered=False,
                formal_interval_proof=False,
                levels=levels,
                unresolved_centers=uncertain[:100].tolist(),
            )
```

```
        side /= 2
        cells = np.concatenate(
            [
                uncertain + np.array([x, y]) * side / 2
                for x, y in [(-1, -1), (-1, 1), (1, -1), (1, 1)]
            ]
        )
        nearest = np.maximum(np.abs(cells) - side / 2, 0)
        cells = cells[(nearest**2).sum(axis=1) <= 1800**2 + 1e-6]

if __name__ == '__main__':
    import json
    from pathlib import Path

    report = certify(ring_grid())
    print(json.dumps(report))
    output = Path(__file__).with_name('ring21-r999-certificate.json')
    assert not output.exists()
    output.write_text(json.dumps(dict(points=ring_grid(), **report), indent=2) + '\n')
```

empirical_discovery.py

```
# 经验发现采样

import math
import numpy as np
from scipy.stats import qmc

class EmpiricalDiscovery:
    def __init__(self, problem, tolerance, power=13, stress=0):
        samples = qmc.Halton(5, scramble=True, seed=902611).random(2**power)
        r = 1770 * np.sqrt(samples[:, 0])
        a = 2 * math.pi * samples[:, 1]
        self.points = np.column_stack([r * np.cos(a), r * np.sin(a)])
        self.receive2 = (1000 + 500 * samples[:, 2]) ** 2
        directions = 2 * math.pi * samples[:, 3]
        self.axes = np.column_stack([np.cos(directions), np.sin(directions)])
        self.omni = np.ones(len(r), dtype=bool) if problem == 3 else samples[:, 4] < 0.5
        if stress:
            # 公开支持集压力配置强调短接收
            # 半径和平均先验漏掉的边界与切向
            radial = np.linspace(
                250 if stress == 1 else 150, 1770, 7 if stress == 1 else 12
            )
            na = 48 if stress == 1 else 96
            nd = 24 if stress == 1 else 48
            rr, aa, dd = np.meshgrid(
                radial, np.arange(na), np.arange(nd), indexing='ij'
            )
            theta = 2 * math.pi * (aa / na + (rr - radial[0]) / (1770 * na))
            direction = theta + 2 * math.pi * dd / nd
            extra = np.column_stack(
                [(rr * np.cos(theta)).ravel(), (rr * np.sin(theta)).ravel()]
            )
            self.points = np.concatenate([self.points, extra])
            self.receive2 = np.r_[self.receive2, np.full(len(extra), 1000.0**2)]
            self.axes = np.concatenate(
                [
                    self.axes,
```

```
                    np.column_stack(
                        [np.cos(direction).ravel(), np.sin(direction).ravel()]
                    ),
                ]
            )
            self.omni = np.r_[self.omni, np.full(len(extra), problem == 3)]
        self.unseen = np.ones(len(self.points), dtype=bool)
        self.tolerance = tolerance
        self.cache = {}

    def visible(self, p):
        key = tuple(round(float(x), 4) for x in p)
        if key not in self.cache:
            d = np.asarray(p) - self.points
            self.cache[key] = ((d * d).sum(axis=1) <= self.receive2) & (
                self.omni | ((d * self.axes).sum(axis=1) >= 0)
            )
        return self.cache[key]

    def add(self, p):
        self.unseen &= ~self.visible(p)

    def adjust(self, remaining, targets, position, blend=False):
        if not remaining:
            return remaining
        pending = list(remaining)
        budget = int(self.tolerance * len(self.unseen))

        def miss(points):
            unseen = self.unseen.copy()
            for p in points:
                unseen &= ~self.visible(p)
            return int(unseen.sum())

        # 残差质量满足阈值时，至少保留原计划质量
        # 超过经验阈值时的处理
        bound = max(budget, miss(pending))
        for p in sorted(pending, key=lambda p: -math.dist(position, p)):
            others = [q for q in pending if q != p]
            if miss(others) <= bound:
                pending = others
        # 仅当剩余设计仍在漏检预算内，
        # 源服务点才能替代发现点
        for target in targets:
            if not pending:
                break
            p = min(pending, key=lambda p: math.dist(p, target))
            if math.dist(p, target) < 2 or math.dist(p, target) > 850:
                continue
            others = [q for q in pending if q != p]
            for fraction in ((1.0, 0.75, 0.5, 0.25) if blend else (1.0,)):
                q = tuple(
                    round(p[k] + fraction * (target[k] - p[k]), 6) for k in (0, 1)
                )
                if miss(others + [q]) <= bound:
                    pending = others + [q]
                    break
        # 有界缓存：旧候选位置不再需要
        if len(self.cache) > 1000:
            self.cache = {}
        return pending
```

flexroute.py

```
# 环形点位移与路线调整

import math
import numpy as np
from scipy.optimize import minimize
from routing import route

def margins(points):
    radii = np.linalg.norm(points, axis=1)
    angles = np.unwrap(np.arctan2(points[:, 1], points[:, 0]))
    alpha = np.minimum(
        np.arccos(np.clip((1800**2 + radii**2 - 999.9**2) / (3600 * radii), -1, 1)),
        np.arccos(np.clip((1000**2 + radii**2 - 999.9**2) / (2000 * radii), -1, 1)),
    )
    gaps = np.diff(np.r_[angles, angles[0] + 2 * math.pi])
    return np.r_[alpha + np.roll(alpha, -1) - gaps, gaps]

def adjust(ring, remaining, centers, position, iterations=20):
    free = [i for i, p in enumerate(ring) if p in remaining]
    if not free:
        return ring, remaining
    points = np.array(ring)
    radii = np.linalg.norm(points, axis=1)
    theta = np.unwrap(np.arctan2(points[:, 1], points[:, 0]))
    seed = np.r_[radii[free], np.zeros(len(free))]
    nodes = [('ring', i, tuple(p)) for i, p in enumerate(points) if i in free]
    nodes += [('source', i, p) for i, p in enumerate(centers)]
    tour = route(nodes, position, 4)

    def unpack(x):
        copy = points.copy()
        n = len(free)
        for j, i in enumerate(free):
            a = theta[i] + x[n + j] / 1300
            copy[i] = (x[j] * math.cos(a), x[j] * math.sin(a))
        return copy

    def cost(x):
        changed = unpack(x)
        last = position
        total = 0.0
        for kind, i, p in tour:
            q = changed[i] if kind == 'ring' else p
            total += math.dist(last, q)
            last = q
        return total

    result = minimize(
        cost,
        seed,
        method='SLSQP',
        bounds=[(1125, 1620)] * len(free) + [(-500, 500)] * len(free),
        constraints=[dict(type='ineq', fun=lambda x: margins(unpack(x)) * 1000)],
        options=dict(maxiter=iterations, ftol=0.05),
    )
    changed = [tuple(round(float(v), 6) for v in p) for p in unpack(result.x)]
    if min(margins(np.array(changed))) < -1e-7 or cost(result.x) >= cost(seed):
        return ring, remaining
    replacement = {ring[i]: changed[i] for i in free}
```

```
    return changed, [replacement.get(p, p) for p in remaining]
```

policy.py

```
# 回执驱动策略入口：tenth-v1 只使用公开回执，不读取场景、种子或真值

import math
import numpy as np
from scipy.spatial import Delaunay, ConvexHull
import core

def hex_grid():
    s = 970.0
    points = [
        (s * (i + j / 2), s * math.sqrt(3) * j / 2)
        for i in range(-2, 3)
        for j in range(-2, 3)
        if max(abs(i), abs(j), abs(i + j)) <= 2
    ]
    points += [
        (
            1875 * math.cos(math.radians(30 + 60 * k)),
            1875 * math.sin(math.radians(30 + 60 * k)),
        )
        for k in range(6)
    ]
    array = np.array(points)
    hull = ConvexHull(array)
    assert all(-b / np.linalg.norm(n) > 1800 for *n, b in hull.equations)
    triangles = Delaunay(array).simplices
    max_edge = max(
        math.dist(points[i], points[j]) for tri in triangles for i in tri for j in tri
    )
    assert max_edge < 1000
    return points

def center_radius(poly):
    angle = core.principal(poly)
    u, v = core.axes(angle)
    xs = [core.dot(p, u) for p in poly]
    ys = [core.dot(p, v) for p in poly]
    middle = core.world((min(xs) + max(xs)) / 2, (min(ys) + max(ys)) / 2, u, v)
    return middle, max(math.dist(middle, p) for p in poly), u, v, max(xs) - min(xs)

def solve(problem, action, variant, _forced=None, _observer=None):
    if variant == 'tenth-v1':
        variant = 'corrsensor-2-10' if problem == 3 else 'visibilityguard-1'
    if variant == 'ninth-v1':
        variant = 'corrsensor-2-10' if problem == 3 else 'visibleprobe-1'
    corr_clear = variant.startswith('corrclear-')
    corr_clear_radius = float(variant.split('-')[1]) if corr_clear else 0
    if corr_clear:
        variant = 'corrsensor-2-10'
    corr_sensor = variant.startswith('corrsensor-')
    sensor_parts = variant.split('-') if corr_sensor else []
    if corr_sensor:
        variant = 'correlated-' + sensor_parts[1]
    if variant == 'eighth-v1':
        variant = 'correlated-0' if problem == 3 else 'visibleprobe-1'
```

```
    visible_grid = variant.startswith('visiblegrid-')
    prior_guard = variant.startswith('visibilityguard-')
    prior_guard_level = int(variant.split('-')[1]) if prior_guard else 0
    if visible_grid or prior_guard:
        variant = 'visibleprobe-1'
    visible_probe = variant.startswith('visibleprobe-')
    visible_mode = int(variant.split('-')[1]) if visible_probe else 0
    if visible_probe:
        variant = 'merged-1' if problem == 3 or visible_grid else 'edense-15-0'
    travel_micro = variant.startswith('travelmicro-')
    travel_mode = int(variant.split('-')[1]) if travel_micro else 0
    if travel_micro:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    correlated = variant.startswith('correlated-')
    correlated_mode = int(variant.split('-')[1]) if correlated else 0
    if correlated:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    controls = variant.startswith('controls-')
    control_values = tuple(map(float, variant.split('-')[1:])) if controls else ()
    if controls:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    q4_action = variant.startswith('q4act-')
    q4_action_threshold = int(variant.split('-')[1]) if q4_action else 0
    if q4_action:
        variant = 'edense-15-0'
    cover_plan = variant.startswith('coverplan-')
    cover_weight = float(variant.split('-')[1]) if cover_plan else 0
    if cover_plan:
        variant = 'eadapt-0' if problem == 3 else 'edense-15-0'
    value_policy = variant.startswith('value1-')
    value_margin = float(variant.split('-')[1]) if value_policy else 0
    last_value_key = None
    if value_policy:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    staged = variant.startswith('staged-')
    stage_limit = int(variant.split('-')[1]) if staged else 0
    stage_scans = 0
    if staged:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    safety_check = variant.startswith('scheck-')
    safety_done = False
    safety_mode = False
    if safety_check:
        variant = 'smooth-' + variant.split('-')[1]
    harvest = variant.startswith('harvest-')
    harvest_margin = float(variant.split('-')[1]) if harvest else 0
    if harvest:
        variant = 'eadapt-0' if problem == 3 else 'edense-15-0'
    stress_smooth = variant.startswith('sstress-')
    stress_level = int(variant.split('-')[1]) if stress_smooth else 0
    if stress_smooth:
        variant = 'smooth-' + variant.split('-')[2]
    guarded_smooth = variant.startswith('sguard-')
    smooth_guard = float(variant.split('-')[1]) if guarded_smooth else 0
    if guarded_smooth:
        variant = 'smooth-' + variant.split('-')[2]
    smooth_filter = variant.startswith('smoothf-')
    smooth_filter_mode = int(variant.split('-')[2]) if smooth_filter else 0
    if smooth_filter:
        variant = 'smooth-' + variant.split('-')[1]
    smooth_dense = variant.startswith('sdense-')
```

```
    smooth_density = int(variant.split('-')[1]) if smooth_dense else 0
    if smooth_dense:
        variant = 'smooth-' + variant.split('-')[2]
    smooth = variant.startswith('smooth-')
    smooth_passes = int(variant.split('-')[1]) if smooth else 0
    if smooth:
        variant = 'eadapt-0' if problem == 3 else 'edense-15-0'
    micro_route = variant.startswith('mroute-')
    micro_route_weight = float(variant.split('-')[1]) if micro_route else 0
    if micro_route:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    lookahead = variant.startswith('lookahead-')
    lookahead_worlds = int(variant.split('-')[1]) if lookahead else 0
    if lookahead:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    lookahead_base = variant
    decision_index = 0
    forced_history = []
    action_prefix = []
    last_plan_key = None
    navigation = variant.startswith('nav-')
    navigation_bias = float(variant.split('-')[1]) if navigation else 0
    if navigation:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    negative_filter = variant.startswith('filter-') or smooth_filter or correlated
    filter_mode = (
        correlated_mode
        if correlated
        else (
            smooth_filter_mode
            if smooth_filter
            else int(variant.split('-')[1]) if negative_filter else 0
        )
    )
    if negative_filter and not smooth_filter and not correlated:
        variant = 'merged-1' if problem == 3 else 'edense-15-0'
    if variant == 'empirical-v4':
        variant = 'eadapt-0.0005' if problem == 3 else 'edense-15-0'
    empirical_dense = variant.startswith('edense-')
    empirical_power = int(variant.split('-')[1]) if empirical_dense else 13
    if smooth_dense:
        empirical_power = smooth_density
    if empirical_dense:
        variant = 'eblend-' + variant.split('-')[2]
    empirical_blend = variant.startswith('eblend-')
    if empirical_blend:
        variant = 'eadapt-' + variant.split('-')[1]
    sticky = variant.startswith('sticky-')
    sticky_mode = int(variant.split('-')[1]) if sticky else 0
    retained_tour = []
    last_discovery = -1
    if sticky:
        variant = 'merged-1'
    service_scan = variant.startswith('service-')
    service_values = tuple(map(float, variant.split('-')[1:])) if service_scan else ()
    if service_scan:
        variant = 'eadapt-' + str(service_values[0])
    empirical_config = variant.startswith('econf-')
    config_values = (
        tuple(map(float, variant.split('-')[1:])) if empirical_config else ()
    )
```

```
    if empirical_config:
        variant = 'eadapt-' + str(config_values[0])
    empirical_adapt = variant.startswith('eadapt-')
    empirical_tolerance = float(variant.split('-')[1]) if empirical_adapt else 0
    if empirical_adapt:
        variant = 'merged-1'
    empirical = variant.startswith('emp-')
    empirical_parts = variant.split('-')[1:] if empirical else []
    empirical_guard_done = False
    if empirical:
        variant = 'merged-1'
    rollout = variant.startswith('rollout-')
    rollout_worlds = int(variant.split('-')[1]) if rollout else 0
    rollout_decision = 0
    if rollout:
        variant = 'merged-1'
    forecast = variant.startswith('forecast-')
    forecast_scale = float(variant.split('-')[1]) if forecast else 0
    if forecast:
        variant = 'skip-50' if problem == 4 else 'skip-2000'
    merged = variant.startswith('merged-')
    merged_mode = int(variant.split('-')[1]) if merged else 0
    if merged:
        variant = 'skip-2000' if problem == 3 else 'skip-50'
    informed = variant.startswith('informed-')
    information_weight = float(variant.split('-')[1]) if informed else 0
    if informed:
        variant = 'skip-50' if problem == 4 else 'skip-2000'
    skip_known = variant.startswith('skip-')
    skip_radius = float(variant.split('-')[1]) if skip_known else 0
    if staged:
        skip_radius = 50
    if skip_known:
        variant = 'micro-100' if problem == 4 else 'micro-150'
    micro = variant.startswith('micro-')
    micro_distance = float(variant.split('-')[1]) if micro else 0
    if corr_sensor:
        micro_distance = float(sensor_parts[2])
    if controls:
        micro_distance *= control_values[0]
    if micro:
        variant = 'cleartry-45'
    if empirical_config:
        micro_distance = config_values[1]
    cleartry = variant.startswith('cleartry-')
    exploratory_radius = float(variant.split('-')[1]) if cleartry else 45
    if corr_clear:
        exploratory_radius = corr_clear_radius
    if controls:
        exploratory_radius = control_values[1]
    if cleartry:
        variant = 'bracket-6' if problem == 4 else 'count-tune-90-250'
    if empirical_config:
        exploratory_radius = config_values[2]
    layered = variant.startswith('layered-')
    layer_radius = float(variant.split('-')[1]) if layered else 0
    layer_outer = False
    if layered:
        variant = 'bracket-6' if problem == 4 else 'count-tune-90-250'
    early = variant.startswith('early-')
    negative_limit = int(variant.split('-')[1]) if early else 3
```

```
    if controls:
        negative_limit = int(control_values[4])
    if early:
        variant = 'bracket-6'
    bracket = variant.startswith('bracket-')
    bracket_steps = int(variant.split('-')[1]) if bracket else 0
    if bracket:
        variant = 'count-prefetch-1' if problem == 4 else 'count-tune-90-250'
    count_stop = variant.startswith('count-')
    if count_stop:
        variant = variant.removeprefix('count-')
    prefetch = variant.startswith('prefetch-')
    prefetch_count = int(variant.split('-')[1]) if prefetch else 0
    if staged:
        prefetch_count = 0
    if prefetch:
        variant = 'rings-8' if problem == 4 else 'multi-8'
    belief = variant.startswith('belief-')
    belief_mode = variant.split('-')[1] if belief else ''
    if belief:
        variant = 'tune-90-250'
    base_geometry = globals()['center_radius']

    def center_radius(poly):
        middle, radius, u, v, width = base_geometry(poly)
        if controls and control_values[5] and radius > 40:
            area = sum(
                a[0] * b[1] - b[0] * a[1] for a, b in zip(poly, poly[1:] + poly[:1])
            )
            if abs(area) > 1e-6:
                centroid = tuple(
                    sum(
                        (a[k] + b[k]) * (a[0] * b[1] - b[0] * a[1])
                        for a, b in zip(poly, poly[1:] + poly[:1])
                    )
                    / (3 * area)
                    for k in (0, 1)
                )
                middle = tuple(
                    middle[k] + control_values[5] * (centroid[k] - middle[k])
                    for k in (0, 1)
                )
                radius = max(math.dist(middle, p) for p in poly)
        if belief_mode in ('mean', 'both') and radius > 80:
            twice_area = sum(
                a[0] * b[1] - b[0] * a[1] for a, b in zip(poly, poly[1:] + poly[:1])
            )
            if abs(twice_area) > 1e-6:
                middle = tuple(
                    sum(
                        (a[k] + b[k]) * (a[0] * b[1] - b[0] * a[1])
                        for a, b in zip(poly, poly[1:] + poly[:1])
                    )
                    / (3 * twice_area)
                    for k in (0, 1)
                )
                radius = max(math.dist(middle, p) for p in poly)
        return middle, radius, u, v, width

    angular = variant.startswith('angular-')
    angular_gain = float(variant.split('-')[1]) * math.pi / 180 if angular else 0
```

```
    if angular:
        variant = 'tune-90-250'
    weighted = variant.startswith('weighted-')
    scan_weight = float(variant.split('-')[1]) if weighted else 0
    if weighted:
        variant = 'rings-8' if problem == 4 else 'multi-8'
    rings = variant.startswith('rings-')
    rings_starts = int(variant.split('-')[1]) if rings else 0
    if rings:
        variant = 'tune-90-250'
    stationary = variant.startswith('stationary-')
    stationary_ratio = float(variant.split('-')[1]) if stationary else 0
    if stationary:
        variant = 'tune-90-250'
    polar = variant.startswith('polar-')
    polar_iterations = int(variant.split('-')[1]) if polar else 0
    if polar:
        variant = 'tune-90-250'
    if merged and problem == 3:
        polar = True
        polar_iterations = 40
    if empirical or empirical_adapt:
        polar = False
    multi = variant.startswith('multi-')
    route_starts = int(variant.split('-')[1]) if multi else 0
    if multi:
        variant = 'tune-90-250'
    tuned = variant.startswith('tune-')
    tuning = tuple(map(float, variant.split('-')[1:])) if tuned else None
    if tuned:
        variant = 'act300' if problem == 3 else 'seed100'
    if q4_action:
        variant = 'act' + str(q4_action_threshold)
    adaptive = variant.startswith('adaptive')
    routed = variant.startswith(
        ('route', 'joint', 'prune', 'compact', 'flex', 'seed', 'act')
    )
    actionlevel = variant.startswith('act')
    seeded = variant.startswith('seed') or actionlevel
    seed_done = False
    if micro:
        seed_done = True
    if prefetch:
        seed_done = True
    flex = variant.startswith('flex')
    prune = variant.startswith('prune')
    if prune:
        from coverage import Coverage

        coverage = Coverage(problem, 20)
    if flex:
        from coverage import Coverage

        coverage = Coverage(problem, 40 if problem == 3 else 20)
    joint = variant.startswith('joint')
    lateral = routed and 'side' in variant
    if adaptive:
        from coverage import Coverage

        coverage = Coverage(problem)
    anchors = list(core.POINTS) if problem == 3 else hex_grid()
```

```
    if rings:
        assert problem == 4
        from discovery import ring_grid

        anchors = ring_grid()
    if seeded and problem == 3:
        anchors = [(0.0, 0.0)] + [
            (1150 * math.cos(k * math.pi / 3), 1150 * math.sin(k * math.pi / 3))
            for k in range(6)
        ]
    if flex and problem == 3:
        anchors = [(0.0, 0.0)] + [
            (1250 * math.cos(k * math.pi / 3), 1250 * math.sin(k * math.pi / 3))
            for k in range(6)
        ]
    if variant.startswith('compact') and problem == 3:
        r = float(variant.removeprefix('compact'))
        anchors = [(0.0, 0.0)] + [
            (r * math.cos(k * math.pi / 3), r * math.sin(k * math.pi / 3))
            for k in range(6)
        ]
    if variant == 'active36' and problem == 4:
        anchors = [
            (float(x), float(y))
            for x in range(-1800, 1801, 720)
            for y in range(-1800, 1801, 720)
        ]
    fallback_anchors = list(anchors)
    if empirical:
        if empirical_parts[0] == 'ring':
            n = int(empirical_parts[1])
            r = float(empirical_parts[2])
            anchors = [(0.0, 0.0)] + [
                (
                    round(r * math.cos(2 * math.pi * k / n), 6),
                    round(r * math.sin(2 * math.pi * k / n), 6),
                )
                for k in range(n)
            ]
        elif empirical_parts[0] == 'box':
            x, y, central = map(float, empirical_parts[1:])
            anchors = [(0.0, central), (0.0, -central)] + [
                (a * x, b * y) for a in (-1, 1) for b in (-1, 1)
            ]
        elif empirical_parts[0] == 'q4':
            from discovery import ring_grid

            outer, inner, ri, ro = map(float, empirical_parts[1:])
            anchors = ring_grid(
                outer=int(outer), inner=int(inner), r_inner=ri, r_outer=ro
            )
        else:
            raise ValueError('unknown empirical geometry')
    remaining = list(anchors)
    if safety_check:
        from empirical_discovery import EmpiricalDiscovery

        safety_prior = EmpiricalDiscovery(problem, 0, 13, 2)
    if lookahead and _forced is None:
        from empirical_discovery import EmpiricalDiscovery
```

```
        rollout_prior = EmpiricalDiscovery(problem, 0, 12)
    if empirical_adapt:
        from empirical_discovery import EmpiricalDiscovery

        empirical_design = EmpiricalDiscovery(
            problem,
            empirical_tolerance,
            empirical_power,
            prior_guard_level or stress_level,
        )
    if informed or forecast or rollout:
        from search_prior import SearchPrior

        discovery_prior = SearchPrior(problem)
    if angular:
        assert problem == 3
        from angular import AngularCover

        angle_cover = AngularCover()
        remaining = [(0.0, 0.0)]
    if polar:
        assert problem == 3
        ring = anchors[1:]
    if adaptive:
        # 密集保底回路支持保守单元证书
        remaining = [
            (float(x), float(y))
            for x in range(-2000, 2001, 500)
            for y in range(-2000, 2001, 500)
            if math.hypot(x, y) < 2600
        ]
    position = (0.0, 0.0)
    cleared = set()
    known = {}
    history = {c: [] for c in range(1, 21)}
    counts = {
        'discovery': 0,
        'localize': 0,
        'exploratory_clear': 0,
        'guaranteed_clear': 0,
        'fallback_clear': 0,
    }
    if value_policy:
        counts.update(value_decisions=0, value_changes=0)
    progress = {}
    negative_counts = {}
    fallbacks = {}
    geometry_cache = {}

    def source_geometry(c):
        geometry = center_radius(known[c])
        if not negative_filter:
            return geometry
        key = len(history[c])
        if c not in geometry_cache or geometry_cache[c][0] != key:
            if correlated:
                from correlated_bearing import estimate
            else:
                from belief_filter import estimate
            geometry_cache[c] = (
                key,
```

```
                estimate(known[c], history[c], problem, geometry, filter_mode),
            )
        return geometry_cache[c][1]

    def send(kind, p, c, purpose):
        nonlocal position
        p = tuple(round(float(v), 6) for v in p)
        obs = action(kind, p, c)
        if lookahead and _forced is None:
            action_prefix.append((kind, p, c, dict(obs)))
        counts[purpose] += 1
        history[c].append((kind, p, obs))
        position = p
        if kind == 'measure' and obs['kind'] in ('near', 'direction'):
            known[c] = (
                core.initial_polygon(p, obs)
                if c not in known
                else core.posterior(known[c], p, obs)
            )
            if belief_mode in ('disk', 'both'):
                for k in range(64):
                    normal = (math.cos(k * math.pi / 32), math.sin(k * math.pi / 32))
                    known[c] = core.clip(known[c], normal, 1800 + 1e-5)
            if not known[c]:
                raise ValueError('empty belief')
        if kind == 'clear' and obs['kind'] == 'success':
            cleared.add(c)
            known.pop(c, None)
        return obs

    def measure_once(p, c, purpose):
        p = tuple(round(float(v), 6) for v in p)
        for kind, q, obs in history[c]:
            if kind == 'measure' and q == p:
                return obs
        return send('measure', p, c, purpose)

    def scan(p):
        prior_known = set(known)
        for c in range(1, 21):
            if c not in cleared and (not (adaptive or prune) or c not in known):
                if (
                    skip_known
                    and c in known
                    and center_radius(known[c])[1] < skip_radius
                ):
                    continue
                measure_once(p, c, 'discovery' if c not in known else 'localize')
        if adaptive or prune or flex:
            coverage.add(p)
        if angular:
            angle_cover.add(p)
        if informed or forecast or rollout:
            discovery_prior.add(p)
        if empirical_adapt:
            empirical_design.add(p)
        if safety_check:
            safety_prior.add(p)
        if lookahead and _forced is None:
            rollout_prior.add(p)
        if (
```

```
            micro
            and not ((service_scan or harvest) and scan_at_service)
            and not (staged and stage_scans < stage_limit)
        ):
            newly = [
                c
                for c in known
                if c not in prior_known and center_radius(known[c])[1] > 100
            ]
            if newly:
                angles = [k * math.pi / 6 for k in range(12)]
                angles += [
                    math.atan2(
                        center_radius(known[c])[0][1] - p[1],
                        center_radius(known[c])[0][0] - p[0],
                    )
                    + sign * math.pi / 2
                    for c in newly
                    for sign in (-1, 1)
                ]
                options = [
                    (
                        p[0] + micro_distance * math.cos(a),
                        p[1] + micro_distance * math.sin(a),
                    )
                    for a in angles
                ]
                if micro_route:
                    options = [p] + [
                        (p[0] + r * math.cos(a), p[1] + r * math.sin(a))
                        for r in (75, 150, 300, 450)
                        for a in angles
                    ]
                    from routing import route

                    route_nodes = [('source', c, source_geometry(c)[0]) for c in known]
                    route_nodes += [('anchor', anchor, anchor) for anchor in remaining]

                def score(q):
                    gain = 0.0
                    for c in newly:
                        mid, r, *_ = center_radius(known[c])
                        bearing = (
                            math.degrees(math.atan2(mid[1] - q[1], mid[0] - q[0])) % 360
                        )
                        poly = core.posterior(
                            known[c], q, dict(kind='direction', bearing=bearing)
                        )
                        if poly:
                            gain += math.log(max(r, 1) / max(center_radius(poly)[1], 1))
                    if micro_route:
                        tour = route(route_nodes, q, 1)
                        path = [p, q] + [n[2] for n in tour]
                        length = sum(math.dist(a, b) for a, b in zip(path, path[1:]))
                        return micro_route_weight * gain - length
                    return gain

                if travel_micro:
                    targets = [center_radius(known[c])[0] for c in newly]
                    if travel_mode < 4:
                        target = min(targets, key=lambda t: math.dist(p, t))
```

```
                    a = math.atan2(target[1] - p[1], target[0] - p[0])
                    fraction = (0.5, 0.75, 1.0, 1.25)[travel_mode]
                    near = tuple(
                        p[k] + fraction * (target[k] - p[k]) for k in (0, 1)
                    )
                    options = [
                        (
                            near[0] + sign * 50 * math.sin(a),
                            near[1] - sign * 50 * math.cos(a),
                        )
                        for sign in (-1, 1)
                    ]
                else:
                    from routing import route

                    route_nodes = [
                        ('source', c, source_geometry(c)[0]) for c in known
                    ]
                    route_nodes += [
                        ('anchor', anchor, anchor) for anchor in remaining
                    ]
                    for target in targets:
                        a = math.atan2(target[1] - p[1], target[0] - p[0])
                        for fraction in (0.5, 0.75, 1.0):
                            near = tuple(
                                p[k] + fraction * (target[k] - p[k]) for k in (0, 1)
                            )
                            options.extend(
                                (
                                    near[0] + sign * 50 * math.sin(a),
                                    near[1] - sign * 50 * math.cos(a),
                                )
                                for sign in (-1, 1)
                            )
                    original_score = score

                    def score(q):
                        tour = route(route_nodes, q, 1)
                        path = [p, q] + [n[2] for n in tour]
                        length = sum(
                            math.dist(a, b) for a, b in zip(path, path[1:])
                        )
                        return (100, 300, 600, 1000)[
                            travel_mode - 4
                        ] * original_score(q) - length

            q = max(options, key=score)
            for c in newly:
                measure_once(q, c, 'localize')

    def clear_source(c):
        if stationary:
            mid, rad, *_ = center_radius(known[c])
            if rad > 30 and not any(
                kind == 'measure' and math.dist(q, position) < 0.01
                for kind, q, o in history[c]
            ):
                angle = (
                    math.degrees(math.atan2(mid[1] - position[1], mid[0] - position[0]))
                    % 360
                )
```

```
                predicted = core.posterior(
                    known[c], position, dict(kind='direction', bearing=angle)
                )
                if predicted and center_radius(predicted)[1] < rad * stationary_ratio:
                    measure_once(position, c, 'localize')
                    return
        if actionlevel and c in fallbacks:
            q = min(fallbacks[c], key=lambda p: math.dist(position, p))
            fallbacks[c].remove(q)
            if send('clear', q, c, 'fallback_clear')['kind'] == 'success':
                return
            if not fallbacks[c]:
                raise ValueError('complete cover exhausted without success')
            return
        negatives = negative_counts.get(c, 0) if actionlevel else 0
        start = progress.get(c, 0) if actionlevel else 0
        for attempt in range(start, min(start + 1, 8) if actionlevel else 8):
            progress[c] = attempt + 1
            middle, radius, u, v, width = source_geometry(c)
            safe_radius = (
                max(math.dist(middle, p) for p in known[c])
                if negative_filter
                else radius
            )
            if radius < 19.8 or (
                radius < exploratory_radius and (attempt >= 1 or cleartry)
            ):
                clear_point = middle
                if merged and merged_mode and safe_radius < 19.8:
                    distance = math.dist(position, middle)
                    step = min(distance, 19.99 - safe_radius)
                    if distance > 1e-8:
                        clear_point = tuple(
                            middle[k] + (position[k] - middle[k]) * step / distance
                            for k in (0, 1)
                        )
                obs = send(
                    'clear',
                    clear_point,
                    c,
                    'guaranteed_clear' if safe_radius < 19.8 else 'exploratory_clear',
                )
                if obs['kind'] == 'success':
                    return
            # 几何驱动的横向测量，从最近端进入
            offset = min(120, max(30, width * 0.15))
            advance = min(100, width * 0.20)
            if tuned:
                offset = min(tuning[0], max(10, width * 0.15))
                advance = min(tuning[1], width * 0.20)
            if controls:
                offset = min(tuning[0], max(10, width * control_values[2]))
                advance = min(tuning[1], width * control_values[3])
            candidates = [
                (
                    middle[0] + sign * v[0] * offset + end * u[0] * advance,
                    middle[1] + sign * v[1] * offset + end * u[1] * advance,
                )
                for sign in (-1, 1)
                for end in (-1, 1)
            ]
```

```
            if lateral and attempt == 0 and radius > 100:
                positives = [
                    (q, o)
                    for kind, q, o in history[c]
                    if kind == 'measure' and o['kind'] == 'direction'
                ]
                if len(positives) == 1:
                    first, o = positives[0]
                    along, across = core.axes(o['bearing'])
                    distance = float(variant.split('side')[-1] or 150)
                    candidates = [
                        (
                            first[0] + sign * across[0] * distance + along[0] * 50,
                            first[1] + sign * across[1] * distance + along[1] * 50,
                        )
                        for sign in (-1, 1)
                    ]
            if visible_probe and problem == 4:
                positives = [
                    p
                    for k, p, o in history[c]
                    if k == 'measure' and o['kind'] in ('direction', 'near')
                ]
                for p in positives:
                    distance = math.dist(p, middle)
                    if distance < 1:
                        continue
                    along = tuple((p[k] - middle[k]) / distance for k in (0, 1))
                    across = (-along[1], along[0])
                    for side in (-1, 1):
                        for forward in (0.5, 1.0, 2.0):
                            candidates.append(
                                tuple(
                                    middle[k]
                                    + forward * offset * along[k]
                                    + side * offset * 0.5 * across[k]
                                    for k in (0, 1)
                                )
                            )
            used = [q for kind, q, o in history[c] if kind == 'measure']
            candidates = [
                p for p in candidates if all(math.dist(p, q) > 2 for q in used)
            ]
            if not candidates:
                break
            if visible_probe and problem == 4:
                from visibility_probe import probabilities

                ps = probabilities(
                    known[c], history[c], (middle, radius, u, v, width), candidates
                )
                penalty = (0, 200, 600, 1500, 5000)[visible_mode]
                q = min(
                    zip(candidates, ps),
                    key=lambda pair: math.dist(position, pair[0])
                    + penalty * (1 - pair[1]),
                )[0]
            else:
                q = min(candidates, key=lambda p: math.dist(position, p))
            obs = measure_once(q, c, 'localize')
            if actionlevel and int(variant.removeprefix('act')):
```

```
                for other in list(known):
                    if other == c:
                        continue
                    _, other_radius, *_ = center_radius(known[other])
                    if other_radius > int(variant.removeprefix('act')):
                        measure_once(q, other, 'localize')
            if joint:
                for other in list(known):
                    if other == c:
                        continue
                    _, other_radius, *_ = center_radius(known[other])
                    threshold = float(variant.removeprefix('joint') or 40)
                    if other_radius > threshold:
                        measure_once(q, other, 'localize')
            if obs['kind'] == 'no_signal':
                negatives += 1
            negative_counts[c] = negatives
            if negatives >= negative_limit:
                break
        if actionlevel and progress.get(c, 0) < 8 and negatives < negative_limit:
            return
        if c in cleared:
            return
        if bracket and negatives >= negative_limit:
            positives = [
                (p, o)
                for k, p, o in history[c]
                if k == 'measure' and o['kind'] == 'direction'
            ]
            negatives_seen = [
                p
                for k, p, o in history[c]
                if k == 'measure' and o['kind'] == 'no_signal'
            ]
            if positives and negatives_seen:
                p, o = min(positives, key=lambda pair: math.dist(position, pair[0]))
                along, _ = core.axes(o['bearing'])
                distances = [
                    core.dot((q[0] - p[0], q[1] - p[1]), along) for q in negatives_seen
                ]
                distances = [d for d in distances if d > 10]
                if distances:
                    low = 0.0
                    high = min(distances)
                    q = (p[0] + along[0] * high, p[1] + along[1] * high)
                    first = measure_once(q, c, 'localize')
                    if first['kind'] == 'no_signal':
                        for _ in range(bracket_steps):
                            mid = (low + high) / 2
                            q = (p[0] + along[0] * mid, p[1] + along[1] * mid)
                            if high - low < 35:
                                if (
                                    send('clear', q, c, 'exploratory_clear')['kind']
                                    == 'success'
                                ):
                                    return
                                break
                            obs = measure_once(q, c, 'localize')
                            if obs['kind'] == 'no_signal':
                                high = mid
                            else:
```

```
                                low = mid
                            if obs['kind'] == 'near':
                                if (
                                    send('clear', q, c, 'exploratory_clear')['kind']
                                    == 'success'
                                ):
                                    return
        # 纯探索估计不计入保证清除
        middle, *_ = source_geometry(c)
        if send('clear', middle, c, 'exploratory_clear')['kind'] == 'success':
            return
        if actionlevel:
            if c not in fallbacks:
                fallbacks[c] = list(core.cover(known[c], position, position).points)
            plan = [fallbacks[c].pop(0)] if fallbacks[c] else []
        else:
            plan = core.cover(known[c], position, position).points
        for q in plan:
            if send('clear', q, c, 'fallback_clear')['kind'] == 'success':
                return
        if actionlevel and fallbacks.get(c):
            return
        raise ValueError('complete cover exhausted without success')

    # 每个锚点都会访问，但活动源服务和后续锚点顺序
    # 由当前回执几何决定，不依赖隐藏源数停止
    scan_at_service = False
    while (
        remaining
        or known
        or (empirical and not empirical_guard_done)
        or (safety_check and not safety_done)
    ):
        if len(cleared) == 16:
            return dict(completed=True, reason='maximum_count_reached', counts=counts)
        if safety_check and not remaining and not known:
            if not safety_prior.unseen.any():
                safety_done = True
                break
            safety_mode = True
            points = safety_prior.points[safety_prior.unseen]
            choices = [tuple(p) for p in np.unique(points, axis=0)]
            if len(choices) > 128:
                choices = choices[:: max(1, len(choices) // 128)]
            choices += fallback_anchors

            def guard_gain(p):
                hit = int((safety_prior.unseen & safety_prior.visible(p)).sum())
                return hit / (math.dist(position, p) + 200)

            remaining = [max(choices, key=guard_gain)]
            if guard_gain(remaining[0]) <= 0:
                raise ValueError('safety sample cover stalled')
        if empirical and not remaining and not known:
            empirical_guard_done = True
            if len(cleared) < 10:
                remaining = list(fallback_anchors)
            else:
                break
        if count_stop and len(cleared) + len(known) == 16:
            remaining = []
```

```
        if staged and remaining and stage_scans < stage_limit:
            from routing import route

            q = route([('anchor', p, p) for p in remaining], position, 8)[0][1]
            remaining.remove(q)
            scan(q)
            stage_scans += 1
            continue
        if prefetch and remaining and counts['discovery'] and prefetch_count:
            q = min(remaining, key=lambda p: math.dist(position, p))
            remaining.remove(q)
            scan(q)
            prefetch_count -= 1
            continue
        if seeded and not seed_done and known:
            seed_done = True
            distance = 100.0 if actionlevel else float(variant.removeprefix('seed'))
            options = [
                (
                    distance * math.cos(k * math.pi / 6),
                    distance * math.sin(k * math.pi / 6),
                )
                for k in range(12)
            ]

            def score(q):
                gain = 0
                for c in known:
                    mid, r, *_ = center_radius(known[c])
                    bearing = (
                        math.degrees(math.atan2(mid[1] - q[1], mid[0] - q[0])) % 360
                    )
                    poly = core.posterior(
                        known[c], q, dict(kind='direction', bearing=bearing)
                    )
                    if poly:
                        new = center_radius(poly)[1]
                        gain += max(0, math.log(max(r, 1) / max(new, 1)))
                return gain

            q = max(options, key=score)
            for c in list(known):
                measure_once(q, c, 'localize')
        if angular and angle_cover.points:
            remaining = angle_cover.pending(position)
            if not remaining and not known:
                break
        if flex and remaining and known:
            # 将发现路点移向回执推得的目标
            # 仅在整体覆盖计划仍有效时
            for c in known:
                target = center_radius(known[c])[0]
                p = min(remaining, key=lambda p: math.dist(p, target))
                if math.dist(p, target) > 600 or math.dist(p, target) < 5:
                    continue
                for fraction in (1.0, 0.75, 0.5, 0.25):
                    q = tuple(
                        round(p[i] + fraction * (target[i] - p[i]), 6) for i in range(2)
                    )
                    if coverage.covered(
                        coverage.points + [x for x in remaining if x != p] + [q]
```

```
                ).all():
                    remaining.remove(p)
                    remaining.append(q)
                    break
        if prune and remaining:
            for p in sorted(remaining, key=lambda p: -math.dist(position, p)):
                if coverage.covered(
                    coverage.points + [q for q in remaining if q != p]
                ).all():
                    remaining.remove(p)
        if adaptive and coverage.complete and not known:
            return dict(
                completed=True,
                reason='adaptive_coverage_complete',
                counts=counts,
                discovery_points=len(coverage.points),
                cleared=sorted(cleared),
            )
        choice = None
        if empirical_adapt and counts['discovery'] and not safety_mode:
            if cover_plan and len(known) + len(cleared) < 16:
                from probe_cover import choose as choose_cover

                remaining = choose_cover(
                    empirical_design,
                    remaining,
                    fallback_anchors,
                    [source_geometry(c)[0] for c in known],
                    position,
                    30 * (20 - len(known) - len(cleared)) * cover_weight,
                )
            else:
                remaining = empirical_design.adjust(
                    remaining,
                    [source_geometry(c)[0] for c in known],
                    position,
                    empirical_blend,
                )
            if smooth and remaining:
                from smooth_discovery import adjust as smooth_adjust

                remaining = smooth_adjust(
                    empirical_design,
                    remaining,
                    [source_geometry(c)[0] for c in known],
                    position,
                    smooth_passes,
                    smooth_guard,
                )
            if not remaining and not known:
                break
        if polar and known:
            from flexroute import adjust

            ring, remaining = adjust(
                ring,
                remaining,
                [source_geometry(c)[0] for c in known],
                position,
                polar_iterations,
            )
```

```
        if routed:
            nodes = [
                (
                    'source',
                    c,
                    (
                        min(fallbacks[c], key=lambda p: math.dist(position, p))
                        if c in fallbacks
                        else source_geometry(c)[0]
                    ),
                )
                for c in known
            ]
            nodes += [('anchor', p, p) for p in remaining]
            if forecast:
                nodes += [
                    ('forecast', i, p)
                    for i, p in enumerate(
                        discovery_prior.forecast(
                            20 - len(known) - len(cleared), forecast_scale
                        )
                    )
                ]
            if layered and not layer_outer:
                inner_nodes = [
                    n
                    for n in nodes
                    if math.hypot(*n[2])
                    < (
                        layer_radius
                        if n[0] == 'source'
                        else (1100 if problem == 4 else 1)
                    )
                ]
                if inner_nodes:
                    nodes = inner_nodes
                else:
                    layer_outer = True
            pending = list(nodes)
            tour = []
            last = position
            while pending:
                n = min(pending, key=lambda n: math.dist(last, n[2]))
                pending.remove(n)
                tour.append(n)
                last = n[2]
            for _ in range(5):
                improved = False
                for i in range(len(tour) - 1):
                    a = position if i == 0 else tour[i - 1][2]
                    for j in range(i + 1, len(tour)):
                        b = tour[i][2]
                        cpoint = tour[j][2]
                        before = math.dist(a, b)
                        after = math.dist(a, cpoint)
                        if j + 1 < len(tour):
                            d = tour[j + 1][2]
                            before += math.dist(cpoint, d)
                            after += math.dist(b, d)
                        if after < before - 1e-6:
                            tour[i : j + 1] = reversed(tour[i : j + 1])
```

```
                    improved = True
            if not improved:
                break
        choice = next(n for n in tour if n[0] != 'forecast')
        if multi or (rings and rings_starts):
            from routing import route

            choice = next(
                n
                for n in route(
                    nodes,
                    position,
                    rings_starts if rings else route_starts,
                    scan_weight,
                )
                if n[0] != 'forecast'
            )
        if sticky:
            from routing import route, persistent_route

            if not retained_tour or (
                sticky_mode == 2 and counts['discovery'] != last_discovery
            ):
                retained_tour = route(nodes, position, 8)
            else:
                retained_tour = persistent_route(
                    nodes, retained_tour, position, sticky_mode == 3
                )
            last_discovery = counts['discovery']
            choice = retained_tour[0]
        if navigation:
            from routing import route

            if navigation_bias >= 1000:
                choice = min(nodes, key=lambda n: math.dist(position, n[2]))
            else:

                def cost(first):
                    rest = route([n for n in nodes if n != first], first[2], 2)
                    path = [position, first[2]] + [n[2] for n in rest]
                    return sum(
                        math.dist(a, b) for a, b in zip(path, path[1:])
                    ) + navigation_bias * math.dist(position, first[2])

                choice = min(nodes, key=cost)
        if informed:
            from routing import route

            best_score = math.inf
            for first in nodes:
                q = first[2]
                rest = route([n for n in nodes if n != first], q, 2)
                path = [position, q] + [n[2] for n in rest]
                value = sum(math.dist(a, b) for a, b in zip(path, path[1:]))
                if first[0] == 'anchor':
                    value -= information_weight * discovery_prior.gain(
                        q, 20 - len(known) - len(cleared)
                    )
                if value < best_score:
                    best_score = value
                    choice = first
```

```
        if rollout:
            from rollout_planner import choose

            choice = choose(
                nodes,
                choice,
                position,
                known,
                cleared,
                discovery_prior,
                rollout_worlds,
                rollout_decision,
            )
            rollout_decision += 1
        if value_policy:
            key = (len(known), len(cleared), counts['discovery'])
            if known and len(nodes) > 2 and key != last_value_key:
                from routing_features import candidates_and_features
                from routing_model_v1 import predict

                options, features = candidates_and_features(
                    dict(
                        problem=problem,
                        position=position,
                        nodes=nodes,
                        choice=choice,
                        known=known,
                        cleared=cleared,
                        history=history,
                    )
                )
                predicted = [predict(problem, x) for x in features]
                best = min(range(len(options)), key=lambda i: predicted[i])
                counts['value_decisions'] += 1
                if predicted[best] < predicted[0] - value_margin:
                    choice = options[best]
                    counts['value_changes'] += int(best != 0)
                last_value_key = key
        if _forced is not None and decision_index < len(_forced):
            selected = _forced[decision_index]
            matches = [n for n in nodes if n[0] == selected[0] and n[1] == selected[1]]
            assert len(matches) == 1, 'forced node mismatch'
            choice = matches[0]
        if lookahead and _forced is None:
            key = (len(cleared), counts['discovery'])
            if known and len(nodes) > 2 and key != last_plan_key:
                from policy_lookahead import choose

                choice, diagnostic = choose(
                    solve,
                    problem,
                    lookahead_base,
                    nodes,
                    choice,
                    known,
                    cleared,
                    history,
                    rollout_prior,
                    action_prefix,
                    forced_history,
                    lookahead_worlds,
```

```
                    decision_index,
                )
                last_plan_key = key
        if _observer is not None:
            _observer(
                dict(
                    problem=problem,
                    decision=decision_index,
                    position=position,
                    nodes=nodes,
                    choice=choice,
                    known=known,
                    cleared=cleared,
                    history=history,
                    counts=counts,
                    forced=forced_history,
                )
            )
        forced_history.append(choice)
        decision_index += 1
        if known and (not routed or choice[0] == 'source'):
            c = min(
                known, key=lambda c: math.dist(position, center_radius(known[c])[0])
            )
            if routed:
                c = choice[1]
            target = center_radius(known[c])[0]
            nearest = (
                min(remaining, key=lambda p: math.dist(position, p))
                if remaining
                else None
            )
            if (
                routed
                or adaptive
                or nearest is None
                or math.dist(position, target) < math.dist(position, nearest) * 1.3
            ):
                clear_source(c)
                if harvest and c in cleared and remaining:
                    from routing import route

                    centers = [source_geometry(channel)[0] for channel in known]

                    def planned_length(probes):
                        nodes = [('source', i, p) for i, p in enumerate(centers)] + [
                            ('anchor', p, p) for p in probes
                        ]
                        tour = route(nodes, position, 4)
                        path = [position] + [n[2] for n in tour]
                        return sum(math.dist(a, b) for a, b in zip(path, path[1:]))

                    old_length = planned_length(remaining)
                    unseen_before = empirical_design.unseen.copy()
                    empirical_design.unseen &= ~empirical_design.visible(position)
                    trial = empirical_design.adjust(
                        remaining, centers, position, empirical_blend
                    )
                    empirical_design.unseen = unseen_before
                    channels_now = sum(
                        channel not in cleared
```

```
                        and not (
                            skip_known
                            and channel in known
                            and center_radius(known[channel])[1] < skip_radius
                        )
                        for channel in range(1, 21)
                    )
                    # 用公开源数先验估计缺失频道代价
                    # 不使用场景真实源数
                    absent = max(4, 20 - max(13, len(cleared) + len(known)))
                    action_delta = 30 * (
                        channels_now - absent * (len(remaining) - len(trial))
                    )
                    if (
                        old_length - planned_length(trial)
                        > action_delta + harvest_margin
                    ):
                        scan_at_service = True
                        scan(position)
                        scan_at_service = False
                        remaining = trial
                if service_scan and c in cleared and remaining:
                    hit = int(
                        (
                            empirical_design.unseen & empirical_design.visible(position)
                        ).sum()
                    )
                    if hit / len(empirical_design.unseen) >= service_values[1]:
                        scan_at_service = True
                        scan(position)
                        scan_at_service = False
                if (
                    angular
                    and c in cleared
                    and angle_cover.gain(position) > angular_gain
                ):
                    scan(position)
                if prune and remaining:
                    scan(position)
                if adaptive and not coverage.complete:
                    scan(position)
                if variant in ('opportunistic', 'opportunistic36'):
                    scan(position)
                continue
        if routed:
            q = choice[1]
        elif adaptive:
            q = max(
                remaining,
                key=lambda p: coverage.gain(p)
                / (
                    math.dist(position, p)
                    + float(variant.removeprefix('adaptive') or 300)
                ),
            )
        else:
            q = min(remaining, key=lambda p: math.dist(position, p))
        remaining.remove(q)
        scan(q)
    if angular:
        assert angle_cover.points and not angle_cover.gaps()
```

```
        return dict(
            completed=True,
            reason='angular_annulus_covered',
            counts=counts,
            discovery_points=len(angle_cover.points),
            discovery_positions=angle_cover.points,
            cleared=sorted(cleared),
        )
    return dict(
        completed=True,
        reason=(
            'empirical_scan_exhausted'
            if empirical or empirical_adapt
            else 'guaranteed_grid_exhausted'
        ),
        counts=counts,
        discovery_points=len(anchors),
        cleared=sorted(cleared),
    )
```

policy_lookahead.py

```
# 前瞻策略包装

import math
import numpy as np

def sample_known(poly, history, problem, rng):
    polygon = np.asarray(poly)
    triangles = np.asarray(
        [(polygon[0], polygon[j], polygon[j + 1]) for j in range(1, len(polygon) - 1)]
    )
    if not len(triangles):
        point = polygon.mean(axis=0)
        return dict(point=point, radius=1500.0, direction=0.0, omni=True, fallback=True)
    sides = triangles[:, 1:] - triangles[:, :1]
    areas = np.abs(sides[:, 0, 0] * sides[:, 1, 1] - sides[:, 0, 1] * sides[:, 1, 0])
    weights = (
        areas / areas.sum()
        if areas.sum() > 1e-12
        else np.full(len(areas), 1 / len(areas))
    )
    n = 4096
    selected = triangles[rng.choice(len(triangles), n, p=weights)]
    z = rng.random((n, 2))
    s = np.sqrt(z[:, 0])
    points = (
        (1 - s[:, None]) * selected[:, 0]
        + (s * (1 - z[:, 1]))[:, None] * selected[:, 1]
        + (s * z[:, 1])[:, None] * selected[:, 2]
    )
    radius = 1000 + 500 * rng.random(n)
    angles = rng.uniform(-math.pi, math.pi, n)
    axes = np.column_stack([np.cos(angles), np.sin(angles)])
    omni = np.ones(n, dtype=bool) if problem == 3 else rng.random(n) < 0.5
    valid = (points * points).sum(axis=1) <= 1770**2 + 1e-5
    likelihood = np.zeros(n)
    for kind, p, o in history:
        d = np.asarray(p) - points
```

```
        distance = np.linalg.norm(d, axis=1)
        visible = (distance <= radius) & (omni | ((d * axes).sum(axis=1) >= 0))
        if kind == 'clear':
            if o['kind'] != 'success':
                valid &= distance > 20 - 1e-5
        elif o['kind'] == 'no_signal':
            valid &= ~visible
        else:
            valid &= visible
            if o['kind'] == 'near':
                valid &= distance <= 5 + 1e-5
            else:
                error = (
                    np.degrees(np.arctan2(-d[:, 1], -d[:, 0])) - o['bearing'] + 180
                ) % 360 - 180
                valid &= np.abs(error) <= 1.011
                likelihood -= 0.5 * (error / 0.4) ** 2
    ix = np.flatnonzero(valid)
    fallback = not len(ix)
    if fallback:
        # 仅用于预测的保底，不改变真实源多边形
        positives = [
            np.asarray(p)
            for kind, p, o in history
            if kind == 'measure' and o['kind'] != 'no_signal'
        ]
        point = polygon.mean(axis=0)
        direction = (
            np.mean(positives, axis=0) - point if positives else np.array([1.0, 0.0])
        )
        return dict(
            point=point,
            radius=1500.0,
            direction=math.atan2(direction[1], direction[0]),
            omni=problem == 3,
            fallback=True,
        )
    w = np.exp(likelihood[ix] - likelihood[ix].max())
    i = int(rng.choice(ix, p=w / w.sum()))
    return dict(
        point=points[i],
        radius=float(radius[i]),
        direction=float(angles[i]),
        omni=bool(omni[i]),
        fallback=False,
    )

def sample_worlds(problem, known, cleared, history, prior, number, decision):
    rng = np.random.default_rng(911223 + decision * 193)
    k = len(known) + len(cleared)
    mass = float(prior.unseen.mean())
    counts = np.arange(max(10, k), 17)
    probabilities = np.array(
        [math.comb(int(n), k) * max(mass, 1e-12) ** (int(n) - k) for n in counts]
    )
    probabilities /= probabilities.sum()
    pool = np.flatnonzero(prior.unseen)
    channels = sorted(set(range(1, 21)) - set(known) - set(cleared))
    worlds = []
    for _ in range(number):
        n = int(rng.choice(counts, p=probabilities)) if len(pool) else k
```

```
        world = {
            c: sample_known(poly, history[c], problem, rng) for c, poly in known.items()
        }
        for c in rng.choice(channels, n - k, replace=False) if n > k else []:
            i = int(rng.choice(pool))
            world[int(c)] = dict(
                point=prior.points[i],
                radius=float(math.sqrt(prior.receive2[i])),
                direction=math.atan2(prior.axes[i, 1], prior.axes[i, 0]),
                omni=bool(prior.omni[i]),
                fallback=False,
            )
        worlds.append((world, max(n, 1), float(rng.uniform(0, 2 * math.pi))))
    return worlds

def choose(
    solve,
    problem,
    base_variant,
    nodes,
    default,
    known,
    cleared,
    history,
    prior,
    prefix,
    forced,
    world_count,
    decision,
):
    position = prefix[-1][1] if prefix else (0.0, 0.0)
    candidates = [default]
    for kind in ('source', 'anchor'):
        options = [n for n in nodes if n[0] == kind and n not in candidates]
        if options:
            candidates.append(min(options, key=lambda n: math.dist(position, n[2])))
    if len(candidates) < 2:
        return default, dict(simulations=0, fallback_world_sources=0)
    worlds = sample_worlds(
        problem, known, cleared, history, prior, world_count, decision
    )
    scores = []
    fallbacks = sum(s['fallback'] for w, n, phase in worlds for s in w.values())
    for candidate in candidates:
        values = []
        for sources, total, phase in worlds:
            state = dict(index=0, position=position, detector=1, time=0.0, actions=0)
            for kind, p, c, o in prefix:
                if kind == 'measure':
                    state['detector'] = c
            active = dict(sources)

            def action(kind, p, c):
                index = state['index']
                state['index'] += 1
                if index < len(prefix):
                    old_kind, old_p, old_c, old_o = prefix[index]
                    assert (
                        kind == old_kind and c == old_c and math.dist(p, old_p) < 2e-6
                    ), 'prefix mismatch'
                    return dict(old_o)
```

```
                state['actions'] += 1
                if state['actions'] > 750:
                    raise TimeoutError('forecast action cap')
                state['time'] += math.dist(state['position'], p) / 5
                state['position'] = p
                source = active.get(c)
                distance = math.dist(p, source['point']) if source else math.inf
                if kind == 'clear':
                    success = source is not None and distance <= 20
                    state['time'] += 5 if success else 3
                    if success:
                        active.pop(c)
                    return dict(kind='success' if success else 'no_target_in_range')
                state['time'] += 5 + int(state['detector'] != c)
                state['detector'] = c
                visible = source is not None and distance <= source['radius']
                if visible and not source['omni']:
                    d = np.asarray(p) - source['point']
                    visible = (
                        d[0] * math.cos(source['direction'])
                        + d[1] * math.sin(source['direction'])
                        >= 0
                    )
                if not visible:
                    return dict(kind='no_signal')
                if distance <= 5:
                    return dict(kind='near')
                d = source['point'] - np.asarray(p)
                error = (
                    0.6
                    * math.sin(p[0] / 170 + phase + c)
                    * math.cos(p[1] / 170 - phase - c)
                )
                bearing = (math.degrees(math.atan2(d[1], d[0])) + error) % 360
                return dict(kind='direction', bearing=round(bearing, 2) % 360)

            try:
                solve(problem, action, base_variant, _forced=forced + [candidate])
                cost = state['time'] / total + 5000 * len(active)
            except TimeoutError:
                cost = state['time'] / total + 5000 * len(active)
            values.append(cost)
        scores.append(float(np.mean(values)))
    best = int(np.argmin(scores))
    # 要求相对基线有小幅余量，减少蒙特卡洛抖动
    if scores[0] - scores[best] < 2:
        best = 0
    return candidates[best], dict(
        simulations=len(candidates) * len(worlds),
        fallback_world_sources=fallbacks,
        scores=scores,
        changed=best != 0,
    )
```

probe_cover.py

```
# 可见性探测覆盖点选择

import math, warnings
```

```
import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import csc_matrix
from routing import route

def choose(design, remaining, original, centers, position, scan_cost=300):
    if not design.unseen.any():
        return []
    candidates = list(
        dict.fromkeys(
            tuple(round(float(x), 6) for x in p)
            for p in [position] + list(centers) + list(remaining) + list(original)
        )
    )
    masks = np.column_stack([design.visible(p)[design.unseen] for p in candidates])
    useful = masks.any(axis=0)
    masks = masks[:, useful]
    candidates = [p for p, good in zip(candidates, useful) if good]
    if not masks.any(axis=1).all():
        return remaining
    # 相同覆盖行定义同一二元约束
    _, indices = np.unique(np.packbits(masks, axis=1), axis=0, return_index=True)
    matrix = masks[indices].astype(float)
    service = route([('source', i, p) for i, p in enumerate(centers)], position, 4)
    path = [position] + [n[2] for n in service]
    costs = []
    for j, p in enumerate(candidates):
        insertion = math.dist(path[-1], p)
        for a, b in zip(path, path[1:]):
            insertion = min(
                insertion, math.dist(a, p) + math.dist(p, b) - math.dist(a, b)
            )
        costs.append(scan_cost + max(0, insertion) + j * 1e-7)
    with warnings.catch_warnings():
        warnings.filterwarnings('ignore', message='Unrecognized options detected')
        result = milp(
            np.array(costs),
            integrality=np.ones(len(costs)),
            bounds=Bounds(0, 1),
            constraints=LinearConstraint(csc_matrix(matrix), 1, np.inf),
            options=dict(time_limit=0.5, mip_rel_gap=0.01, threads=1),
        )
    if result.x is None:
        return remaining
    chosen = result.x > 0.5
    if not masks[:, chosen].any(axis=1).all():
        return remaining
    return [p for p, keep in zip(candidates, chosen) if keep]
```

rollout_planner.py

```
# 事后事件推演规划

import math
import numpy as np
from routing import route

def sample_polygon(poly, rng):
    p = np.array(poly)
```

```
    origin = p[0]
    areas = np.abs(np.cross(p[1:-1] - origin, p[2:] - origin))
    if areas.sum() < 1e-8:
        return tuple(p.mean(axis=0))
    for _ in range(12):
        i = int(rng.choice(len(areas), p=areas / areas.sum())) + 1
        a, b = rng.random(2)
        if a + b > 1:
            a, b = 1 - a, 1 - b
        point = origin + a * (p[i] - origin) + b * (p[i + 1] - origin)
        if np.linalg.norm(point) <= 1800:
            return tuple(point)
    return tuple(p.mean(axis=0))

def sample_world(prior, known, cleared, rng):
    known_count = len(known) + len(cleared)
    active_weights = prior.weights * prior.active
    mass = float(active_weights.sum())
    choices = list(range(max(10, known_count), 17))
    probs = np.array(
        [
            math.comb(20 - known_count, n - known_count)
            / math.comb(20, n)
            * mass ** (n - known_count)
            for n in choices
        ]
    )
    if not probs.sum():
        number = known_count
    else:
        number = int(rng.choice(choices, p=probs / probs.sum()))
    sources = {c: (sample_polygon(poly, rng), 1500, -1) for c, poly in known.items()}
    channels = [c for c in range(1, 21) if c not in known and c not in cleared]
    missing = number - known_count
    if missing:
        selected = rng.choice(channels, size=missing, replace=False)
        particles = rng.choice(
            len(active_weights), size=missing, p=active_weights / mass
        )
        for c, j in zip(selected, particles):
            sources[int(c)] = (
                tuple(prior.points[j]),
                float(prior.radii[j]),
                int(prior.directions[j]),
            )
    return sources, number

def visible(source, p):
    g, r, theta = source
    dx = p[0] - g[0]
    dy = p[1] - g[1]
    return math.hypot(dx, dy) <= r and (
        theta < 0
        or dx * math.cos(theta * math.pi / 6) + dy * math.sin(theta * math.pi / 6) >= 0
    )

def future_cost(first, anchors, estimates, sources, position, cleared_count, number):
    remaining = list(anchors)
    known = dict(estimates)
    hidden = {c: s for c, s in sources.items() if c not in known}
    cost = 0.0
```

```
    choice = first
    newly_cleared = 0
    while remaining or known:
        kind, key, estimate = choice
        if kind == 'source':
            # 已知源在其估计位置仍需信息
            g = sources[key][0]
            cost += math.dist(position, estimate) / 5 + math.dist(estimate, g) / 5 + 5
            position = g
            known.pop(key)
            newly_cleared += 1
        else:
            p = key
            remaining.remove(p)
            cost += math.dist(position, p) / 5 + 6 * (
                20 - cleared_count - newly_cleared - len(known)
            )
            position = p
            found = []
            for c, s in hidden.items():
                if visible(s, p):
                    found.append(c)
            if found:
                cost += 15 + 5 * len(found)
            for c in found:
                known[c] = hidden.pop(c)[0]
        if cleared_count + newly_cleared + len(known) == 16:
            remaining = []
        nodes = [('source', c, p) for c, p in known.items()] + [
            ('anchor', p, p) for p in remaining
        ]
        if not nodes:
            break
        choice = route(nodes, position, 1)[0]
    return cost / max(1, number)

def choose(nodes, baseline, position, known, cleared, prior, worlds, decision):
    candidates = [baseline]
    candidates += sorted(nodes, key=lambda n: math.dist(position, n[2]))[:3]
    anchors = [n for n in nodes if n[0] == 'anchor']
    if anchors:
        candidates.append(
            max(
                anchors,
                key=lambda n: prior.gain(n[2], 20 - len(known) - len(cleared))
                / (100 + math.dist(position, n[2])),
            )
        )
    candidates = list(dict.fromkeys(candidates))
    if len(candidates) < 2:
        return baseline
    rng = np.random.default_rng(19301 + decision)
    samples = [sample_world(prior, known, cleared, rng) for _ in range(worlds)]
    estimates = {n[1]: n[2] for n in nodes if n[0] == 'source'}
    best = None
    best_value = math.inf
    for first in candidates:
        value = (
            sum(
                future_cost(
                    first,
```

```
                    [n[1] for n in anchors],
                    estimates,
                    sources,
                    position,
                    len(cleared),
                    number,
                )
                for sources, number in samples
            )
            / worlds
        )
        if value < best_value:
            best = first
            best_value = value
    return best
```

routing.py

```
# 多起点最近邻与 2-opt 路线

import math
import random

def persistent_route(nodes, previous, position, improve=False):
    pending = list(nodes)
    tour = []
    for old in previous:
        options = [
            n
            for n in pending
            if n[0] == old[0] and (n[0] != 'source' or n[1] == old[1])
        ]
        if not options:
            continue
        n = min(options, key=lambda n: math.dist(n[2], old[2]))
        if n[0] == 'anchor' and math.dist(n[2], old[2]) > 850:
            continue
        pending.remove(n)
        tour.append(n)
    for n in pending:

        def insertion(j):
            a = position if not j else tour[j - 1][2]
            return math.dist(a, n[2]) + (
                math.dist(n[2], tour[j][2]) - math.dist(a, tour[j][2])
                if j < len(tour)
                else 0
            )

        j = min(range(len(tour) + 1), key=insertion)
        tour.insert(j, n)
    if improve:
        # 改进局部顺序时保留远端
        for _ in range(5):
            changed = False
            for i in range(len(tour) - 2):
                a = position if i == 0 else tour[i - 1][2]
                b = tour[i][2]
                for j in range(i + 1, len(tour) - 1):
```

```
                    c = tour[j][2]
                    d = tour[j + 1][2]
                    if (
                        math.dist(a, c) + math.dist(b, d)
                        < math.dist(a, b) + math.dist(c, d) - 1e-6
                    ):
                        tour[i : j + 1] = reversed(tour[i : j + 1])
                        changed = True
                        break
            if not changed:
                break
    return tour

def route(nodes, position, starts=8, scan_weight=0):
    if len(nodes) < 2:
        return nodes
    points = [position] + [n[2] for n in nodes]
    dist = [[math.dist(a, b) for b in points] for a in points]
    source = [False] + [n[0] == 'source' for n in nodes]

    def cost(p):
        total = sum(dist[a][b] for a, b in zip([0] + p, p))
        anchors = 0
        for i in p:
            if source[i]:
                total += scan_weight * anchors
            else:
                anchors += 1
        return total

    def improve(p):
        for iteration in range(15):
            gain = 0
            change = None
            for i in range(len(p) - 1):
                a = 0 if i == 0 else p[i - 1]
                b = p[i]
                anchors = int(not source[b])
                sources = int(source[b])
                inversions = 0
                for j in range(i + 1, len(p)):
                    c = p[j]
                    d = p[j + 1] if j + 1 < len(p) else None
                    if source[c]:
                        inversions += anchors
                        sources += 1
                    else:
                        anchors += 1
                    delta = dist[a][c] - dist[a][b]
                    if d is not None:
                        delta += dist[b][d] - dist[c][d]
                    delta += scan_weight * (anchors * sources - 2 * inversions)
                    if delta < gain - 1e-6:
                        gain = delta
                        change = (i, j)
            if change is None:
                break
            i, j = change
            p[i : j + 1] = reversed(p[i : j + 1])
        # 单点重定位可跳出 2-opt 局部最优
        for iteration in range(8):
```

```
            gain = 0
            change = None
            for i, x in enumerate(p):
                a = 0 if i == 0 else p[i - 1]
                b = p[i + 1] if i + 1 < len(p) else None
                removed = -dist[a][x] + (
                    dist[a][b] - dist[x][b] if b is not None else 0
                )
                short = p[:i] + p[i + 1 :]
                for j in range(len(short) + 1):
                    c = 0 if j == 0 else short[j - 1]
                    d = short[j] if j < len(short) else None
                    delta = (
                        removed
                        + dist[c][x]
                        + (dist[x][d] - dist[c][d] if d is not None else 0)
                    )
                    if scan_weight:
                        crossed = p[i + 1 : j + 1] if j > i else p[j:i]
                        inversion_delta = sum(
                            int(not source[k]) if source[x] else -int(source[k])
                            for k in crossed
                        )
                        delta += scan_weight * inversion_delta * (1 if j > i else -1)
                    if delta < gain - 1e-6:
                        gain = delta
                        change = (i, j)
            if change is None:
                break
            i, j = change
            x = p.pop(i)
            p.insert(j, x)
        return p

    randomizer = random.Random(193)
    best = None
    best_cost = math.inf
    for attempt in range(starts):
        pending = list(range(1, len(points)))
        p = []
        last = 0
        while pending:
            options = sorted(pending, key=lambda i: dist[last][i])
            k = (
                0
                if attempt == 0
                else randomizer.randrange(min(3, len(options))) if len(p) < 4 else 0
            )
            n = options[k]
            p.append(n)
            pending.remove(n)
            last = n
        p = improve(p)
        value = cost(p)
        if value < best_cost:
            best = p
            best_cost = value
    return [nodes[i - 1] for i in best]
```

search_prior.py

```
# 搜索先验

import math
import numpy as np

class SearchPrior:
    def __init__(self, problem):
        points = []
        radii = []
        directions = []
        weights = []
        for i in range(1, 13):
            r = 1800 * math.sqrt((i - 0.5) / 12)
            for j in range(36):
                p = (r * math.cos(j * math.pi / 18), r * math.sin(j * math.pi / 18))
                for receive in (1000, 1250, 1500):
                    configs = (
                        [(-1, 1.0)]
                        if problem == 3
                        else [(-1, 0.5)] + [(k, 0.5 / 12) for k in range(12)]
                    )
                    for k, w in configs:
                        points.append(p)
                        radii.append(receive)
                        directions.append(k)
                        weights.append(w)
        self.points = np.array(points)
        self.radii = np.array(radii)
        self.directions = np.array(directions)
        self.weights = np.array(weights)
        self.weights /= self.weights.sum()
        self.active = np.ones(len(points), dtype=bool)
        self.centers, self.center_ids = np.unique(
            self.points, axis=0, return_inverse=True
        )

    def visible(self, p):
        delta = p - self.points
        distance = np.linalg.norm(delta, axis=1)
        projection = delta[:, 0] * np.cos(self.directions * math.pi / 6) + delta[
            :, 1
        ] * np.sin(self.directions * math.pi / 6)
        return (distance <= self.radii) & ((self.directions < 0) | (projection >= 0))

    def add(self, p):
        self.active &= ~self.visible(np.array(p))

    def gain(self, p, unknown_channels):
        mass = self.weights[self.active].sum()
        hit = self.weights[self.active & self.visible(np.array(p))].sum()
        return unknown_channels * 0.65 * hit / (0.35 + 0.65 * mass)

    def forecast(self, unknown_channels, scale):
        weights = np.bincount(
            self.center_ids,
            weights=self.weights * self.active,
            minlength=len(self.centers),
        )
        mass = weights.sum()
```

```
        if mass < 1e-10:
            return []
        expected = unknown_channels * 0.65 * mass / (0.35 + 0.65 * mass)
        count = min(8, max(0, round(expected * scale)))
        if not count:
            return []
        centers = [self.centers[int(np.argmax(weights))]]
        for _ in range(count - 1):
            d = (
                ((self.centers[:, None, :] - np.array(centers)[None, :, :]) ** 2)
                .sum(axis=2)
                .min(axis=1)
            )
            centers.append(self.centers[int(np.argmax(weights * d))])
        centers = np.array(centers)
        for _ in range(12):
            labels = (
                ((self.centers[:, None, :] - centers[None, :, :]) ** 2)
                .sum(axis=2)
                .argmin(axis=1)
            )
            for i in range(count):
                mask = labels == i
                if weights[mask].sum():
                    centers[i] = (self.centers[mask] * weights[mask, None]).sum(
                        axis=0
                    ) / weights[mask].sum()
        return [tuple(p) for p in centers]
```

smooth_discovery.py

```
# 平滑发现调整

import math
import numpy as np
from routing import route

def adjust(design, remaining, centers, position, passes=4, guard=0):
    if not remaining:
        return remaining
    source = design.points[design.unseen]
    r2 = design.receive2[design.unseen]
    axes = design.axes[design.unseen]
    omni = design.omni[design.unseen]
    if not len(source):
        return []
    probes = np.asarray(remaining, dtype=float).copy()
    initial = probes.copy()
    for iteration in range(passes):
        delta = probes[None, :, :] - source[:, None, :]
        distance = np.linalg.norm(delta, axis=2)
        projection = (delta * axes[:, None, :]).sum(axis=2)
        margin = np.minimum(
            np.sqrt(r2)[:, None] - distance, np.where(omni[:, None], np.inf, projection)
        )
        owner = margin.argmax(axis=1)
        if (margin.max(axis=1) < -1e-7).any():
            return remaining
        nodes = [('anchor', i, tuple(p)) for i, p in enumerate(probes)]
```

```
        nodes += [('source', i, p) for i, p in enumerate(centers)]
        tour = route(nodes, position, 4 if iteration == 0 else 1)
        moved = False
        for j, node in enumerate(tour):
            if node[0] != 'anchor':
                continue
            i = node[1]
            p = probes[i]

            def live(n):
                return probes[n[1]] if n[0] == 'anchor' else np.asarray(n[2])

            left = np.asarray(position) if j == 0 else live(tour[j - 1])
            right = live(tour[j + 1]) if j + 1 < len(tour) else left
            line = right - left
            length2 = float(line @ line)
            target = (
                left + np.clip(float((p - left) @ line) / length2, 0, 1) * line
                if length2 > 1e-9
                else left
            )
            d = target - p
            a = float(d @ d)
            if a < 1:
                continue
            ix = owner == i
            if not ix.any():
                probes[i] = target
                moved = True
                continue
            relative = p - source[ix]
            b = 2 * (relative @ d)
            dist2 = (relative * relative).sum(axis=1)
            allowed = r2[ix]
            if guard:
                allowed = np.maximum(np.sqrt(dist2), np.sqrt(allowed) - guard) ** 2
            c = dist2 - allowed
            roots = (-b + np.sqrt(np.maximum(0, b * b - 4 * a * c))) / (2 * a)
            fraction = min(1.0, max(0.0, float(roots.min())))
            oriented = ~omni[ix]
            if oriented.any():
                normals = axes[ix][oriented]
                s = (relative[oriented] * normals).sum(axis=1)
                rate = normals @ d
                if guard:
                    s = s - np.minimum(s, guard)
                closing = rate < 0
                if closing.any():
                    fraction = min(
                        fraction, max(0.0, float((s[closing] / -rate[closing]).min()))
                    )
            fraction = max(0.0, fraction - 1e-7)
            if fraction * math.sqrt(a) > 1:
                probes[i] = p + fraction * d
                moved = True
        if not moved:
            break
    changed = [tuple(round(float(x), 6) for x in p) for p in probes]
    # 取整后独立重算，不依赖缓存
    for start in range(0, len(source), 4096):
        d = np.asarray(changed)[None, :, :] - source[start : start + 4096, None, :]
```

```
        visible = (d * d).sum(axis=2) <= r2[start : start + 4096, None]
        visible &= omni[start : start + 4096, None] | (
            (d * axes[start : start + 4096, None, :]).sum(axis=2) >= 0
        )
        if not visible.any(axis=1).all():
            return remaining
    return changed
```

visibility_probe.py

```
import math
import numpy as np

def orientation_mass(angles):
    angles = np.sort(angles, axis=1)
    counts = np.isfinite(angles).sum(axis=1)
    mass = np.zeros(len(angles))
    for n in np.unique(counts):
        ix = counts == n
        a = angles[ix, :n]
        if not n:
            mass[ix] = 1
            continue
        gaps = np.diff(np.concatenate([a, a[:, :1] + 2 * math.pi], axis=1), axis=1)
        mass[ix] = np.maximum(0, gaps.max(axis=1) - math.pi) / (2 * math.pi)
    return mass

def probabilities(poly, history, geometry, candidates):
    mid, rad, u, v, width = geometry
    pos = [
        (p, o)
        for k, p, o in history
        if k == 'measure' and o['kind'] in ('near', 'direction')
    ]
    neg = [p for k, p, o in history if k == 'measure' and o['kind'] == 'no_signal']
    if not pos:
        return [1.0] * len(candidates)
    polygon = np.asarray(poly)
    u = np.asarray(u)
    v = np.asarray(v)
    x = polygon @ u
    y = polygon @ v
    pts = (
        np.linspace(x.min(), x.max(), 97)[:, None, None] * u
        + np.linspace(y.min(), y.max(), 9)[None, :, None] * v
    ).reshape(-1, 2)
    keep = (pts * pts).sum(axis=1) <= 1770**2
    for a, b in zip(polygon, np.roll(polygon, -1, axis=0)):
        e = b - a
        d = pts - a
        keep &= e[0] * d[:, 1] - e[1] * d[:, 0] >= -1e-6
    for k, p, o in history:
        if k == 'clear' and o['kind'] != 'success':
            keep &= np.linalg.norm(pts - p, axis=1) > 20
    pts = pts[keep]
    if not len(pts):
        return [1.0] * len(candidates)
```

```
    pd = np.array([p for p, o in pos])[None, :, :] - pts[:, None, :]
    pa = np.arctan2(pd[:, :, 1], pd[:, :, 0]) % (2 * math.pi)
    minimum = np.maximum(1000, np.linalg.norm(pd, axis=2).max(axis=1))
    logweight = np.zeros(len(pts))
    for j, (p, o) in enumerate(pos):
        if o['kind'] == 'direction':
            error = (np.degrees(pa[:, j]) + 180 - o['bearing'] + 180) % 360 - 180
            logweight -= 0.5 * (error / 0.4) ** 2
    weight = np.exp(logweight - logweight.max()) * np.maximum(0, 1500 - minimum) / 500
    nd = (
        np.array(neg)[None, :, :] - pts[:, None, :]
        if neg
        else np.empty((len(pts), 0, 2))
    )
    nr = np.linalg.norm(nd, axis=2)
    na = np.arctan2(-nd[:, :, 1], -nd[:, :, 0]) % (2 * math.pi)
    qd = np.asarray(candidates)[None, :, :] - pts[:, None, :]
    qr = np.linalg.norm(qd, axis=2)
    qa = np.arctan2(qd[:, :, 1], qd[:, :, 0]) % (2 * math.pi)
    base = 0.0
    numerator = np.zeros(len(candidates))
    for fraction in (0.125, 0.375, 0.625, 0.875):
        receive = minimum + fraction * np.maximum(0, 1500 - minimum)
        inside = nr <= receive[:, None]
        angles = np.concatenate([pa, np.where(inside, na, np.inf)], axis=1)
        omni = (~inside.any(axis=1)) * 0.46
        base += weight @ (0.54 * orientation_mass(angles) + omni)
        for j in range(len(candidates)):
            new = np.concatenate([angles, qa[:, j, None]], axis=1)
            likelihood = (0.54 * orientation_mass(new) + omni) * (qr[:, j] <= receive)
            numerator[j] += weight @ likelihood
    return list(numerator / base) if base > 1e-20 else [1.0] * len(candidates)
```